IEEE P802.1AB-2016-Rev/D1.1
October 16, 2025
Revision of IEEE Std 802.1AB-2016 as amended by IEEE Std 802.1ABcu-2021
and IEEE Std 802.1ABdh-2021

Draft Standard for Local and Metropolitan Area Networks—

Station and Media Access Control Connectivity Discovery

Prepared by the

Maintenance Task Group of IEEE 802.1

Sponsor

LAN/MAN Standards Committee of the IEEE Computer Society

This and the following cover pages are not part of the draft. They provide revision and other information for IEEE 802.1 Working Group members and will be updated as convenient. **New participants: Please read these cover pages**, they contain information that should help you contribute effectively to this standards development project. The Introduction to the current draft should be useful to all readers.

The text proper of this draft begins with the Title page.

This document is a draft revision of IEEE Std 802.1AB-2016 as updated by published and draft corrigenda and amendments (if, and as, noted on the Title page), and may include (in addition to the main subject of the amendment, as per the PAR) the agreed or proposed resolution of Maintenance items and technical and editorial corrections, to the description of existing functionality(see below).

These cover pages provide an Introduction to the current draft, an introduction to Participation in 802.1 standards development, a summary of the PAR (Project Authorization Request) and CSD,for this project, and a general discussion of Draft development.

These cover pages will be replaced for SA Ballot by a briefer version providing information for that ballot, with space for commentary on, and hyperlinks to, changes that occur in SA Ballot.

Introduction to the current draft[1]

Draft P802.1AB-2016-Rev-d1.1 includes resolutions to all comments from the First Working Group ballot on P802.1AB-2016-Rev-d1.0. The changes include removal of the YANG module ieee802-types. Other corrections include corrections to the PICS for YANG and various editorial corrections.

Draft P802.1AB-2016-Rev-d1.0 includes resolution to all comments from the Task Group recirculation ballot run on P802.1AB-2016-Rev-d0.2. The changes include updates to clause 8 verb conjugations and revisions to the MIB and YANG modules.

Draft P802.1AB-2016-Rev-d0.2 includes resolutions to all comments from the first Task Force ballot run on draft P802.1AB-2016-Rev-d0.1. The changes include: updates to the introductory materials supporting the added YANG and XLLDP features; removal of all “shalls” from clause 8 as well as renumbering the XLLDP section; many changes have been made to eliminate the use of “MIB” replacing the term with the generic information repository (RP), these changes include changes to the names of state machine procedures prefixed with “mib” which are now prefixed by “rp”; additionally some state machine improvements were made.

Draft P802.1AB-2016-Rev-d0.1 rolls up the amendments IEEE Std 802.1ABcu-2021 and IEEE Std 802.1ABdh-2021 into IEEE Std 802.1AB-2016. Text added by the amendment IEEE Std 802.1ABcu-2021 outside of clause 12 (which is completely added by 802.1ABcu) is highlighted by the used of blue text. Text added by the amendment IEEE Std 802.1ABdh-2021 is highlighted by the used of magenta text. The titles of figures changed by these amendments is highlighted, however the actual figure is not highlighted. The highlighting of inserted text will be removed in a future revision of this draft.

Maintenance items and technical and editorial corrections

This draft includes the following proposed or agreed resolutions of maintenance items that were not addressed by rolled-up amendments.

— Maintenance item 0339 was implemented in clause 12

This draft does not include any technical corrections to the base standard beyond the project subject matter.

This draft does not include any editorial corrections to the base standard beyond the project subject matter.

YANG modules

The YANG modules specified by this standard are attached to the draft pdf as plain text (UTF-8) .yang files.

[1]The whole or parts of the introduction, possibly updated, to past drafts may be retained at the Editor’s discretion, with the most recent introduction first. The introduction to each draft may solicit input on specific subjects.

Sources

This draft, IEEE P802.1AB-2016-Rev/D1.1, has been prepared from a set of Framemaker files with conditional text that supports the production of an amendment draft and a preliminary rollup of that amendment draft into the text of the base standard.

For a description of the use of conditional text and other FrameMaker and IEEE Std 802.1Q Style considerations applicable to this draft see the EDITOR-PLEASE-READ-ME file in the FrameMaker books used to generate this draft.

Participation in 802.1 standards development

All participants in IEEE 802.1 activities should be aware of the Working Group Policies and Procedures, and their obligations under the IEEE Patent Policy, the IEEE Standards Association (SA) Copyright Policy, and the IEEE SA Participation Policy. For information on these policies see 1.ieee802.org/rules/ and the slides presented at the beginning of each of our Working Group and Task Group meeting.

The IEEE SA PAR (Project Authorization Request) and CSD (Criteria for Standards Development established by IEEE 802) are summarized in these cover pages and links are provided to the full text of both PAR and CSD. As part of the IEEE 802® process, the text of the PAR and CSD of each project is reviewed regularly to ensure their continued validity. A vote of “Approve” on this draft is also an affirmation by the voter that the PAR and CSD for this project are still valid.

Comments on this draft are encouraged. NOTE: All issues related to IEEE standards presentation style, formatting, spelling, etc. are routinely handled between the 802.1 Editor and the IEEE Staff Editors prior to publication, after balloting and the process of achieving agreement on the technical content of the standard is complete. Readers are urged to devote their valuable time and energy only to comments that materially affect either the technical content of the document or the clarity of that technical content. Comments should not simply state what is wrong, but also what might be done to fix the problem.

Full participation in the work of IEEE 802.1 requires attendance at IEEE 802 meetings. Information on 802.1 activities, working papers, and email distribution lists etc. can be found on the 802.1 Website:

http://ieee802.org/1/

Use of the email distribution list is not presently restricted to 802.1 members, and the working group has a policy of considering comments from all who are interested and willing to contribute to the development of the draft. Individuals not attending meetings have helped to identify sources of misunderstanding and ambiguity in past projects. The email lists exist primarily to allow the members of the working group to develop standards, and are not a general forum. All contributors to the work of 802.1 should familiarize themselves with the IEEE patent policy and anyone using the email distribution list will be assumed to have done so. Information can be found at http://standards.ieee.org/db/patents/

Comments on this draft may be sent to the 802.1 email exploder, to the Editors, or to the Chairs of the 802.1 Working Group and Time-Sensitive Networking (TSN) Task Group.

Paul Bottorff
Editor, IEEE Std 802.1AB
Email:paul.bottorff@hpe.com

Mark Hantel
Chair, 802.1 Maintenance Task Group
Email:mrhantel@ra.rockwell.com

Glenn Parsons
Chair, 802.1 Working Group
+1 514-379-9037
Email: glenn.parsons@ericsson.com

NOTE: Comments whose distribution is restricted in any way cannot be considered, and may not be acknowledged.

All participants in IEEE standards development have responsibilities under the IEEE patent policy and should familiarize themselves with that policy, see http://standards.ieee.org/about/sasb/patcom/materials.html

PAR (Project Authorization Request) and CSD

Extracts from the PAR, as approved by IEEE NesCom, Sep 26th, 2024:

https://development.standards.ieee.org/myproject-web/app#viewpar/15809/11716

This is a maintenance PAR so did not require an IEEE 802 CSD (Criteria for Standards Development).

The Scope and Purpose of the base standard were changed from IEEE Std 802.1AB-2016 as follows.

PAR Revised Scope of the Project:

The standard defines defines a protocol and management elements, suitable for advertising information to stations attached to the same IEEE 802 LAN, for the purpose of populating physical topology and device discovery management information databases. The protocol facilitates the identification of stations connected by IEEE 802 LANs/MANs, their points of interconnection, and access points for management protocols. This standard defines a protocol that a) Advertises connectivity and management information about the local station to adjacent stations on the same IEEE 802 LAN. b) Receives network management information from adjacent stations on the same IEEE 802 LAN. c) Operates with all IEEE 802 access protocols and network media. d) Establishes a network management information schema and object definitions that are suitable for storing connection information about adjacent stations. e) Provides compatibility with the IETF Physical Topology Management Information Base (MIB).

PAR Revised Purpose of the Project:

This standard specifies the necessary protocol and management elements to a) Facilitate multi-vendor inter-operability and the use of standard management tools to discover and make available physical topology information for network management. b) Make it possible for network management to discover certain configuration inconsistencies or malfunctions that can result in impaired communication at higher layers. c) Provide information to assist network management in making resource changes and/or reconfigurations that correct configuration inconsistencies or malfunctions identified in b) above. d) Provide a MIB to describe a network's physical topology and associated systems within that topology. e) Provide YANG configuration and operational models supporting Station and Media Access Control Connectivity Discovery.

PAR Revised Need for the Project:

This revision project is needed to incorporate approved amendments and corrigenda, to incorporate technical and editorial corrections to existing functionality, and to ensure that consistency is maintained in the consolidated text.

Draft development

During the early stages of draft development, 802.1 editors have a responsibility to attempt to craft technically coherent drafts from the resolutions of ballot comments and from the other discussions that take place in the working group meetings. Preparation of drafts often exposes inconsistencies in editor's instructions or exposes the need to make choices between approaches that were not fully apparent in the meeting. Choices and requests by the editors' for contributions on specific issues will be found in the editors' Introduction to the current draft and at appropriate points in the draft.

Any text with a Cyan background (as in this sentence) is temporary, with conditional tag 'Editor comment', inserted by the Editors to solicit comment, suggest a future change, or act simply as an aide memoire.Text can also highlighted to be draw it to the readers' attention, using conditional tag 'Editor highlight'. In both these case conditional tagging helps location, and eventual removal, of text or highlighting and can control whether or not it is displayed.

The ballot comments received on each draft, and the editors' proposed and final disposition of comments on working group drafts, are part of the audit trail of the development of the standard and are available, along with all the revisions of the draft on the 802.1 website (for address see above).

During the early stages of draft development the proposed text can be moved around a great deal, and even minor rearrangement can lead to a lot of 'change', not all of which is noteworthy from the point of the reviewer, so the use of automatic change bars is not very effective. In early drafts change bars may be omitted or applied manually, with a view to drawing the readers attention to the most significant areas of change. Readers interested in viewing every change are encouraged to use Adobe Acrobat to compare the document with their selected prior draft. Note that the FrameMaker change bar feature is useless when it comes to indicating changes to Figures.

This draft has been prepared from a set of Framemaker files with conditional text that supports the production of an amendment draft and a preliminary roll up of that amendment draft into the text of the base standard, i.e. IEEE Std 802.1Q as of the last Revision as amended by prior amendments (usually as of the close of their successful SA ballots) as noted on the Title Page and the first Cover Page. The editor may make preliminary roll ups available to check consistency with the base standard and cross-references to text that does not appear in this amendment. Roll ups may also be recorded as part of the approved P802.1Q Revision project.

For a description of the use of conditional text and other FrameMaker and IEEE Std 802.1Q Style considerations applicable to this draft see the EDITOR-PLEASE-READ-ME file in the FrameMaker books used to generate these drafts.

There are generally multiple amendments under development at any time, and while they will add or amend different clauses in the base standard, there are some clauses (notably Clauses 12, 48, and the PICS Annexes that all are likely to change). They need to be fully integrated before or during SA Ballot, and complete that ballot in serial order to avoid future problems.

Records of participants in the development of the standard are added after SA Ballot, as part of pre-publication editing by IEEE Staff.

IEEE P802.1AB-2016-Rev/D1.1

October 16, 2025

Revision of IEEE Std 802.1AB-2016 as amended by IEEE Std 802.1ABcu-2021 and IEEE Std 802.1ABdh-2021

Draft Standard for Local and Metropolitan Area Networks—

Station and Media Access Control Connectivity Discovery

Prepared by the

Maintenance Task Group of IEEE 802.1

Sponsor

LAN/MAN Standards Committee of the IEEE Computer Society

IEEE Standards Department
445 Hoes Lane
Piscataway, NJ 08854, USA

Abstract: A protocol and a set of managed objects that can be used for discovering the physical topology from adjacent stations in IEEE 802® LANs are defined in this document.

IEEE 802.1AB™, link layer discovery protocol, management information base, topology discovery, topology information

The Institute of Electrical and Electronics Engineers, Inc.
3 Park Avenue, New York, NY 10016-5997, USA

PDF: ISBN 978-X-XXX-XXX-X STDXXXXX
Print: ISBN 978-X-XXX-XXX-X STDPDXXXXX

Important Notices and Disclaimers Concerning IEEE Standards Documents

IEEE Standards documents are made available for use subject to important notices and legal disclaimers. These notices and disclaimers, or a reference to this page (https://standards.ieee.org/ipr/disclaimers.html), appear in all IEEE standards and may be found under the heading “Important Notices and Disclaimers Concerning IEEE Standards Documents.”

Notice and Disclaimer of Liability Concerning the Use of IEEE Standards Documents

IEEE Standards documents are developed within IEEE Societies and subcommittees of IEEE Standards Association (IEEE SA) Board of Governors. IEEE develops its standards through an accredited consensus development process, which brings together volunteers representing varied viewpoints and interests to achieve the final product. IEEE Standards are documents developed by volunteers with scientific, academic, and industry-based expertise in technical working groups. Volunteers involved in technical working groups are not necessarily members of IEEE or IEEE SA and participate without compensation from IEEE. While IEEE administers the process and establishes rules to promote fairness in the consensus development process, IEEE does not independently evaluate, test, or verify the accuracy of any of the information or the soundness of any judgments contained in its standards.

IEEE makes no warranties or representations concerning its standards, and expressly disclaims all warranties, express or implied, concerning all standards, including but not limited to the warranties of merchantability, fitness for a particular purpose and non-infringement. IEEE Standards documents do not guarantee safety, security, health, or environmental protection, or compliance with law, or guarantee against interference with or from other devices or networks. In addition, IEEE does not warrant or represent that the use of the material contained in its standards is free from patent infringement. IEEE standards documents are supplied “AS IS” and “WITH ALL FAULTS.”

Use of an IEEE standard is wholly voluntary. The existence of an IEEE Standard does not imply that there are no other ways to produce, test, measure, purchase, market, or provide other goods and services related to the scope of the IEEE standard. Furthermore, the viewpoint expressed at the time a standard is approved and issued is subject to change brought about through developments in the state of the art and comments received from users of the standard.

In publishing and making its standards available, IEEE is not suggesting or rendering professional or other services for, or on behalf of, any person or entity, nor is IEEE undertaking to perform any duty owed by any other person or entity to another. Any person utilizing any IEEE Standards document should rely upon their own independent judgment in the exercise of reasonable care in any given circumstances or, as appropriate, seek the advice of a competent professional in determining the appropriateness of a given IEEE standard.

IN NO EVENT SHALL IEEE BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO: THE NEED TO PROCURE SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE PUBLICATION, USE OF, OR RELIANCE UPON ANY STANDARD, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE AND REGARDLESS OF WHETHER SUCH DAMAGE WAS FORESEEABLE.

Translations

The IEEE consensus balloting process involves the review of documents in English only. In the event that an IEEE standard is translated, only the English language version published by IEEE is the approved IEEE standard.

Use by artificial intelligence systems

In no event shall material in any IEEE Standards documents be used for the purpose of creating, training, enhancing, developing, maintaining, or contributing to any artificial intelligence systems without the express, written consent of the IEEE SA in advance. “Artificial intelligence” refers to any software, application, orother system that uses artificial intelligence, machine learning, or similar technologies, to analyze, train, process, or generate content. Requests for consent can be submitted using the Contact Us form.[2]

Official statements

A statement, written or oral, that is not processed in accordance with the IEEE SA Standards Board Operations Manual is not, and shall not be considered or inferred to be, the official position of IEEE or any of its committees and shall not be considered to be or be relied upon as, a formal position of IEEE or IEEE SA. At lectures, symposia, seminars, or educational courses, an individual presenting information on IEEE standards shall make it clear that the presenter’s views should be considered the personal views of that individual rather than the formal position of IEEE, IEEE SA, the Standards Committee, or the Working Group. Statements made by volunteers may not represent the formal position of their employer(s) or affiliation(s). News releases about IEEE standards issued by entities other than IEEE SA should be considered the view of the entity issuing the release rather than the formal position of IEEE or IEEE SA.

Comments on standards

Comments for revision of IEEE Standards documents are welcome from any interested party, regardless of membership affiliation with IEEE or IEEE SA. However, **IEEE does not provide interpretations, consulting information, or advice pertaining to IEEE Standards documents**.

Suggestions for changes in documents should be in the form of a proposed change of text, together with appropriate supporting comments. Since IEEE standards represent a consensus of concerned interests, it is important that any responses to comments and questions also receive the concurrence of a balance of interests. For this reason, IEEE and the members of its Societies and subcommittees of the IEEE SA Board of Governors are not able to provide an instant response to comments or questions except in those cases where the matter has previously been addressed. For the same reason, IEEE does not respond to interpretation requests. Any person who would like to participate in evaluating comments or revisions to an IEEE standard is welcome to join the relevant IEEE SA working group. You can indicate interest in a working group using the Interests tab in the Manage Profile & Interests area of the IEEE SA myProject system.[3] An IEEE Account is needed to access the application.

Comments on standards should be submitted using the Contact Us form.[2]

Laws and regulations

Users of IEEE Standards documents should consult all applicable laws and regulations. Compliance with the provisions of any IEEE Standards document does not constitute compliance to any applicable regulatory requirements. Implementers of the standard are responsible for observing or referring to the applicable regulatory requirements. IEEE does not, by the publication of its standards, intend to urge action that is not in compliance with applicable laws, and these documents may not be construed as doing so.

Data privacy

Users of IEEE Standards documents should evaluate the standards for considerations of data privacy and data ownership in the context of assessing and using the standards in compliance with applicable laws and regulations.

[2] Available at: https://standards.ieee.org/content/ieee-standards/en/about/contact/.
[3] Available at: https://development.standards.ieee.org/myproject-web/public/view.html#landing.

Copyrights

Photocopies

Updating of IEEE Standards documents

Users of IEEE Standards documents should be aware that these documents may be superseded at any time by the issuance of new editions or may be amended from time to time through the issuance of amendments, corrigenda, or errata. An official IEEE document at any point in time consists of the current edition of the document together with any amendments, corrigenda, or errata then in effect.

Every IEEE standard is subjected to review at least every 10 years. When a document is more than 10 years old and has not undergone a revision process, it is reasonable to conclude that its contents, although still of some value, do not wholly reflect the present state of the art. Users are cautioned to check to determine that they have the latest edition of any IEEE standard.

In order to determine whether a given document is the current edition and whether it has been amended through the issuance of amendments, corrigenda, or errata, visit IEEE Xplore or contact IEEE.[4] For more information about the IEEE SA or IEEE's standards development process, visit the IEEE SA Website.

Errata

Errata, if any, for all IEEE standards can be accessed on the IEEE SA Website.[5] Search for standard number and year of approval to access the web page of the published standard. Errata links are located under the Additional Resources Details section. Errata are also available in IEEE Xplore. Users are encouraged to periodically check for errata.

Patents

IEEE Standards are developed in compliance with the IEEE SA Patent Policy.[6]

Attention is called to the possibility that implementation of this standard may require use of subject matter covered by patent rights. By publication of this standard, no position is taken by the IEEE with respect to the existence or validity of any patent rights in connection therewith. If a patent holder or patent applicant has filed a statement of assurance via an Accepted Letter of Assurance, then the statement is listed on the IEEE SA Website at https://standards.ieee.org/about/sasb/patcom/patents.html. Letters of Assurance may indicate whether the Submitter is willing or unwilling to grant licenses under patent rights without compensation or under reasonable rates, with reasonable terms and conditions that are demonstrably free of any unfair discrimination to applicants desiring to obtain such licenses.

[4] Available at: https://ieeexplore.ieee.org/browse/standards/collection/ieee.
[5] Available at: https://standards.ieee.org/standard/index.html.
[6] Available at: https://standards.ieee.org/about/sasb/patcom/materials.html.

Essential Patent Claims may exist for which a Letter of Assurance has not been received. The IEEE is not responsible for identifying Essential Patent Claims for which a license may be required, for conducting inquiries into the legal validity or scope of Patents Claims, or determining whether any licensing terms or conditions provided in connection with submission of a Letter of Assurance, if any, or in any licensing agreements are reasonable or non-discriminatory. Users of this standard are expressly advised that determination of the validity of any patent rights, and the risk of infringement of such rights, is entirely their own responsibility. Further information may be obtained from the IEEE Standards Association.

IMPORTANT NOTICE

IEEE Standards do not guarantee or ensure safety, security, health, or environmental protection, or ensure against interference with or from other devices or networks. Technologies, application of technologies, and recommended procedures in various industries evolve over time. The IEEE standards development process allows participants to review developments in industries, technologies, and practices, and to determine what, if any, updates should be made to the IEEE standard. During this evolution, the technologies and recommendations in IEEE standards may be implemented in ways not foreseen during the standards development. IEEE Standards development activities consider research and information presented to the standards development group in developing any safety recommendations. Other information about safety practices, changes in technology or technology implementation, or impact by peripheral systems also may be pertinent to safety considerations during implementation of the standard. Implementers and users of IEEE Standards documents are responsible for determining and complying with all appropriate safety, security, environmental, health, data privacy, and interference protection practices and all applicable laws and regulations.

Participants

<<The following lists will be updated in the usual way prior to publication>>

At the time this standard was submitted to the IEEE-SA Standards Board for approval, the IEEE 802.1 Working Group had the following membership:

Glenn Parsons, *Chair*
Jessy V. Rouyer, *Vice Chair and Editor*
Mark Hantel, *Maintenance Task Group Chair*
Paul Bottorff, *Editor*

<<TBA>>

The following members of the individual balloting committee voted on this standard. Balloters may have voted for approval, disapproval, or abstention.

<<TBA>>

When the IEEE-SA Standards Board approved this standard on XX Month 20xx, it had the following membership:

<<TBA>>

<<TBA>>

*Member Emeritus

Historical participants

Included is a historical list of participants in the IEEE 802.1 Working Group who have dedicated their valuable time, energy, and knowledge to the creation of this material.

The following individuals participated in the 2005 publication of this standard.

Tony Jeffree, *Chair*
Paul Congdon, *Vice Chair*
Bill Lane, *Technical Editor*
Mick Seaman, *Interworking Task Group Chair*

Les Bell
Paul Bottorff
Jim Burns
Marco Carugi
Dirceu Cavendish
Arjan de Heer
Anush Elangovan
Hesham Elbakoury
David Elie-Dit-Cosaque
Norm Finn
David Frattura
Gerard Goubert
Steve Haddock
Ran Ish-Shalom
Atsushi Iwata
Neil Jarvis
Manu Kaycee
Hal Keen
Roger Lapuh
Loren Larsen
Joe Lawrence
Yannick Le Goff
Marcus Leech
Mahalingam Mani
Dinesh Mohan
Bob Moskowitz
Don O'Connor
Don Pannell
Glenn Parsons
Ken Patton
Frank Reichstein
John Roese
Allyn Romanow
Dan Romascanu
Jessy V. Rouyer
Ali Sajassi
Dolors Sala
Muneyoshi Suzuki
Jonathan Thatcher
Michel Thorsen
Dennis Volpano
Karl Weber
Ludwig Winkel
Michael D. Wright

The following individuals participated in the 2009 publication of this standard.

Tony Jeffree, *Chair and Editor*
Paul Congdon, *Vice Chair*
Stephen Haddock, *Chair, Interworking Task Group*

Osama Aboul-Magd
Zehavit Alon
Caitlin Bestler
Jan Bialkowski
Rob Boatright
Jean-Michel Bonnamy
Paul Bottorff
Rudolf Brandner
Craig W. Carlson
Weiying Cheng
Rao Cherukuri
Paul Congdon
Diego Crupnicoff
Claudio Desanti
Zhemin Ding
Linda Dunbar
Hesham M. Elbakoury
David Elie-Dit-Cosaque
János Farkas
Donald Fedyk
Norman Finn
Robert Frazier
John Fuller
Geoffrey Garner
Anoop Ghanwani
Franz Goetz
Yannick Le Goff
Eric Gray
Karanvir Grewal
Craig Gunther
Mitch Gusat
Asif Hazarika
Charles Hudson
Romain Insler
Michael Johas Teener
Abhay Karandikar
Prakash Kashyap
Hal Keen
Keti Kilcrease
Yongbum Kim
Philippe Klein
Mike Ko
Vinod Kumar
Bruce Kwan
Kari Laihonen
Michael Lerer
Gael Mace
Ben Mack-Cran
David Martin
Riccardo Martinotti
Alan McGuire
James McIntosh
Menucher Menuchery
John Messenger
Matthew Mora
Eric Multanen
Kevin Nolish
Hiroshi Ohta
David Olsen
Donald Pannell
Glenn Parsons
Joseph Pelissier
David Peterson
Hayim Porat
Max Pritikin
Ananda Rajagopal
Karen T. Randall
Guenter Roeck
Josef Roese
Derek J. Rohde
Dan Romascanu
Moran Roth
Jessy V. Rouyer
Jonathan Sadler
Ali Sajassi
Joseph Salowey
Panagiotis Saltsidis
Satish Sathe
John Sauer
Michael Seaman
Koichiro Seto
Himanshu Shah
Nurit Sprecher
Kevin B. Stanton
Robert A. Sultan
Muneyoshi Suzuki
George Swallow
Attila Takacs
Patricia Thaler
Oliver Thorp
Manoj Wadekar
Yuehua Wei
Brian Weis
Bert Wijnen
Michael D. Wright
Chien-Hsien Wu
Ken Young
Glen Zorn

The following individuals participated in the development of Corrigendum 1 to the 2009 publication of this standard.

Tony Jeffree, *Chair and Editor*
Glenn Parsons, *Vice-Chair and Maintenance Task Group Chair*

Ting Ao
Kenneth Boehlke
Christian Boiger
Brad Booth
Paul Bottorff
Jeffrey Catlin
Xin Chang
Weiying Cheng
Diego Crupnicoff
Rodney Cummings
Donald Eastlake, III
János Farkas
Donald Fedyk
Norman Finn
Andre Fredette
Geoffrey Garner
Anoop Ghanwani
Franz Goetz
Mark Gravel
Eric Gray
Robert Grow
Yingjie Gu
Craig Gunther
Stephen Haddock
Hitoshi Hayakawa
Mirko Jakovljevic
Markus Jochim
Michael Johas Teener
Girault Jones
Daya Kamath
Hal Keen
Yongbum Kim
Philippe Klein
Oliver Kleineberg
Jeff Lynch
Ben Mack-Crane
John Messenger
Eric Multanen
Henry Muyshondt
David Olsen
Donald Pannell
Karen Randall
Josef Roese
Dan Romascanu
Jessy Rouyer
Ali Sajassi
Panagiotis Saltsidis
Rick Schell
Michael Seaman
Koichiro Seto
Daniel Sexton
Rakesh Sharma
Johannes Specht
Kevin Stanton
Wilfried Steiner
Patricia Thaler
Jeremy Touve
Albert Tretter
Maarten Vissers
Yuehua Wei
Min Xiao

The following individuals participated in the development of Corrigendum 2 to the 2009 publication of this standard.

Glenn Parsons, *Working Group Chair*
John Messenger, *Vice-Chair and Maintenance Task Group Chair*
Tony Jeffree, *Editor*

Ting Ao
Christian Boiger
Paul Bottorff
David Chen
Feng Chen
Weiying Cheng
Diego Crupnicoff
Rodney Cummings
Patrick Diamond
Aboubacar Kader Diarra
János Farkas
Norman Finn
Geoffrey Garner
Anoop Ghanwani
Mark Gravel
Eric W. Gray
Craig Gunther
Stephen Haddock
Hitoshi Hayakawa
Jeremy Hitt
Rahil Hussain
Michael Johas Teener
Peter Jones
Hal Keen
Marcel Kiessling
Yongbum Kim
Philippe Klein
Jouni Korhonen
Jeff Lynch
Ben Mack-Crane
Christophe Mangin
James McIntosh
Eric Multanen
Donald Pannell
Karen Randall
Maximilian Riegel
Dan Romascanu
Jessy V. Rouyer
Panagiotis Saltsidis
Behcet Sarikaya
Michael Seaman
Daniel Sexton
Johannes Specht
Kevin B. Stanton
Wilfried Steiner
Vahid Tabatabaee
Patricia Thaler
Jeremy Touve
Karl Weber
Yuehua Wei
Brian Weis
Jordon Woods
Juan-Carlos Zuniga

The following individuals participated in the 2016 publication of this standard.

Glenn Parsons, *Chair*
John Messenger, *Vice Chair and Maintenance Task Group Chair*
Tony Jeffree, *Editor*

Christian Boiger
Paul Bottorff
David Chen
Feng Chen
Weiying Cheng
Rodney Cummings
János Farkas
Norman Finn
Geoffrey Garner
Eric Gray
Craig Gunther
Stephen Haddock
Mark Hantel
Marc Holness
Michael Johas Teener
Hal Keen
Stephan Kehrer
Marcel Kiessling
Philippe Klein
Jouni Korhonen
Yizhou Li
Christophe Mangin
Tom McBeath
James McIntosh
Hiroki Nakano
Bob Noseworthy
Donald R. Pannell
Walter Pienciak
Karen Randall
Maximilian Riegel
Dan Romascanu
Jessy Rouyer
Panagiotis Saltsidis
Michael Seaman
Daniel Sexton
Johannes Specht
Wilfried Steiner
Patricia Thaler
David Thornburg
Jeremy Touve
Paul Unbehagen
Karl Weber
Brian Weis
Jordon Woods
Helge Zinner
Juan Carlos Zuniga

The following members of the individual balloting committee voted on this standard. Balloters may have voted for approval, disapproval, or abstention.

Thomas Alexander
Butch Anton
Lee Armstrong
Stefan Aust
Christian Boiger
Nancy Bravin
William Byrd
Juan Carreon
Rodney Cummings
Douglas Dorr
János Farkas
Yukihiro Fujimoto
David Gregson
Randall Groves
Craig Gunther
Stephen Haddock
Marek Hajduczenia
Jerome Henry
Marco Hernandez
Guido Hiertz
Werner Hoelzl
Noriyuki Ikeuchi
Sergiu Iordanescu
Atsushi Ito
Raj Jain
Tony Jeffree
Michael Johas Teener
Adri Jovin
Shinkyo Kaku
Piotr Karocki
Stuart Kerry
Yongbum Kim
Paul Lambert
Robert Landman
David Lewis
Arthur H. Light
William Lumpkins
Michael Lynch
Elvis Maculuba
Jonathon McLendon
Richard Mellitz
John Messenger
Charles Moorwood
Michael Newman
Nick S. A. Nikjoo
Satoshi Obara
Alon Regev
Maximilian Riegel
Robert Robinson
Benjamin Rolfe
Dan Romascanu
Jessy Rouyer
John Santhoff
Bartien Sayogo
Michael Seaman
Thomas Starai
Eugene Stoudenmire
Rene Struik
Walter Struppler
Michael Swearingen
Patricia Thaler
Mark-Rene Uchida
Lorenzo Vangelista
Dmitri Varsanofiev
George Vlantis
Khurram Waheed
Karl Weber
Hung-Yu Wei
Natalie Wienckowski
Andreas Wolf
Oren Yuen
Zhen Zhou

Introduction

This introduction is not part of IEEE Std 802.1AB™-2016, IEEE Standard for Local and metropolitan area networks—Station and Media Access Control Connectivity Discovery.

This revision of IEEE Std 802.1AB incorporates the following amendments into the base text of the 2016 revision:

— IEEE Std 802.1ABcu-2021
— IEEE Std 802.1ABdh-2021

This standard contains state-of-the-art material. The area covered by this standard is undergoing evolution. Revisions are anticipated within the next few years to clarify existing material, to correct possible errors, and to incorporate new related material. Information on the current revision state of this and other IEEE 802 standards may be obtained from

Secretary, IEEE-SA Standards Board
445 Hoes Lane
Piscataway, NJ 08854-4141
USA

Contents

List of figures

List of tables

IEEE Standard for
Local and metropolitan area networks—

Station and Media Access Control Connectivity Discovery

IMPORTANT NOTICE: IEEE Standards documents are not intended to ensure safety, security, health, or environmental protection, or ensure against interference with or from other devices or networks. Implementers of IEEE Standards documents are responsible for determining and complying with all appropriate safety, security, environmental, health, and interference protection practices and all applicable laws and regulations.

1. Overview

The Link Layer Discovery Protocol (LLDP) specified in this standard allows stations attached to an IEEE 802® LAN to advertise, to other stations attached to the same IEEE 802 LAN, the major capabilities provided by the system incorporating that station, the management address or addresses of the entity or entities that provide management of those capabilities, and the identification of the station's point of attachment to the IEEE 802 LAN required by those management entity or entities. Basic LLDP provides a mechanism for advertising information that can fit within a single Protocol Data Unit (PDU), while Extended LLDP (XLLDP) allows advertisement of information spanning multiple PDUs.

The information distributed via this protocol is stored in an information repository (RP), which can support both a standard Management Information Base (MIB) for SNMP and standard data models defined in YANG. This enables the information to be accessed by a Network Management System (NMS) through compatible management protocols, such as SNMP or NETCONF.

1.1 Scope

The scope of this standard is to define a protocol and management elements, suitable for advertising information to stations attached to the same IEEE 802 LAN, for the purpose of populating physical topology and device discovery management information databases. The protocol facilitates the identification of stations connected by IEEE 802 LANs/MANs, their points of interconnection, and access points for management protocols.

This standard defines a protocol that

a) Advertises connectivity and management information about the local station to adjacent stations on the same IEEE 802 LAN.
b) Receives network management information from adjacent stations on the same IEEE 802 LAN.
c) Operates with all IEEE 802 access protocols and network media.
d) Establishes a network management information schema and object definitions that are suitable for storing connection information about adjacent stations.
e) Can provide compatibility with the IETF PTOPO MIB (IETF RFC 2922 [B8]). [2]
f) Can provide compatibility with the IETF Data Model for Network Topologies (RFC 8345 [B15]).[2]

1.2 Purpose

This standard specifies the necessary protocol and management elements to

a) Facilitate multi-vendor inter-operability and the use of standard management tools to discover and make available physical topology information for network management.
b) Make it possible for network management to discover certain configuration inconsistencies or malfunctions that can result in impaired communication at higher layers.
c) Provide information to assist network management in making resource changes and/or re-configurations that correct configuration inconsistencies or malfunctions identified in b) above.
d) Provide IETF MIB (IETF RFC 2922 [B8]) to describe a network's physical topology and associated systems within that topology.
e) Provide YANG configuration and operational models supporting Station and Media Access Control Connectivity Discovery.

[2] The numbers in brackets correspond to those in the bibliography in Annex D.

2. Normative references

The following referenced documents are indispensable for the application of this document (i.e., they must be understood and used, so each referenced document is cited in the text and its relationship to this document is explained). For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments or corrigenda) applies.

IEEE Std 802®, IEEE Standard for Local and Metropolitan Area Networks: Overview and Architecture. [3], [4]

IEEE Std 802.1AC™, IEEE Standard for Local and metropolitan area networks—Media Access Control (MAC) Service Definition.

IEEE Std 802.1AE™, IEEE Standard for Local and Metropolitan Area Networks—Media Access Control (MAC) Security.

IEEE Std 802.1AX™, IEEE Standard for Local and Metropolitan Area Networks—Link Aggregation.

IEEE Std 802.1Q™, IEEE Standards for Local and Metropolitan Area Networks: Bridges and Bridged Networks.

IEEE Std 802.1X™, IEEE Standard for Local and Metropolitan Area Networks—Port-Based Network Access Control.

IEEE Std 802.3™, IEEE Standard for Information technology—Telecommunications and information exchange between systems—Local and metropolitan area networks—Specific requirements—Part 3: Carrier Sense Multiple Access with Collision Detection (CSMA/CD) Access Method and Physical Layer Specifications.

IETF RFC 2863, The Interfaces Group MIB, June 2000. [5]

IETF RFC 3046, DHCP Relay Agent Information Option, January 2001.

IETF RFC 3232, Assigned Numbers: RFC 1700 is Replaced by an On-line Database, January 2002. [6]

IETF RFC 3410, Introduction and Applicability Statements for Internet Standard Management Framework, December 2002.

IETF RFC 3417, Transport Mappings for the Simple Network Management Protocol (SNMP), December 2002.

IETF RFC 3418, Management Information Base (MIB) for the Simple Network Management Protocol (SNMP), December 2002.

IETF RFC 3629, UTF-8, a transformation format of ISO 10646, November 2003.

IETF RFC 4502, Remote Network Monitoring Management Information Base Version 2, May 2006.

[3] IEEE and 802 are registered trademarks in the U.S. Patent & Trademark Office, owned by The Institute of Electrical and Electronics Engineers, Incorporated.

[4] IEEE publications are available from The Institute of Electrical and Electronics Engineers (http://standards.ieee.org/).

[5] IETF documents (i.e., RFCs) are available for download at http://www.rfc-archive.org/.

[6] The IETF RFC 3232 ianaAddressFamilyNumbers on-line database module is accessible at (http://www.iana.org/).

IETF RFC 4639, Cable Device Management Information Base for Data-Over-Cable Service Interface Specification (DOCSIS) Compliant Cable Modems and Cable Modem Termination Systems, December 2006.

IETF RFC 4789, Simple Network Management Protocol (SNMP) over IEEE 802 Networks, November 2006.

IETF RFC 6241, Network Configuration Protocol (NETCONF), June 2011. [7]

IETF RFC 6933, Entity MIB (Version 4), May 2013.IETF RFC 6991, Common YANG Data Types, July 2013.

IETF RFC 6991, Common YANG Data Types, July 2013.

IETF RFC 7950, The YANG 1.1 Data Modeling Language, August 2016.

IETF RFC 8343, A YANG Data Model for Interface Management, March 2018.

IETF RFC 8349, A YANG Data Model for Routing Management (NMDA Version), March 2018.

ISO/IEC 8824-1 [ITU-T Rec. X.680 (2002)], Abstract Syntax Notation One (ASN.1): Specification of Basic Notation. [8]

[7] IETF RFCs are available from the Internet Engineering Task Force (https://www.rfc-archive.org/).

[8] ASN.1 standards are available at http://asn1.elibel.tm.fr/en/standards/index.htm#asn1.

3. Definitions and numerical representation

3.1 Definitions

For the purposes of this document, the following terms and definitions apply. The *IEEE Standards Dictionary Online* should be consulted for terms not defined in this clause. [9]

alpha-numeric information: Information that is encoded using the 8-bit Universal Character Set (UCS)/Unicode Transformation Format (UTF-8) octet sequence [IETF RFC 3629].

chassis: A physical component incorporating one or more IEEE 802® LAN stations and their associated application functionality.

chassis identifier: An administratively assigned name that identifies the particular chassis within the context of an administrative domain that comprises one or more networks.

Extended Link Layer Discovery Protocol (XLLDP): An LLDP extension used to retrieve the frames beyond the Normal LLDPDU of a multi-frame LLDP database.

Extension LLDPDU (XPDU): An LLDPDU containing an Extension Identifier TLV and not containing a Time to Live TLV or Extension Request TLV.

Extension Request LLDPDU (XREQ): An LLDPDU containing an Extension Request TLV and not containing a Time to Live TLV or Extension Identifier TLV.

IEEE 802® LAN: Local area network (LAN) technologies that provide a media access control (MAC) Service equivalent to the MAC Service defined in IEEE Std 802.1AC.

IEEE 802® LAN station: An IEEE 802-compatible entity that incorporates all the necessary mechanisms to participate in media access control of an IEEE 802 LAN, and that is at least capable of providing the MAC service plus the mandatory capabilities of the LLC.

NOTE 1—For example, routers and host computers are stations in a bridged network. Bridges, in addition to their relay function, also include station capabilities. [10]

Link Layer Discovery Protocol (LLDP): A media-independent protocol capable of running on all IEEE 802® LAN stations and to allow an LLDP agent to learn the connectivity and management information from adjacent stations.

LLDP agent: The protocol entity that implements LLDP for a particular MSAP associated with a Port.

LLDP Information Repository (RP): A management information repository which stores the information exchanged by LLDP in TLVs and can be modeled by SNMP MIBs, YANG, or other data models.

MAC service access point (MSAP): The access point for MAC services provided to the LLC sublayer.

MSAP identifier: The identifier of a MAC service access point.

NOTE 2—In this standard, the concatenation of the chassis identifier and the port identifier is used by LLDP as an MSAP identifier, to identify the port associated with an IEEE 802® LAN station.

[9] *IEEE Standards Dictionary Online* is available at: http://dictionary.ieee.org.
[10] Notes in text, tables, and figures are given for information only and do not contain requirements needed to implement the standard.

management entity: The protocol entity that implements a particular network management protocol and that provides access support to a information repository associated with the protocol and implemented on a host chassis.

Management Information Base (MIB): The instantiation of all MIB modules in a managed entity (e.g., system or device).

Management Information Base module (MIB module): The specification or schema for a data base that can be populated with the information required to support a network management information system.

Manifest LLDPDU: A Normal LLDPDU containing a Manifest TLV.

network: An interconnected group of systems, each comprising one or more IEEE 802® LAN stations.

Network Management System (NMS): A management system that is capable of utilizing the information in a information repository.

Normal LLDPDU: An LLDPDU containing a Time To Live TLV and not containing an Extension Request TLV or an Extension Identifier TLV.

object identifier (OID): An identifier used to name an object. Structurally, an OID consists of a node in a hierarchically-assigned namespace, formally defined in ISO/IEC 8824-1, Abstract Syntax Notation 1 (ASN.1). OIDs are used in this standard to identify MIB modules and the objects they contain.

physical network topology: The identification of systems, of IEEE 802® LAN stations that compose each system, and of the IEEE 802 LAN stations that attach to the same IEEE 802 LAN.

port: The entity in a chassis/system to support an MSAP. A port incorporates one and only one MSAP and identifies the collection of manageable entities that provide the MAC Service at the MSAP.

port identifier: An administratively assigned name that identifies the particular port within the context of a system, where the identification is convenient, local to the system, and persistent for the system's use and management (whereas the MAC address that globally identifies the MSAP can not be).

Scope MAC Address: The MAC address used as the destination address in Normal LLDPDUs, which determines the limits of propagation in the network.

shutdown LLDPDU: A Normal LLDPDU with a TTL value of zero.

system: A managed collection of hardware and software components incorporating one or more chassis, stations and ports.

station only: A non-forwarding IEEE 802® LAN station such as a user workstation, network file server, or print server.

type, length, value (TLV): A short, variable length encoding of an information element consisting of sequential type, length, and value fields where the type field identifies the type of information, the length field indicates the length of the information field in octets, and the value field contains the information, itself.

YANG: IETF defined data modeling language, published as IETF RFC 7950.

YANG model: One or more YANG modules used to configure and monitor the managed element or system.

YANG module: The description of the data model used to configure and monitor the managed element or system. A YANG module defines a hierarchy of nodes that can be used for NETCONF-based (see IETF RFC 7803 [B12]) and RESTCONF-based (see IETF RFC 8040 [B13]) operations.

3.2 Numerical representation

Decimal, hexadecimal, and binary numbers are used within this document. For clarity, decimal numbers are generally used to represent counts, hexadecimal numbers are used to represent addresses, and binary numbers are used to describe bit patterns within binary fields.

Decimal numbers are represented in their usual 0, 1, 2, … format. Hexadecimal numbers are represented by a string of one or more hexadecimal (0–9, A–F) digits followed by the subscript 16, except in C-code contexts, where they are written as `0x123EF2`, etc. Binary numbers are represented by a string of one or more binary (0,1) digits, followed by the subscript 2. Thus the decimal number “26” may also be represented as “$1A_{16}$” or “11010_2”.

MAC addresses, EtherType values, and OUI/EUI values are represented as strings of 8-bit hexadecimal numbers separated by hyphens and without a subscript, as, for example, “01-80-C2-00-00-15” or “AA-55-11”, using the hexadecimal representation defined in IEEE Std 802.

4. Acronyms and abbreviations

AP	Access Point
BSSID	basic service set identification
DA	Destination Address
DS	Distribution System
EPD	Ethertype Protocol Discrimination
EUI	Extended Unique Identifier
FCS	Frame Check Sequence
IANA	Internet Assigned Numbers Authority
ID	Identifier
IETF	Internet Engineering Task Force
KaY	Media access control Security Key Agreement Entity
LLC	logical link control (sublayer)
LLDP	Link Layer Discovery Protocol
LLDPDU	Link Layer Discovery Protocol data unit
LSAP	link service access point
MAC	media access control (sublayer)
MAU	medium attachment unit
MDI	media dependent interface
MIB	Management Information Base (module)
MSAP	Media access control service access point
MSDU	Media access control service data unit
NMS	Network Management System
OID	object identifier
OUI	organizationally unique identifier
PD	Powered Device
PHY	physical (sublayer)

PMD	physical media dependent (sublayer)
PSE	Power Sourcing Equipment
PVID	port virtual local area network identifier
PPVID	port and protocol virtual local area network identifier
PTOPO	the name of the Internet Engineering Task Force physical topology Management Information Base
RFC	Request for comments
RP	LLDP Information Repository
SA	Source Address
SecY	Media access control Security Entity
SNAP	Subnetwork Access Protocol
SNMP	Simple Network Management Protocol
STP	Spanning Tree Protocol
TPMR	Two-port Media access control relay
TLV	type, length, value
TTL	time to live (value)
UCS	Universal Character Set
UML®	Unified Modeling Language™ [11]
UTF-8	8-bit Universal Character Set/Unicode Transformation Format
VID	virtual local area network identifier
XLLDP	Extended Link Layer Discovery Protocol
XPDU	Extension LLDPDU
XREQ	Extension Request LLDPDU

[11] UML® is a registered trademark of the Object Management Group, Inc., and Unified Modeling Language™ is a trademark of the Object Management Group, Inc.

5. Conformance

This clause specifies the mandatory and optional capabilities provided by conformant implementations of this standard.

5.1 Terminology

For consistency with IEEE and existing IEEE 802.1™ standards terminology, requirements placed upon conformant implementations of this standard are expressed using the following terminology:

a) *Shall* is used for mandatory requirements.
b) *May* is used to describe implementation or administrative choices ("may" means "is permitted to," and hence, "may" and "may not" mean precisely the same thing).
c) *Should* is used for recommended choices (the behaviors described by "should" and "should not" are both permissible but not equally desirable choices).

The PICS proforma (see Annex A) reflects the occurrences of the words *shall*, *may*, and *should* within the standard.

The standard avoids needless repetition and apparent duplication of its formal requirements by using is, is not, are, and are not for definitions and the logical consequences of conformant behavior. Behavior that is permitted but is neither always required nor directly controlled by an implementor or administrator, or whose conformance requirement is detailed elsewhere, is described by can. Behavior that never occurs in a conformant implementation or system of conformant implementations is described by can not. The word *allow* is used as a replacement for the phrase "support the ability for," and the word *capability* means "is able to, or can be configured to."

5.2 Protocol Implementation Conformance Statement (PICS)

The supplier of an implementation that is claimed to conform to this standard shall complete a copy of the PICS proforma provided in Annex A and shall provide the information necessary to identify both the supplier and the implementation.

5.3 Required capabilities

A system for which conformance to this standard is claimed shall, for all ports for which support is claimed, include the following capabilities:

a) If port access is controlled by IEEE Std 802.1X, [12] LLDP exchanges shall be supported through the controlled port, as specified in Clause 6 of IEEE Std 802.1X-2020.
b) The destination and source addressing shall conform to 7.1 and 7.2.
c) EtherType encapsulation shall conform to 7.3.
d) LLDPDU recognition and reception shall conform to the operation of the receive state machine as defined in 9.2.10.
e) LLDPDU encapsulation shall conform to the specifications in 8.2.
f) The basic TLV format capability shall be implemented as defined in 8.4.
g) The basic management set of TLVs shall be implemented as defined in 8.5.
h) The Organizationally Specific TLV format capability shall be implemented as defined in 8.7.

[12] Information on references can be found in RFC 3629.

i) The protocol shall conform to the specifications for all Clause 9 subclauses indicated in Table 9-1 for the particular operating mode (transmit only, receive only, or transmit and receive) being implemented.

j) If receipt of LLDPDUs is supported, for every set of TLVs (the basic management set, extended LLDP set, and any organizationally specific sets) supported, support shall be implemented for receipt of every TLV defined in the set.

k) If transmission of LLDPDUs is supported, for every set of TLVs (the basic management set, extended LLDP set, and any organizationally specific sets) supported, support shall be implemented for transmission of every TLV defined in the set.

l) If transmission of LLDPDUs is supported, for every set of TLVs (the basic management set, extended LLDP set, and any organizationally specific sets) supported, a capability shall be implemented for users to determine which optional TLVs are transmitted in any particular LLDPDU.

m) If SNMP is supported, then

1) The system shall conform to the LLDP management specifications in Clause 11 and shall implement the sections of version 2 of the basic LLDP MIB indicated in Table 11-1 for the operating mode being implemented.

2) The system shall support at least one of the transport mappings defined by IETF RFC 3417 or IETF RFC 4789.

n) If neither SNMP nor YANG are supported, the system shall provide storage and retrieval capability equivalent to the functionality specified in 10.1 for the operating mode being implemented.

o) If YANG is supported, then the system shall conform to the LLDP management specifications in 12 and shall implement the sections of the LLDP YANG module for the operating mode being implemented.

p) If the Extended Link Layer Discovery Protocol is supported, then

1) The extended LLDP set of TLVs shall be implemented as defined in 8.6.

2) The Extension Request and Extension LLDPDU formats (8.2) and addressing (7.1.2) shall be implemented.

3) The extended receive state machine (Figure 9-3) shall be implemented.

5.4 Optional capabilities

A system for which conformance to this standard is claimed may, for each port for which support is claimed, support the following capabilities:

a) If port access is controlled by IEEE Std 802.1X, LLDP exchanges may be supported through the uncontrolled port, as specified in Clause 6 of IEEE Std 802.1X-2020.

6. Principles of operation

LLDP is a link layer protocol that allows an IEEE 802 LAN station to advertise the capabilities and current status of the system associated with an MSAP. The MSAP provides the MAC service to an LLC Entity, and that LLC Entity provides an LSAP to an LLDP agent that transmits and receives information to and from the LLDP agents of other stations attached to the same LAN. The information distributed and received in each LLDPDU is stored in one or more Management Information Bases (MIBs) or YANG modules. Figure 6-1 illustrates the LLDP agent and its relationship to its LLC Entity and MSAP, and to additional MIBs or YANG modules designed by the IETF, IEEE 802, and others.

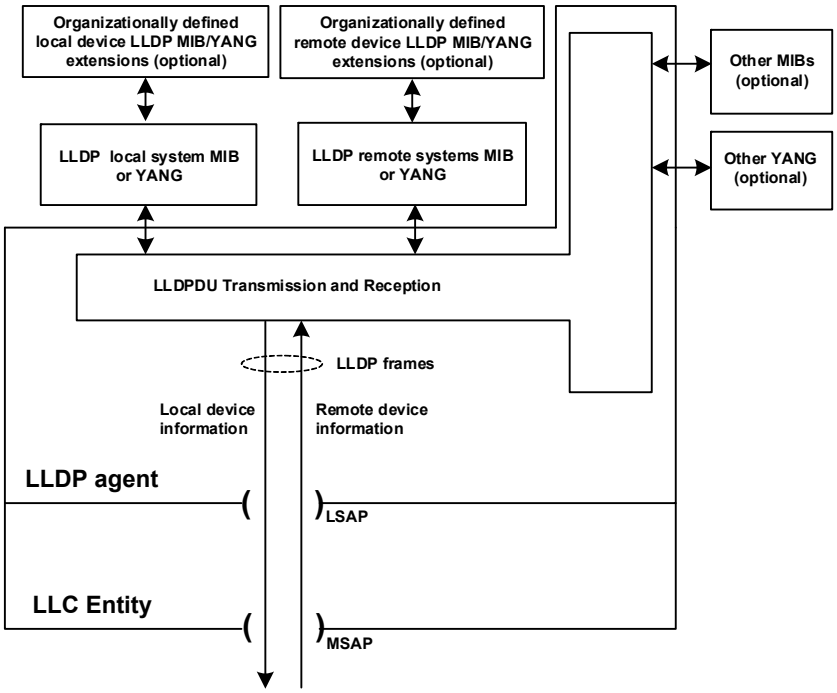


Figure 6-1—LLDP agent and its relationship to its LLC entity

This clause describes general principles of LLDP and Extended LLDP (XLLDP) operation. The following clauses specify transmission, reception, and addressing of LLPDUs (Clause 7); LLDPDU formats (Clause 8); operation of each LLDP agent in detail including state machines (Clause 9); management of LLDP (Clause 10); and the MIB module (Clause 11) or YANG module (Clause 12) used for LLDP management.

NOTE 1—For the purposes of this standard, the terms “LLC” and “LLC entity” include the service provided by the operation of entities that support protocol discrimination using an EtherType; i.e., protocol discrimination based on the Type interpretation of the Length/Type field as specified in IEEE Std 802.3, or the use of the SNAP encapsulation of EtherTypes as described in IEEE Std 802. Hence, “LSAP” in the above description can refer to the use of IEEE Std 802.2 LLC addresses, or an EtherType, as a means of higher layer protocol identification.

NOTE 2—The LLC Entity also provides distinct LSAPs to other higher layer protocol entities (such as those responsible for the Spanning Tree Protocol and IP). In a bridge, the LLC Entity associated with each Bridge Port is modeled as being directly connected to the attached LAN, as discussed in 8.13.9 of IEEE Std 802.1Q-2022, so the operation of LLDP is unaffected by the spanning tree Port State.

<<Editor’s Note: If 802.1Q-2022-Rev is approved before 802.1AB-2016-Rev, references to 802.1Q-2022 will be updated to reference 802.1Q-2022-Rev.>>

6.1 Transmission and reception

The information fields in each LLDP frame are contained in a Link Layer Discovery Protocol Data Unit (LLDPDU) as a sequence of variable length information elements, that each include type, length, and value fields (known as TLVs), where

— Type identifies what kind of information is being sent.

— Length indicates the length of the information string in octets.

— Value is the actual information that needs to be sent (for example, a binary bit map or an alpha-numeric string that can contain one or more fields).

Each LLDPDU begins with a mandatory sequence of three TLVs, as specified in 8.2, and can contain zero or more additional optional TLVs as selected by network management. The first three TLVs are:

1) A Chassis ID TLV.
2) A Port ID TLV.
3) A Time To Live TLV or, if XLLDP is supported, an Extension Request TLV or an Extension Identifier TLV.

The first two TLVs contain the chassis ID and the port ID values. These are concatenated to form a logical MSAP identifier. The combination of the MSAP identifier and the Scope MAC Address identify the LLDP agent/port that is the source of the information in the TLVs. Both the chassis ID and port ID values can be defined in a number of convenient forms. Once selected, however, the chassis ID/port ID value combination remains the same as long as the particular port remains operable (8.5.2, 8.5.3, 9.2.4.1).

The third TLV differentiates between three types of LLDPDUs:

a) A Normal LLDPDU, where the third TLV is a Time To Live TLV that tells the receiving LLDP agent how long all information pertaining to this LLDPDU's MSAP identifier is valid so that all of the associated information can later be automatically discarded by the receiving LLDP agent if the sender fails to update it in a timely manner. It is useful to further differentiate two special cases of a Normal LLDPDU:
   1) A shutdown LLDPDU, where the third TLV is a Time To Live TLV with a TTL value of zero indicating that any information pertaining to this LLDPDU's MSAP identifier is to be discarded immediately. A TTL value of zero can be used, for example, to signal that the sending port has initiated a port shutdown procedure.
   2) A Manifest LLDPDU, where the third TLV is a Time To Live TLV with a non-zero TTL value and one of the subsequent TLVs is a Manifest TLV.
b) An Extension Request LLDPDU (XREQ), where the third TLV is an Extension Request TLV that the recipient of a Normal LLDPDU containing a Manifest TLV uses to request one or more of the Extension LLDPDUs described in the Manifest TLV. The Extension Request LLDPDU does not contain a Time to Live TLV.
c) An Extension LLDPDU (XPDU), where the third TLV is an Extension Identifier TLV. The Extension LLDPDU does not contain a Time to Live TLV.

NOTE —The receive state machine (in 9.2.10 and in prior versions of this standard) specifies that an implementation not supporting XLLDP will discard any received LLDPDUs not containing a TTL TLV as the third TLV (i.e., will discard any received Extension Request or Extension LLDPDUs). The state machine further specifies that only implementations supporting XLLDP will process a Manifest TLV and subsequently transmit Extension Request LLDPDUs, and that only implementations supporting XLLDP will process Extension Request TLVs and subsequently transmit Extension LLDPDUs.

An optional End Of LLDPDU TLV marks the end of the LLDPDU.

The format for the LLDPDU is defined in 8.2. The TLV categories and the basic TLV format are defined in 8.3 and 8.4. The specific format and field contents for the Chassis ID TLV are defined in 8.5.2; for the Port ID TLV, in 8.5.3; for the Time To Live TLV in 8.5.4; and for the End Of LLDPDU TLV, in 8.5.1.

6.2 LLDP operational modes

The information advertised by LLDP is transferred in a single direction. An LLDP agent can periodically advertise information about the capabilities and current status of the system associated with its MSAP identifier. The LLDP agent can also collect information about the capabilities and current status of the system associated with a remote MSAP identifier. LLDP does not itself contain a mechanism for the LLDP agent to solicit specific information from other LLDP agents beyond what is being advertised by those LLDP agents, nor does it provide a specific means of confirming the receipt of information. Extended LLDP allows an LLDP agent collecting information to request detailed information (via an Extension Request LLDPDU) that is only advertised in a summary form (in a Manifest TLV), however this does not change the inherent LLDP property that the choice of information provided is at the discretion of the advertising LLDP agent.

NOTE—The LLDP protocol is designed to advertise information useful for discovering pertinent information about a remote port and to populate topology MIBs or YANG modules. It is not intended to act as a configuration protocol for remote systems, nor as a mechanism to signal control information between ports. During the operation of LLDP, it may be possible to discover configuration inconsistencies between systems on the same IEEE 802 LAN. LLDP does not provide a mechanism to resolve those inconsistencies. Rather, it provides a means to report discovered information to higher layer management entities.

LLDP allows the advertising function and collecting function of the LLDP agent to be separately enabled, making it possible to configure an implementation so the local LLDP agent can either advertise only or collect only, or so the local LLDP agent can advertise and collect LLDP information.

6.3 LLDP information categories

The following basic management TLV set (this set is required in all LLDP implementations) is defined by this standard and can be used to describe the system and/or to assist in the detection of configuration inconsistencies associated with the MSAP identifier:

a) Port Description TLV.
b) System Name TLV.
c) System Description TLV.
d) System Capabilities TLV (indicates both the system’s capabilities and its current primary network function, such as end station, bridge, router).
e) Management Address TLV.

Table 8-1 includes a list of the optional TLVs in the basic management set and provides subclause references for their specific definitions.

Organizationally Specific TLVs can be defined by either the professional organizations or the individual vendors that are involved with the particular functionality being implemented within a system. The basic format and procedures for defining Organizationally Specific TLVs are provided in 8.7.

6.4 TLV selection

Information for constructing the various TLVs to be sent is stored in the LLDP local system information repository (RP). The selection of which particular TLVs to send is under control of network management. Information received from remote LLDP agents is stored in the LLDP remote systems information repository.

6.5 Transmission principles

A transmission cycle can be initiated either by the expiration of a transmit countdown timing counter or by a change in the value of one or more of the information elements (managed objects) associated with the local system. When a transmit cycle is initiated, the LLDP management entity extracts the managed objects from the LLDP local system information repository and formats this information into TLVs. The TLVs are inserted into an LLDPDU that is passed to the LLDP transmit module. When Extended LLDP is supported, some of the TLVs are inserted into one or more Extension LLDPDUs that are passed to the LLDP transmit module when requested by the receipt of a Extension Request LLDPDU. The LLDP transmit module prepends addressing parameters to the LLDPDU as defined in 8.2, 8.3, and 8.4. The LLDPDU and TLV formats are defined in Clause 8. The LLDP transmit state machine is described in 9.2.1 and 9.2.9.

NOTE 1—Because a transmission cycle can be initiated whenever a change occurs within the LLDP local system information repository, it is possible that a series of successive changes over a short period of time could trigger a number of LLDP frames to be sent, each reporting only a single change. LLDP utilizes a transmission delay timer that can be set by network management to limit the number of LLDP transmission cycles in a given time period.

NOTE 2—Under normal circumstances, the information in the receiving LLDP agent's remote systems information repository is refreshed periodically to avoid being discarded due to ageing. To prevent the receiving LLDP agent's remote systems information repository being aged out because a refresh frame has been lost in transmission, the sending LLDP agent typically sets the TTL value so that several refresh cycles can occur before the received repository information ages out.

LLDP can disclose information about a device and network that could be considered sensitive in certain environments. Implementers are encouraged to provide controls that take into account the link protection characteristics when including information in an LLDP message. Different information can be included if an LLDP message is being sent on the uncontrolled port, the controlled port, or a MACsec protected connectivity association.

6.6 Reception principles

The LLDP receive module uses the services of the LLC entity to recognize that the incoming MA_UNITDATA.indication contains the correct combination of destination address and MSDU header values to identify it as resulting from a received LLDPDU. The LLDPDU recognition procedures are defined in 7.4 and 9.2.8.7.

6.6.1 LLDPDU and TLV error handling

The LLDPDU is checked to verify that it contains the correct sequence of three mandatory TLVs at the beginning of the frame, as specified in 8.2, and then each optional TLV is validated in succession. LLDPDUs that contain detectable errors in the first three mandatory TLVs are discarded. Optional TLVs that contain detectable errors are discarded [see 9.2.8.7.2 c)]. TLVs that are not recognized, but that also contain no basic format errors, are assumed to be valid and are stored for possible later retrieval by network management (see 9.2.8.7.1 and 9.2.8.5). If the End Of LLDPDU TLV is present, any octets that follow it are discarded.

TLVs in which the information string length field contains a value that is greater than the sum of the lengths of the fields within the information string are not discarded.

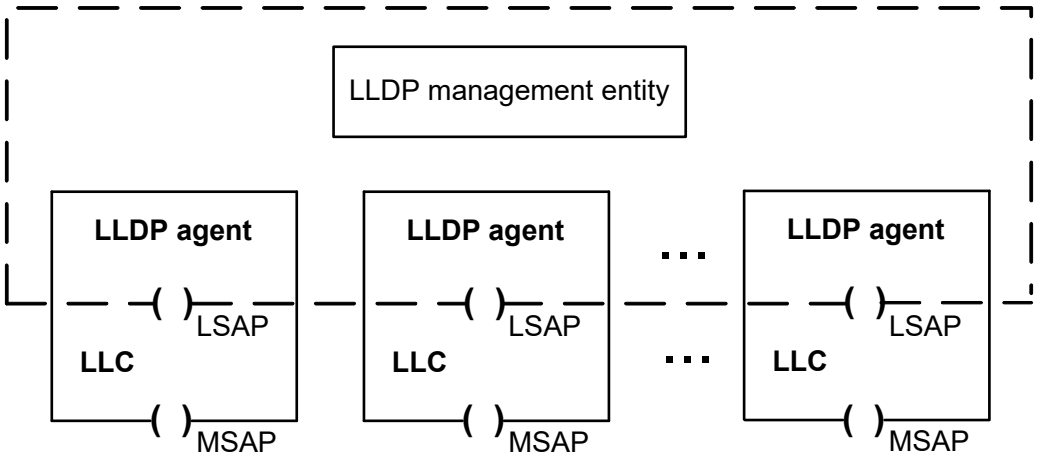
NOTE—This approach allows later versions of a TLV to define additional fields at the end of the information string; implementations based on an earlier version of the TLV can therefore continue to process the fields that were defined in that version and ignore any new fields added at the end of the information string in later versions. Any information contained in such new fields is not stored in the information repository and made available to network management protocols via the standard MIB or YANG modules.

6.6.2 LLDP remote systems information repository update

The LLDP remote systems information repository is updated after all TLVs have been validated. LLDP remote system information repository update procedures are defined in 9.2.8.9, 9.2.8.5, and 9.2.8.4. The LLDP receive state machine is described in 9.2.1 and 9.2.10.

6.7 Systems with multiple LLDP Agents

Each LLDP agent advertises a single set of information in the various TLVs it encodes in each transmission cycle, and is associated with the MSAP that supports the LLC entity that the LLDP agent uses to transmit and receive. Each LLDP agent uses its LSAP directly, without the use of any additional multiplexing or addressing above the LSAP to support the use of that LSAP by multiple LLDP agents; and each LLC entity provides service to one and only one protocol entity at each of its LSAPs that it supports, using the service provided by a single MSAP. It follows that each LLDP agent makes use of a unique MSAP, and that the LLDP agent can be uniquely identified by the receiving agent using the MSAP's identifier and the MAC address that determines the LLDP agent's address scope, as specified in 6.1. A single LLDP management entity can support the operation of multiple LLDP agents within the same system. Figure 6-2 illustrates the relationship between the LLDP agents, LLC Entities, MSAPs, and the LLDP management entity.



LLDP - Link Layer Discovery Protocol
LSAP - Link service access point
MSAP - MAC service access point

Figure 6-2—Relationship between LLDP agents, LLC Entities, MSAPs, and the LLDP management entity

A given system can require the use of multiple LLDP agents because it naturally comprises multiple MSAPs, or because it needs to be able to transmit different information (different sets of TLVs) to different sets of peers. These sets of peers can be determined by the way that the MAC service is supported within the

system (see IEEE Std 802.1AC for a description of the connectionless connectivity that characterizes an IEEE 802 LAN). A MAC Bridge (see IEEE Std 802.1Q), for example, attaches to multiple LANs, each providing the MAC service at a distinct MSAP. A different example is a station that uses port-based network access control (IEEE Std 802.1X) to provide both a Controlled Port and an Uncontrolled Port, i.e., two distinct MSAPs with potentially different connectivity, for transmission and reception to and from a single LAN. Similarly, different destination addresses can be associated with different sets of potential recipients. Table 7-1 identifies group MAC addresses that can be used for LLDPDU transmission, and 7.1 describes the different transmission scopes associated with each address. LLDP can also be used in conjunction with individual MAC addresses, and with other group MAC addresses. Distinct LLDP agents, and hence separate and distinct protocol state machines, local and remote information repository tables, and MSAPs are maintained for each LLDPDU destination address, even if the use of two or more MSAPs can result in transmission and reception on the same LAN.

NOTE—For a given MAC address that is used as a destination address in received LLDPDUs, there is the possibility for a station to receive LLDPDUs from multiple senders. All LLDPDUs that carry that destination MAC address are received by a single LLDP agent, via a single MSAP; however, as the LLPDU carries information that identifies the source of the LLDPDU, information carried in LLPDUs from different senders is able to be recorded independently in the remote systems information repository.

Figure 6-3 illustrates the use of LLDP in a MAC Bridge, with an LLDP agent for each of the two ports shown. Figure 6-4 shows the use of LLDP in an end system with port-based network access control (IEEE Std 802.1X) supported by MACsec (IEEE Std 802.1AE); an LLDP agent is supported by the Controlled Port and an additional LLDP agent may be supported by the Uncontrolled Port. Figure 6-5 shows LLDP in a MAC Bridge that uses port-based network access control on both ports.

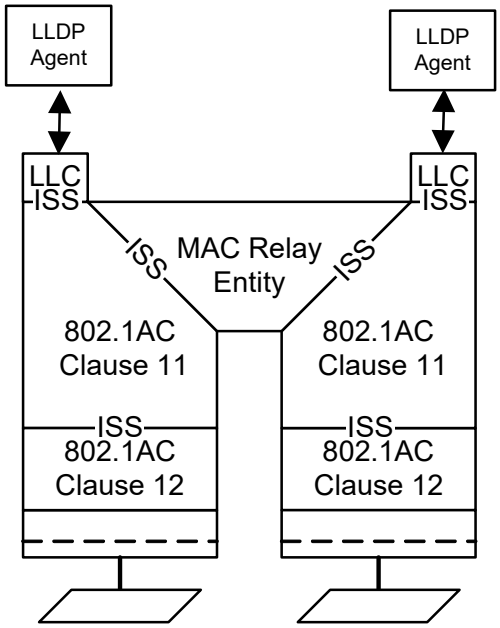


Figure 6-3—LLDP in a MAC Bridge

LLDP frames sent on an unprotected link may be modified by an attacker. If an implementation is going to take action based upon a received LLDP attribute, it is advisable to take into account whether the message was received on a protected link, and allow for the system to be configured such that only information received in protected messages is acted upon.

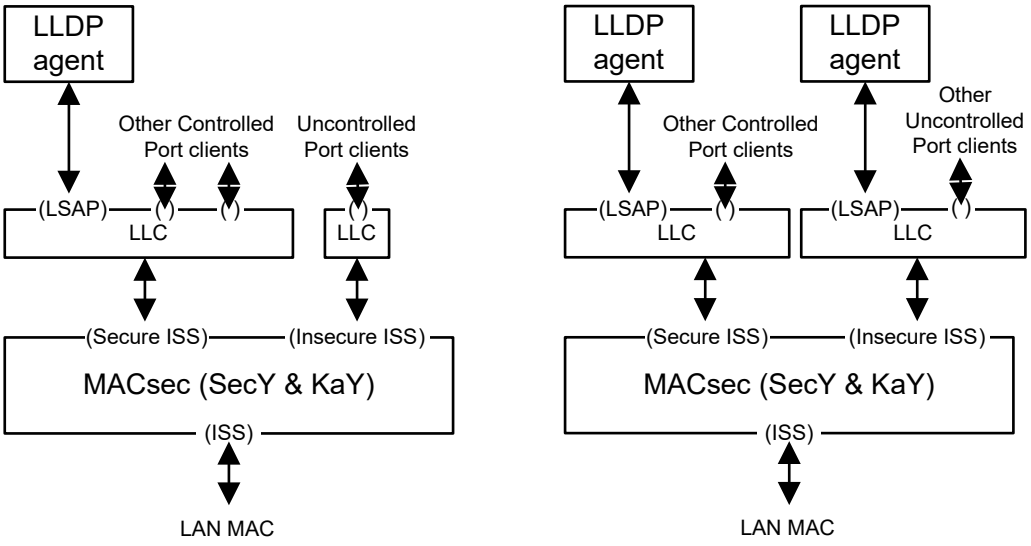


Figure 6-4—LLDP in an end system with port-based network access control

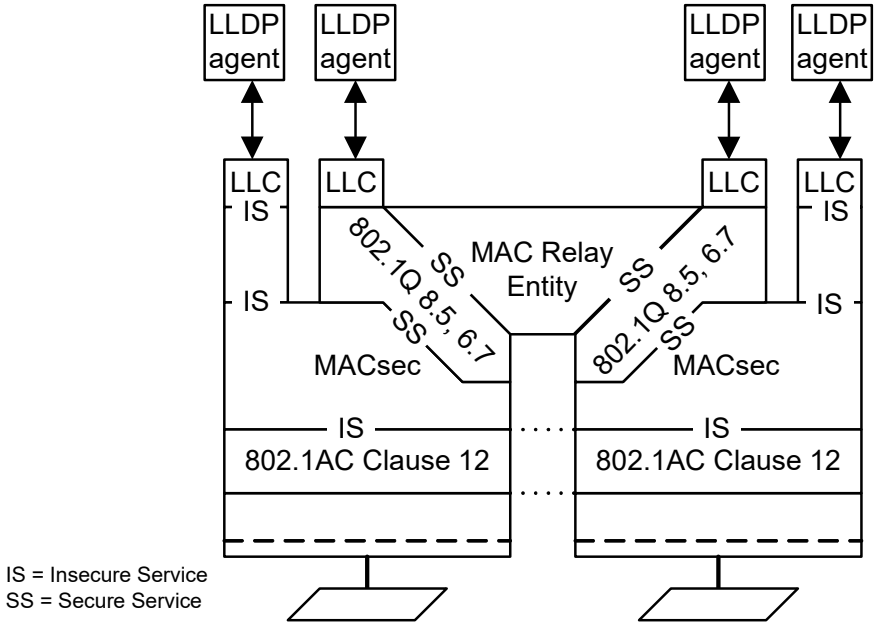


Figure 6-5—LLDP in a MAC Bridge that uses port-based network access control on both ports

The group addresses identified in Table 7-1 are taken from the set specified as reserved addresses by bridging standards. Frames (including those conveying LLDPDUs) that use these destination addresses are filtered or not by the various types of bridge components, as specified by Table 8-1 through Table 8-3 of IEEE Std 802.1Q-2022. The choice of a particular address thus allows a given agent to limit the propagation

of LLDPDUs to an individual LAN, or to allow them a wider scope. Figure 6-6 illustrates the various scopes achievable with the three destination group MAC addresses identified in Table 7-1, their use is specified further in 7.1.

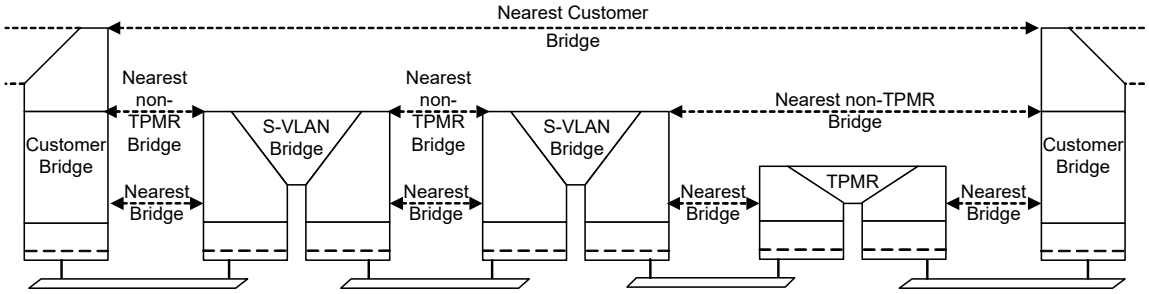


Figure 6-6—Scope of group MAC addresses

More than one LLDP agent, each using a different address scope, can be instantiated for a given system port by adding a simple shim that provides the necessary distinct MSAPs by multiplexing and demultiplexing between those MSAPs and a common MSAP for the port, as illustrated in Figure 6-7. Each MSAP shown in

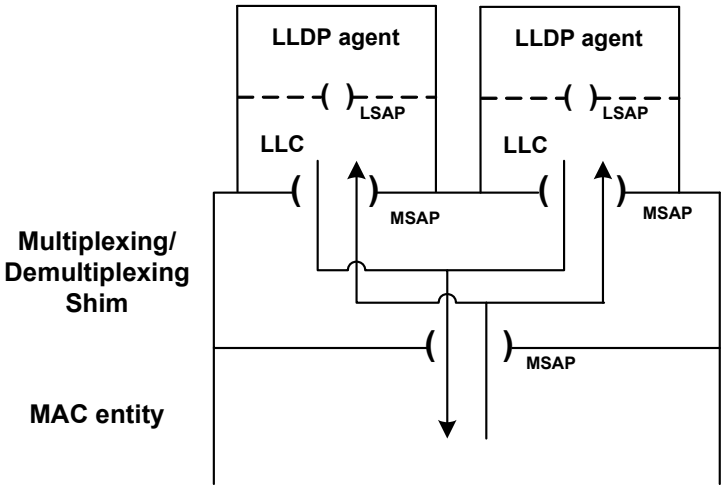


Figure 6-7—Multiplexing and demultiplexing using shims

Figure 6-7 is allowed to share the same MSAP Identifier and individual MAC address. The LLDP agents are differentiated by address scope used, i.e., by the MAC address used as a Destination Address in frames containing a Normal LLDPDU.

6.8 LLDP and Link Aggregation

An LLDP agent can be configured on an IEEE 802.1AX™ Aggregated Port and/or on any number of the physical ports belonging to the Aggregation. Some TLVs (e.g., the Energy-Efficient Ethernet (EEE) TLV designated in subclause 79.3.5 of IEEE Std. 802.3-2022) only make sense when transmitted from an LLDP agent configured on a physical port, and can cause incorrect operation if configured on an Aggregation. Other TLVs (e.g., the DCBX TLVs defined in 38.4 of IEEE Std 802.1Q-2022) make sense only when transmitted from an Aggregated Port, because they carry information applicable to the simulated single

interface represented by the Aggregation. Not all standards that define LLDP TLVs are careful to state whether each TLV is applicable to an Aggregation, to a physical port, or to both. Only LLDP TLVs that are appropriate for transmission on a given Port should be transmitted on that Port.

An LLDP agent configured on an Aggregated Port, or on one of the physical ports of an Aggregation, transmits the Link Aggregation TLV, and processes received LLDPDUs, as specified in F of 802.1AX-2020. An LLDP agent configured on a physical port of an Aggregation uses the LLDP Parser/Multiplexer as specified in 6.1.3 of IEEE Std 802.1AX-2020 for the transmission and reception of LLDPDUs containing TLVs that are specific to that physical port. Since a port not running Link Aggregation is equivalent to an Aggregation with a single physical port, any LLDP agent can transmit the Link Aggregation TLV and process received LLDPDUs as specified in 802.1AX.

NOTE—The consideration of aggregated vs. Physical ports is similar to the distinction among destination MAC addresses discussed in 7.1. In both cases, LLDP TLVs that carry information specific to a physical link should be used only in the LLDP agent with the shortest reach.

6.9 Extended LLDP (XLLDP)

Extended LLDP allows an LLDP agent to advertise more TLVs than would fit within a single LLDPDU by transmitting the additional TLVs in one or more Extension LLDPDUs. The selection of TLVs to be included in LLDPDUs is controlled by management, but the mapping of TLVs to the initial Normal LLDPDU or to an Extension LLDPDU is implementation dependent (10.2.2). When a transmission cycle is initiated, a unique description of each Extension LLDPDU is encoded in a Manifest TLV, which is then included in the Normal LLDPDU transmitted at the beginning of the cycle. A Normal LLDPDU containing a Manifest TLV is referred to as a Manifest LLDPDU. Changes to any of the TLVs in an Extension LLDPDU changes the description included in the Manifest TLV, and thus triggers a new transmission cycle.

An implementation not supporting XLLDP that receives a Manifest TLV treats it as an unrecognized TLV and only stores the TLVs that were included in the Normal LLDPDU. An implementation supporting XLLDP that receives a Manifest TLV then requests the transmission of any new or modified Extension LLDPDUs by sending an Extension Request LLDPDU to the LLDP agent sourcing the Manifest LLDPDU. One or more Extension LLDPDUs can be requested in a single Extension Request LLDPDU. A new Extension Request LLDPDU can be transmitted when all Extension LLDPDUs previously requested have been received. This allows the requesting LLDP agent to pace the rate of information transfer. A timer on the request allows an Extension Request LLDPDU to be retried if it appears that either the request or a response has been lost.

Extension Request LLDPDUs and Extension LLDPDUs are transmitted with an individual MAC address as the destination address. The destination address in an Extension Request LLDPDU is taken from the contents of the Return MAC Address field of the received Manifest TLV. The requested Extension LLDPDUs are transmitted with a destination address taken from the contents the Return MAC Address field of the Extension Request TLV.

7. LLDPDU transmission, reception, and addressing

This standard is intended to be compatible with all IEEE 802 MACs.

LLDP uses the service provided by the LLDP/LSAP and LLC to transmit and receive LLDPDUs. Each LLDPDU is transmitted as a single MAC service request by an LLC entity that uses a single instance of the MAC Service provided at an MSAP. Each incoming LLDP frame is received at the MSAP by the LLC entity as a MAC service indication.

NOTE—For the purposes of this standard, the terms “LLC” and “LLC entity” include the service provided by the operation of entities that support protocol discrimination using an EtherType, i.e., protocol discrimination based on the Type interpretation of the Length/Type field as specified in IEEE Std 802.3, or the use of the SNAP encapsulation of EtherTypes as described in IEEE Std 802. Hence, “LSAP” in the above description can refer to the use of ISO/IEC 8802-2 [B15] LLC addresses, or an EtherType, as a means of higher layer protocol identification.

The parameters of each service request and service indication comprise

a) Destination address
b) Source address
c) EtherType
d) LLDPDU

The LLDP EtherType, used to identify the LLDP protocol, is prepended to the LLDPDU as shown in Figure 7-1 to form the MSDU of the corresponding MAC service request.

Figure 7-1—MSDU format

The values of the parameters used by LLDP, and their encoding by the LLC entity that supports the LLDP LSAP, are specified in the following subclauses.

7.1 Destination address

7.1.1 Destination address for Normal LLDPDUs

The MAC address used as the destination address in Normal LLDPDUs is referred to as the Scope MAC Address because it determines the limits of propagation of the LLDPDU within a network. A set of standard group MAC addresses that can be used by LLDP as the Scope MAC Address is specified in Table 7-1. These addresses are in the range of IEEE Std 802.1Q C-VLAN and MAC Bridge component Reserved addresses (see Table 8-1 in IEEE Std 802.1Q-2022), the S-VLAN component Reserved addresses (see Table 8-2 in IEEE Std 802.1Q-2022), and the TPMR component Reserved addresses (see Table 8-3 in IEEE Std 802.1Q-2022). The choice of address used determines the scope of propagation of LLDPDUs within a bridged LAN, as follows:

a) The ***nearest bridge*** group MAC address is an address that no conformant Two-Port MAC Relay (TPMR) component, S-VLAN component, C-VLAN component, or MAC Bridge component can forward. LLDPDUs transmitted using this destination address can therefore travel no further than those stations that can be reached via a single individual LAN from the originating station.

LLDPDUs received on this address therefore represent information about stations that are attached to the same individual LAN segment as the recipient station.

NOTE 1—This address was selected in order to make it possible to transmit an LLDP frame containing information specific to a single individual LAN, and for that information not to be propagated further than the extent of that individual LAN, to avoid the meaning of that information being misinterpreted. For example, a TLV containing the auto-negotiation status of an IEEE 802.3™ LAN would be open to misinterpretation if it were to be propagated via a Bridge onto a second IEEE 802.3 LAN, and would be meaningless if propagated onto a LAN based on a different MAC technology. This is the "LLDP_Multicast address" that was used in IEEE Std 802.1AB-2005.

b) The ***nearest non-TPMR bridge*** group MAC address is an address that no conformant C-VLAN component, S-VLAN component, or MAC Bridge can forward. However, this address is relayed by TPMR components. Therefore, LLDPDUs received on this address represent information about stations that are not separated from the recipient station by any intervening C-VLAN component, S-VLAN component, or MAC Bridge. There may, however, be one or more TPMR components in the path between the originating and receiving stations.

NOTE 2—This address was selected in order to make it possible to communicate information between adjacent bridges and for that communication to be transparent to the presence or absence of TPMRs in the transmission path. This address is primarily intended for use within provider bridged networks.

c) The ***nearest Customer Bridge*** group MAC address is an address that no conformant C-VLAN component or MAC Bridge forwards. However, this address is relayed by TPMR components and S-VLAN components. Therefore, LLDPDUs received on this address represent information about stations that are not separated from the recipient station by any intervening C-VLAN components (e.g., a C-VLAN Bridge or Provider Edge Bridge). There may, however, be TPMR components and/or S-VLAN components in the path between the originating and receiving stations.

NOTE 3—This address was selected in order to make it possible to communicate information between adjacent Customer Bridges and for that communication to be transparent to the presence or absence of TPMRs or S-VLAN components in the communication path. The scope of this address is the same as that of a customer-to-customer MACSec connection.

Table 7-1—Group MAC addresses used by LLDP

Name	Value	Purpose
Nearest bridge	01-80-C2-00-00-0E	Propagation constrained to a single physical link; stopped by all types of bridge
Nearest non-TPMR bridge	01-80-C2-00-00-03	Propagation constrained by all bridges other than TPMRs; intended for use within provider bridged networks
Nearest Customer Bridge	01-80-C2-00-00-00	Propagation constrained by customer bridges; this gives the same coverage as a customer-customer MACSec connection

NOTE 4—The LSAP or EtherType field is necessary to distinguish between different protocols conveyed in frames with the same group or individual destination address. Prior to revision of this standard to allow multiple address scopes, the destination address 01-80-C2-00-00-00 was only explicitly specified for Spanning Tree Protocol BPDUs. It is therefore possible that some implementations recognize only a frame's destination address before applying priority handling resources intended to guard against BPDU loss. Since LLDP rate limits LLDPDU transmission, the additional consumption of these resources by LLDPDUs is unlikely to pose a problem for robust implementations.

It is assumed in the foregoing definitions of the scope of these group MAC addresses that an end station attached to an individual LAN is a potential recipient of any of these addresses, and therefore such end stations fall within the scope of these addresses as well as the identified bridge components.

In addition to determining the scope of transmission, the support of more than one destination MAC address allows different sets of TLVs to be supported over the different transmission scopes.

In addition to the prescribed support for standard group MAC addresses shown in Table 7-1, implementations of LLDP may support the following as the Scope MAC Address:

d) Any group MAC address.

e) Any individual MAC address.

Support for the use of each of these destination addresses, for both transmission and reception of Normal LLDPDUs, is either mandatory, recommended, permitted, or not permitted, according to the type of system in which LLDP is implemented, as shown in Table 7-2.

Table 7-2—Support for the Scope MAC Address in different systems

Address	C-VLAN Bridge	S-VLAN Bridge	TPMR Bridge	End station
Nearest bridge	Mandatory	Mandatory	Mandatory	Mandatory
Nearest non-TPMR bridge	Mandatory	Mandatory	Not permitted	Recommended
Nearest Customer Bridge	Mandatory	Not permitted	Not permitted	Recommended
Any other group MAC address	Permitted	Permitted	Permitted	Permitted
Any individual MAC address	Permitted	Permitted	Permitted	Permitted

NOTE 5 —Where the implementation supports the use of individual MAC addresses, there is a need for some means whereby the sender can ascertain what address to use, and what scope is associated with that address; how this is achieved is outside the scope of this standard.

NOTE 6—The option of using an individual MAC addresses supports the use of LLDP in IEEE Std 802.11™ [B1] and other wireless LANs, where it is desirable to target specific TLV content at specific logical Ports. In particular, the use of an individual MAC address allows an access point to direct LLDP frames at an individual station with which it is associated.

NOTE 7—If an individual MAC address is specified as the Scope MAC Address, it is used as the destination for transmitted LLDPDUs and also used to validate received LLDPDUs. The primary purpose of using an individual address as the Scope MAC Address is that an LLDP agent configured as “transmit only” sends Normal LLDPDUs to a specific receiving LLDP agent. It is also allowable for an LLDP agent configured as “receive only” to use an individual address (typically the individual MAC address of the LLDP agent’s MSAP) so that only information from an LLDP agent configured as “transmit only” using this same address is collected. Using an individual MAC address as the destination address for an LLDP agent configured to “transmit and receive” is allowed, although it would not typically be useful.

NOTE 8—The destination MAC address used by a given LLDP agent defines only the scope of transmission and the intended recipient(s) of the LLDPDUs; it plays no part in protocol identification. In particular, the group MAC addresses identified in Table 7-1 are not used exclusively by LLDP; other protocols that require to use a similar transmission scope are free to use the same addresses.

7.1.2 Destination address for Extension Request and Extension LLDPDUs

The destination address in an Extension Request LLDPDU or an Extension LLDPDU is an individual address. Extension Request LLDPDUs are sent in response to the receipt of a Manifest LLDPDU, and use the contents of the Return MAC Address field of the Manifest TLV as the destination address of an Extension Request LLDPDU. Likewise, Extension LLDPDUs are sent in response to the receipt of an Extension Request LLDPDU, and use the contents of the Return MAC Address field of the Extension Request TLV as the destination address of the Extension LLDPDU.

7.2 Source address

The source address shall be the individual MAC address of the MSAP supporting the LLDP agent.

7.3 EtherType use and encoding

The EtherType used to identify the LLDP protocol shall be the LLDP EtherType specified in Table 7-3.

Table 7-3—LLDP EtherType

Name	Value
LLDP EtherType	88-CC

Where the LLC entity uses an MSAP that is supported by a specific media access control method (for example, IEEE Std 802.3) or a media access control independent entity (for example, IEEE Std 802.1AE) that directly supports encoding of EtherTypes, the LLC entity shall encode the LLDP EtherType as the two octet LLDPDU header in the MSDU of the corresponding MAC service request.

Where the LLC entity uses an MSAP that is supported by a specific media access control method that does not directly support Ethertype encoding (for example, IEEE Std 802.11 [B1]), the encoding shall be as specified in IEEE Std 802.1AC.

NOTE—Annex C provides example LLDP transmission frame formats for both direct-encoded and SNAP-encoded LLDP EtherType encoding methods.

7.4 LLDPDU reception

The LLDPDU shall be delivered to the LLDP receive module if, and only if

a) The destination MAC address is equal to the individual MAC address of the MSAP supporting the LLDP agent, or the destination MAC address is equal to the Scope MAC Address associated with the corresponding LLDP agent; and

NOTE—The destination MAC address can be one of the addresses in Table 7-1, or any other valid MAC address—see 7.1.

b) The EtherType is equal to the value shown in Table 7-3.

8. LLDPDU and TLV formats

8.1 LLDPDU bit and octet ordering conventions

All LLDPDUs contain an integral number of octets. The octets in an LLDPDU are numbered starting from 1 and increasing in the order they are put into the LLDPDU. The bits in an octet are numbered from 1 to 8, where bit 1 is the low-order bit.

When consecutive bits within an octet are used to represent a binary number, the highest bit number has the most significant value. When consecutive octets are used to represent a binary number, the lower octet number has the most significant value. All TLVs respect these bit and octet ordering conventions.

When the encoding of a field or a number of fields is represented using a diagram

a) Octets are shown with the lowest numbered octet nearest the left of the page, the octet numbering increasing from left to right.
b) Within an octet, bits are shown with bit 8 to the left and bit 1 to the right.

8.2 LLDPDU format

The LLDPDU contains the following ordered sequence of three mandatory TLVs followed by zero or more optional TLVs as shown in Figure 8-1. An End of LLDPDU TLV may be present as the last TLV in the LLDPDU.

a) Three mandatory TLVs are included at the beginning of each LLDPDU and are in the order shown.
    1) Chassis ID TLV
    2) Port ID TLV
    3) Time To Live TLV or, if XLLDP is supported, Extension Request TLV or Extension Identifier TLV
b) Optional TLVs as selected by network management (may be inserted in any order).

NOTE 1—”Optional” in the sense that they are not required for LLDP operation; however, their presence could be required by other system elements that use LLDP.

c) If the End Of LLDPDU TLV is present, it is the last TLV in the LLDPDU.

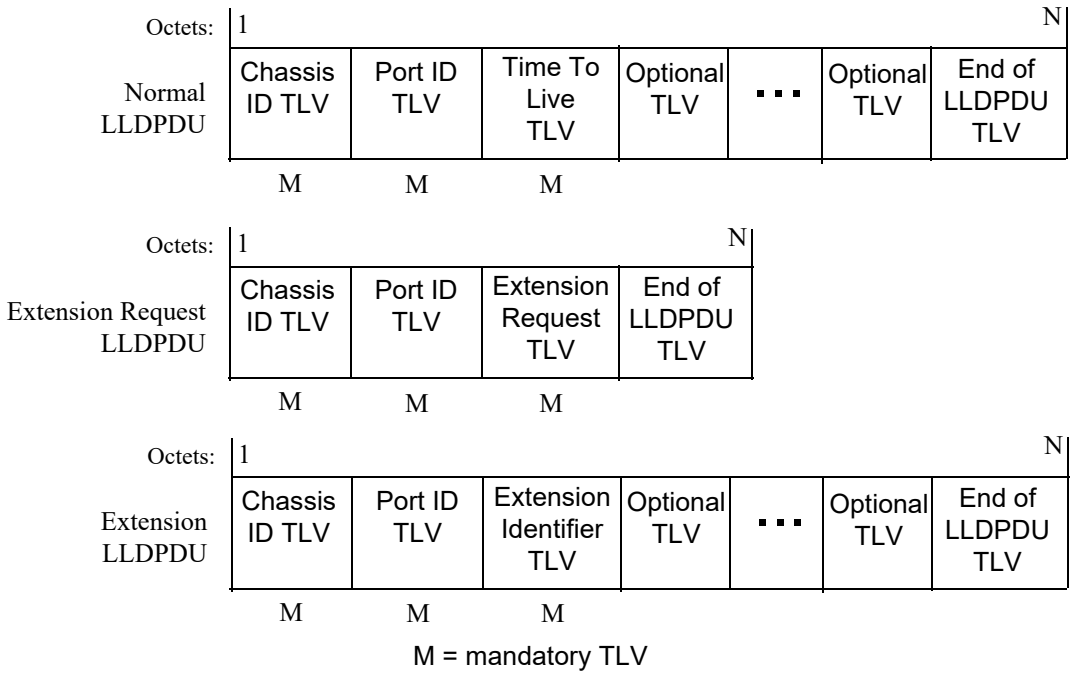


Figure 8-1—LLDPDU formats

The maximum length of the LLDPDU is less than or equal to the maximum information field length allowed by the particular transmission rate and protocol. In IEEE 802.3 MACs, for example, the maximum LLDPDU length is the maximum data field length for the basic, untagged MAC frame (1500 octets).

NOTE 2—Implementations can restrict the maximum length of the LLDPDU to a value smaller than what would be allowed by the underlying MAC technology. For example, an end station or bridge implementing Time-Sensitive Networking (TSN) features could restrict the maximum length to limit delay variation when transmitting frames in time sensitive streams.

NOTE 3—There is no defined minimum length of an LLDPDU, other than that implied by the requirement that conformant implementations support the mandatory TLVs specified in Table 8-1.

8.3 TLV categories

The general format of LLDP TLVs is defined in 8.4. The TLVs are grouped into three categories as follows:

a) A set of TLVs that are considered to be basic to the management of network stations. Support of the basic management set is required capability of all LLDP implementations. Each TLV in this category is identified by a unique TLV type value that indicates the particular kind of information contained in the TLV. The specific definitions and usage rules for the basic management set TLVs are defined in 8.5.

b) A set of TLVs that support Extended LLDP (XLLDP) operation. Support of the extended LLDP set is a required capability of all LLDP implementations supporting XLLDP. Each TLV in this category is identified by a unique TLV type value that indicates the particular kind of information contained in the TLV. The specific definitions and usage rules for the extended LLDP set TLVs are defined in 8.6.

c) Organizationally specific extension sets of TLVs that are defined by standards groups such as IEEE 802.1 and IEEE 802.3 and others to enhance management of network stations that are

operating with particular media and/or protocols. The format of Organizationally Specific TLVs, along with field definitions and usage rules common to all Organizationally Specific TLVs, are defined in 8.7.

1) TLVs in this category are identified by a common TLV type value that indicates the TLV as belonging to the set of Organizationally Specific TLVs.
2) Each organization is identified by its organizationally unique identifier (OUI), consisting of either an Organizationally Unique Identifier (OUI) or a Company ID (CID) [13].
3) Organizationally Specific TLV subtype values indicate the kind of information contained in the TLV.

8.4 Basic TLV format

Figure 8-2 shows the basic TLV format.

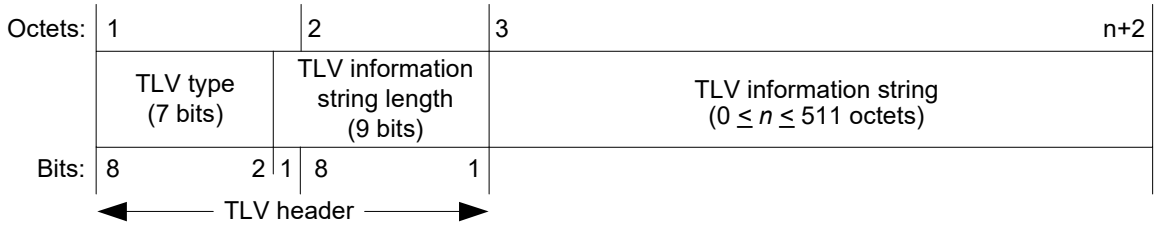


Figure 8-2—Basic TLV format

The TLV type field occupies the seven most significant bits of the first octet of the TLV format. The least significant bit in the first octet of the TLV format is the most significant bit of the TLV information string length field.

8.4.1 TLV type

The TLV type field is seven bits long and identifies the specific TLV. Two classes of TLVs are defined:

a) Mandatory TLVs that are included in all LLDPDUs.
b) Optional TLVs that may be included in LLDPDUs.

Table 8-1 lists the currently defined TLVs, their identifying TLV type values, and whether they are mandatory or optional for inclusion in any particular LLDPDU.

8.4.2 TLV information string length

The TLV information string length field contain the length of the information string, in octets.

8.4.3 TLV information string

The information string

a) May be fixed or variable length.
b) May include one or more information fields with associated subtype identifiers and field length designators as in, for example, the Management Address TLV (see 8.5.9).

[13] Organizationally Unique Identifier and Company ID assignments are administered by the IEEE Registration Authority, and Organizationally Unique Identifier assignments are included in MAC Address-Large (MA-L) assignments; see http://standards.ieee.org/regauth.

Table 8-1—TLV type values

TLV type [a]	TLV name	Usage in LLDPDU	Reference
0	End Of LLDPDU	Optional	8.5.1
1	Chassis ID	Mandatory	8.5.2
2	Port ID	Mandatory	8.5.3
3	Time To Live	Mandatory	8.5.4
4	Port Description	Optional	8.5.5
5	System Name	Optional	8.5.6
6	System Description	Optional	8.5.7
7	System Capabilities	Optional	8.5.8
8	Management Address	Optional	8.5.9
9	Manifest	Optional	8.6.1
10	Extension Request	Optional	8.6.2
11	Extension Identifier	Optional	8.6.3
12–126	Reserved for future standardization	—	—
127	Organizationally Specific TLVs	Optional	—

[a]TLVs with type values 0–8 are members of the basic management set. TLVs with type values 9–11 are members of the extended LLDP set.

c) May contain either binary or alpha-numeric information that is instance specific for the particular TLV type and/or subtype.
   1) Bit 1 in binary bit maps is the least significant bit in the field.
   2) The first octet of an alpha-numeric field is the most significant octet.
   3) Alpha-numeric information is encoded in UTF-8 [IETF RFC 3629].

8.5 Basic management TLV set formats and definitions

8.5.1 End Of LLDPDU TLV

The End Of LLDPDU TLV is a 2-octet, all-zero TLV that is used to mark the end of the TLV sequence in LLDPDUs. The format for this TLV is shown in Figure 8-3.

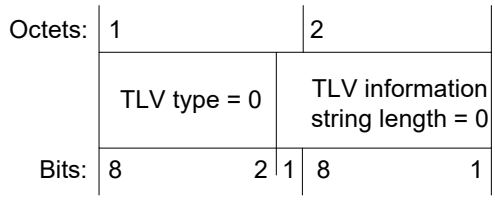


Figure 8-3—End Of LLDPDU TLV format

NOTE—Some IEEE 802 MACs require the data field in a frame to contain a minimum number of octets. For example, the IEEE 802.3 MAC adds pad octets to complete a minimum length data field if the user's data is less than the minimum required length. Since pad octets are unspecified, an End Of LLDPDU TLV can be used to prevent non-zero pad octets from being interpreted by the receiving LLDP agent as another TLV.

8.5.2 Chassis ID TLV

The Chassis ID TLV is a mandatory TLV that identifies the chassis containing the IEEE 802 LAN station associated with the MSAP Identifier associated with the LLDP agent containing the LLDP local system information repository that is the source of the information in TLVs. There are several ways in which a chassis may be identified and a chassis ID subtype is used to indicate the type of component being referenced by the chassis ID field. Each LLDPDU contain one, and only one, Chassis ID TLV and the chassis ID field value remains constant for all LLDPDUs while the MAC status parameter for the Port remains operational.

The Chassis ID TLV is the first TLV in the LLDPDU. Its format is shown in Figure 8-4.

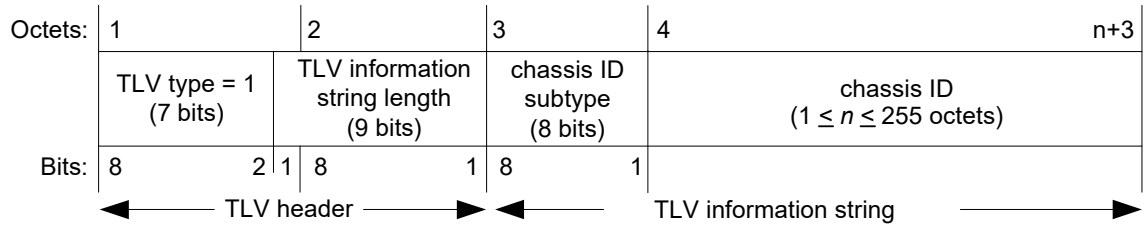


Figure 8-4—Chassis ID TLV format

8.5.2.1 TLV information string length

The TLV information string length field indicates the exact length, in octets, of the (chassis ID subtype + chassis ID) fields.

8.5.2.2 chassis ID subtype

The chassis ID subtype field contains an integer value indicating the basis for the chassis ID entity that is listed in the chassis ID field. The defined chassis ID subtypes and their preferred use order are listed in Table 8-2.

Table 8-2—chassis ID subtype enumeration

ID subtype	ID basis	Reference
0	Reserved	—
1	Chassis component	EntPhysicalAlias when entPhysClass has a value of ‘chassis(3)’ (IETF RFC 6933)
2	Interface alias	IfAlias (IETF RFC 2863)
3	Port component	EntPhysicalAlias when entPhysicalClass has a value ‘port(10)’ or ‘backplane(4)’ (IETF RFC 6933)
4	MAC address	MAC address (IEEE Std 802)
5	Network address	networkAddress [a]
6	Interface name	ifName (IETF RFC 2863)
7	Locally assigned	local[b]
8–255	Reserved	—

[a]networkAddress is an octet string that identifies a particular network address family and an associated network address that are encoded in network octet order. An IP address, for example, would be encoded with the first octet containing the IANA Address Family Numbers enumeration value for the specific address type and octets 2 through *n* containing the address value (for example, the encoding for C0-00-02-0A would indicate the IPv4 address 192.0.2.10).
[b]local is an alpha-numeric string and is locally assigned.

8.5.2.3 chassis ID

The chassis ID field contains an octet string indicating the specific identifier for the particular chassis in this system. Because chassis ID and port ID values are concatenated to form the local MSAP identifier, the value chosen from Table 8-2 for the chassis ID is non-null.

8.5.2.4 Chassis ID TLV usage rules

An LLDPDU contains exactly one Chassis ID TLV.

8.5.3 Port ID TLV

The Port ID TLV is a mandatory TLV that identifies the port component of the MSAP identifier associated with the MSAP Identifier associated with the LLDP agent containing the LLDP local system information repository that is the source of the information in TLVs. As with the chassis, there are several ways in which a port is identified. A port ID subtype is used to indicate how the port is being referenced in the port ID field. Each LLDPDU contain one, and only one, Port ID TLV. The port ID value remain constant for all LLDPDUs while the transmitting port remains operational.

The chosen port is identified in an unambiguous manner (for example, backplane number, management entity, MAC address).

The Port ID TLV is the second TLV in the LLDPDU. Its format is shown in Figure 8-5.

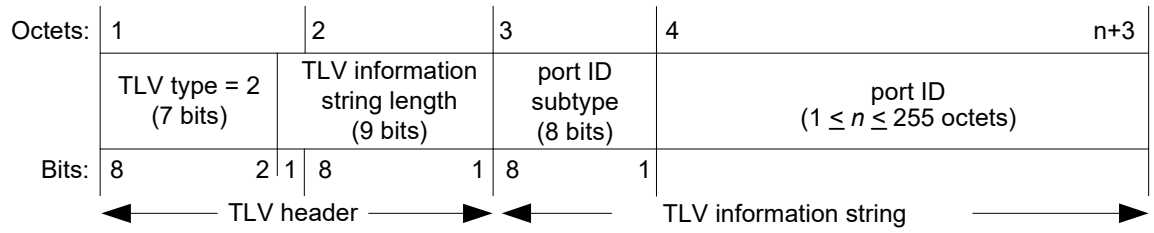


Figure 8-5—Port ID TLV format

8.5.3.1 TLV information string length

The TLV information string length field indicates the length, in octets, of the (port ID subtype + port ID) fields.

8.5.3.2 port ID subtype

The port ID subtype field contains an integer value indicating the basis for the identifier that is listed in the port ID field. The defined port ID subtypes and their preferred use order are listed in Table 8-3.

Table 8-3—port ID subtype enumeration

ID subtype	ID basis	References
0	Reserved	—
1	Interface alias	ifAlias (IETF RFC 2863)
2	Port component	entPhysicalAlias when entPhysicalClass has a value ‘port(10)’ or ‘backplane(4)’ (IETF RFC 6933)
3	MAC address	MAC address (IEEE Std 802)
4	Network address	networkAddress [a]
5	Interface name	ifName (IETF RFC 2863)
6	Agent circuit ID	agent circuit ID (IETF RFC 3046)
7	Locally assigned	local [b]
8–255	Reserved	—

[a]networkAddress is an octet string that identifies a particular network address family and an associated network address that are encoded in network octet order. An IP address, for example, would be encoded with the first octet containing the IANA Address Family Numbers enumeration value for the specific address type and octets 2 through *n* containing the address value (for example, the encoding for C0-00-02-0A would indicate the IP version 4 address 192.0.2.10).

[b]local is an alpha-numeric string and is locally assigned.

8.5.3.3 port ID

The port ID field is an alpha-numeric string that contains the specific identifier for the port from which this LLDPDU was transmitted. Because chassis ID and port ID values are concatenated to form the local MSAP identifier, the value chosen from Table 8-3 for the port ID is non-null.

NOTE—The port ID can contain alphanumeric data or binary data, depending upon the value of the port ID subtype.

8.5.3.4 Port ID TLV usage rules

An LLDPDU contains exactly one Port ID TLV.

8.5.4 Time To Live TLV

The Time To Live TLV indicates the number of seconds that the recipient LLDP agent is to regard the information associated with this MSAP identifier to be valid.

a) When the TTL field is non-zero the receiving LLDP agent is notified to completely replace all information associated with this MSAP identifier with the information in the received LLDPDU.
b) When the TTL field is set to zero, the receiving LLDP agent is notified to delete all system information associated with the LLDP agent/port. This TLV may be used, for example, to signal that the sending port has initiated a port shutdown procedure.

The Time To Live TLV is mandatory for all Normal LLDPDUs, and is the third TLV in the LLDPDU. Its format is shown in Figure 8-6.

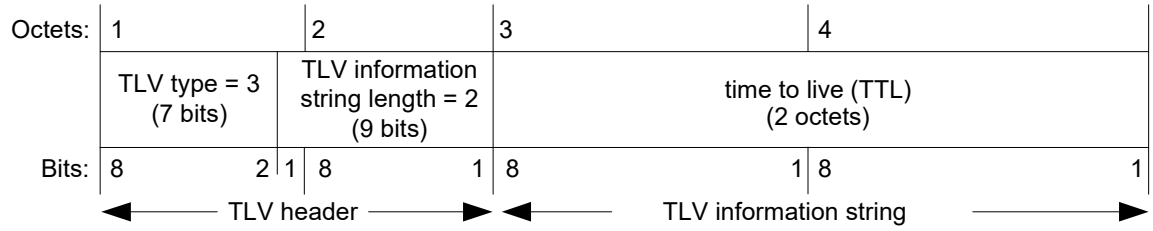


Figure 8-6—Time To Live TLV format

8.5.4.1 time to live (TTL)

The TTL field contains an integer value in the range $0 \geq t \geq 65535$ seconds and is set to the computed value of txTTL at the time the LLDPDU is constructed (see 9.2.5.22).

8.5.4.2 Time To Live TLV usage rules

A Normal LLDPDU contains exactly one Time To Live TLV as the third TLV, and does not contain more than one Time To Live TLV. A Time To Live TLV is not contained in an Extension LLDPDU or an Extension Request LLDPDU.

8.5.5 Port Description TLV

The Port Description TLV allows network management to advertise the IEEE 802 LAN station's port description. The format for this TLV is shown in Figure 8-7.

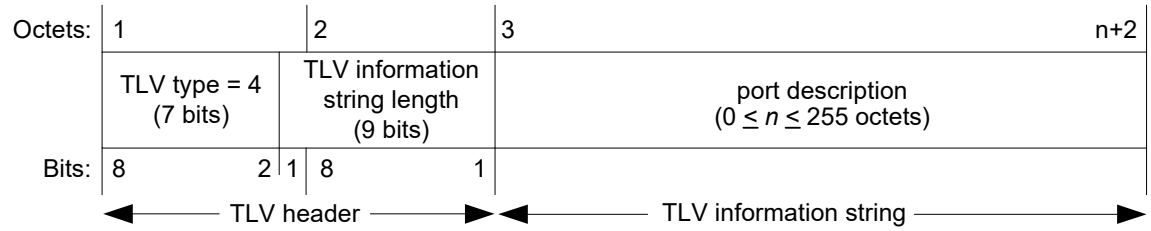


Figure 8-7—Port Description TLV format

8.5.5.1 TLV information string length

The TLV information string length field contains the exact length, in octets, of the port description field.

8.5.5.2 port description

The port description field contains an alpha-numeric string that indicates the port's description. If IETF RFC 2863 is implemented, the ifDescr object should be used for this field.

8.5.5.3 Port Description TLV usage rules

An LLDPDU should not contain more than one Port Description TLV.

8.5.6 System Name TLV

The System Name TLV allows network management to advertise the system's assigned name. The format for this TLV is shown in Figure 8-8.

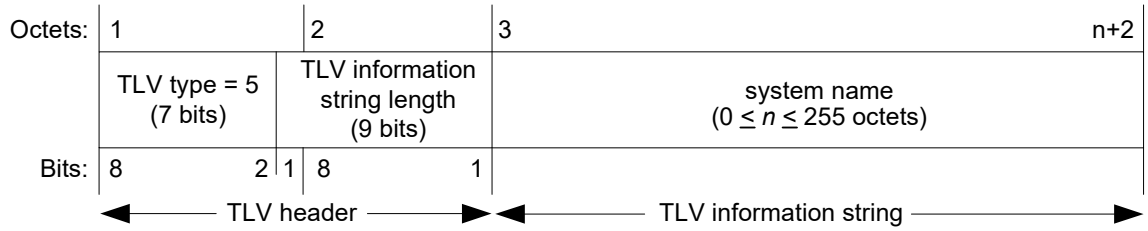


Figure 8-8—System Name TLV format

8.5.6.1 TLV information string length

The TLV information string length field contains the exact length, in octets, of the system name.

8.5.6.2 system name

The system name field contains an alpha-numeric string that indicates the system's administratively assigned name. The system name should be the system's fully qualified domain name. If implementations support IETF RFC 3418, the sysName object should be used for this field.

8.5.6.3 System Name TLV usage rules

An LLDPDU does not contain more than one System Name TLV.

8.5.7 System Description TLV

The System Description TLV allows network management to advertise the system's description. The format for this TLV is shown in Figure 8-9.

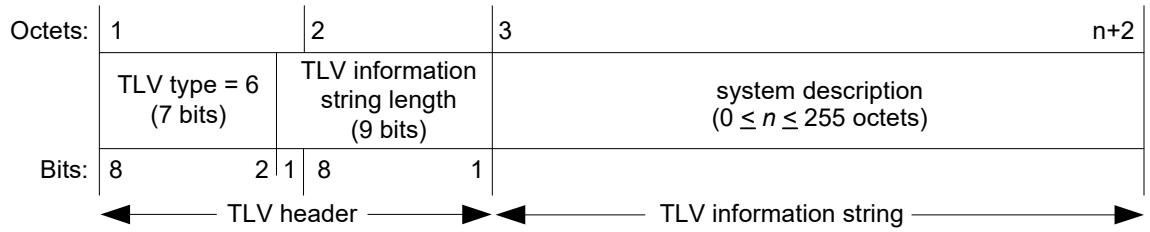


Figure 8-9—System Description TLV format

8.5.7.1 TLV information string length

The TLV information string length field indicates the exact length, in octets, of the system description.

8.5.7.2 system description

The system description field contains an alpha-numeric string that is the textual description of the network entity. The system description should include the full name and version identification of the system's hardware type, software operating system, and networking software. If implementations support IETF RFC 3418, the sysDescr object should be used for this field.

8.5.7.3 System Description TLV usage rules

An LLDPDU does not contain more than one System Description TLV.

8.5.8 System Capabilities TLV

The System Capabilities TLV is an optional TLV that identifies the primary function(s) of the system and whether or not these primary functions are enabled. Figure 8-10 shows the format of this TLV.

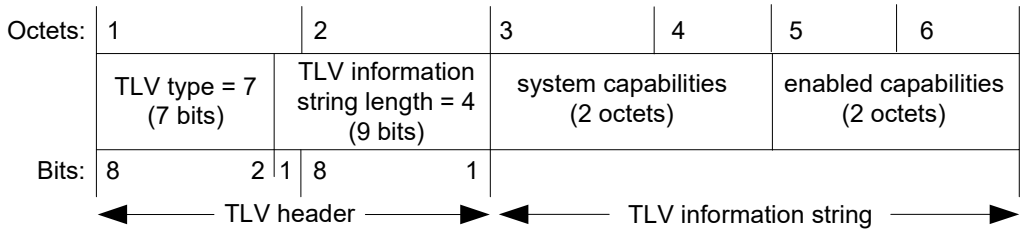


Figure 8-10—System Capabilities TLV format

8.5.8.1 system capabilities

The system capabilities field contains a bit-map of the capabilities that define the primary function(s) of the system. The bit positions for each function and the associated information repository or standard that are likely, but not guaranteed, to be supported are listed in Table 8-4. A binary one in the associated bit indicates the existence of that capability. Individual systems may indicate more than one implemented functional capability (for example, both a bridge and router capability).

Table 8-4—System capabilities

Bit	Capability	Reference
1	Other	—
2	Repeater	IETF RFC 2108 [B6]
3	MAC Bridge component	IEEE Std 802.1Q
4	802.11 Access Point (AP)	IEEE Std 802.11 MIB
5	Router	IETF RFC 1812 [B5]
6	Telephone	IETF RFC 4293 [B9]
7	DOCSIS cable device	IETF RFC 4639 and IETF RFC 4546 [B11]
8	Station Only [a]	IETF RFC 4293 [B9]
9	C-VLAN component	IEEE Std 802.1Q
10	S-VLAN component	IEEE Std 802.1Q
11	Two-port MAC Relay component	IEEE Std 802.1Q
12–16	reserved	—

[a]The Station Only capability is intended for devices that implement only an end station capability, and for which none of the other capabilities in the table apply. Bit 8 should therefore not be set in conjunction with any other bits.

8.5.8.2 enabled capabilities

The enabled capabilities field contains a bit map of the primary functions listed in Table 8-4. A binary one in a bit position indicates that the function associated with that bit is currently enabled.

8.5.8.3 System Capabilities TLV usage rules

An LLDPDU does not contain more than one System Capabilities TLV.

If the system capabilities field does not indicate the existence of a capability that the enabled capabilities field indicates is enabled, the TLV is interpreted as containing an error and is discarded.

8.5.9 Management Address TLV

The Management Address TLV identifies an address associated with the local LLDP agent that may be used to reach higher layer entities to assist discovery by network management. The TLV also provides room for the inclusion of both the system interface number and an object identifier (OID) that are associated with this management address, if either or both are known. Figure 8-11 shows the Management Address TLV format.

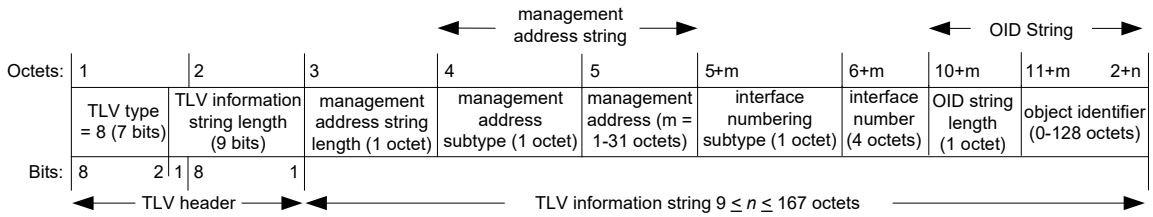


Figure 8-11—Management Address TLV format

8.5.9.1 TLV information string length

The TLV information string length field contains the length, in octets, of all the fields in the TLV information string.

8.5.9.2 management address string length

The management address string length field contains the length, in octets, of the (management address subtype + management address) fields.

8.5.9.3 management address subtype

The management address subtype field contains an integer value indicating the type of address that is listed in the management address field. Enumeration for this field is contained in the ianaAddressFamilyNumbers module of the IETF RFC 3232 on-line database that is accessible through a web page (http://www.iana.org). The management address subtype is contained in the first octet of the management address string.

NOTE—As a consequence of the limitation on the size of the management address field (a maximum of 31 octets), some options for the management address subtype (such as long DNS names) may be impractical.

8.5.9.4 management address

The management address field contains an octet string indicating the particular management address associated with this TLV.

a) The returned address should be the most appropriate for management use, typically a layer 3 address such as the IPv4 address, 192.0.2.10 (see also Table 8-2, footnote a).
b) If no management address is available, the return address should be the MAC address for the station or port. The ianaAddressFamilyNumber for a MAC address is all802 (value 6).

8.5.9.5 interface numbering subtype

The interface numbering subtype field contains an integer value indicating the numbering method used for defining the interface number. The following three values are currently defined:

1) Unknown
2) ifIndex
3) system port number

8.5.9.6 interface number

The interface number field contains the assigned number within the system that identifies the specific interface associated with this management address. If the value of the interface subtype is unknown, this field is set to zero.

8.5.9.7 object identifier (OID) string length

The object identifier string length field contains the length, in octets, of the OID. A value of zero in this field indicates that the OID field is not provided.

8.5.9.8 object identifier

The object identifier field contains an OID that identifies the type of hardware component or protocol entity associated with the indicated management address. The OID is the value portion of the ASN.1 encoding of the object identifier, encoded according to ASN.1 basic encoding rules [ISO/IEC 8824-1]. If no OID is available, this field is not provided.

NOTE—The interface number and OID are included in this TLV to assist NMS discovery by indicating Enterprise Specific or other starting points for the search, such as the Interface or Entity MIB (IETF RFC 6933).

8.5.9.9 Management Address TLV usage rules

Management Address TLVs are subject to the following:

a) At least one Management Address TLV should be included in every LLDPDU.
b) Since there are typically a number of different addresses associated with a MSAP identifier, an individual LLDPDU may contain more than one Management Address TLV.
c) When Management Address TLV(s) are included in an LLDPDU, the included address(es) should be the address(es) offering the best management capability.
d) If more than one Management Address TLV is included in an LLDPDU, each management address is different from the management address in any other management address TLV in the LLDPDU.
e) If an OID is included in the TLV, it is reachable by the management address.
f) In a properly formed Management Address TLV, the TLV information string length is equal to:

   (management address string length) + (OID string length) + 7.

   If the TLV information string length in a received Management Address TLV is incorrect, then it is ignored and processing of that LLDPDU is terminated.

8.6 Extended LLDP set TLV formats and definitions

8.6.1 Manifest TLV

The Manifest TLV allows an LLDP agent supporting XLLDP to advertise more TLVs than fits in a single LLDPDU. The Manifest TLV contains a list of Extension PDUs that convey the additional TLVs. The format for this TLV is shown in Figure 8-12.

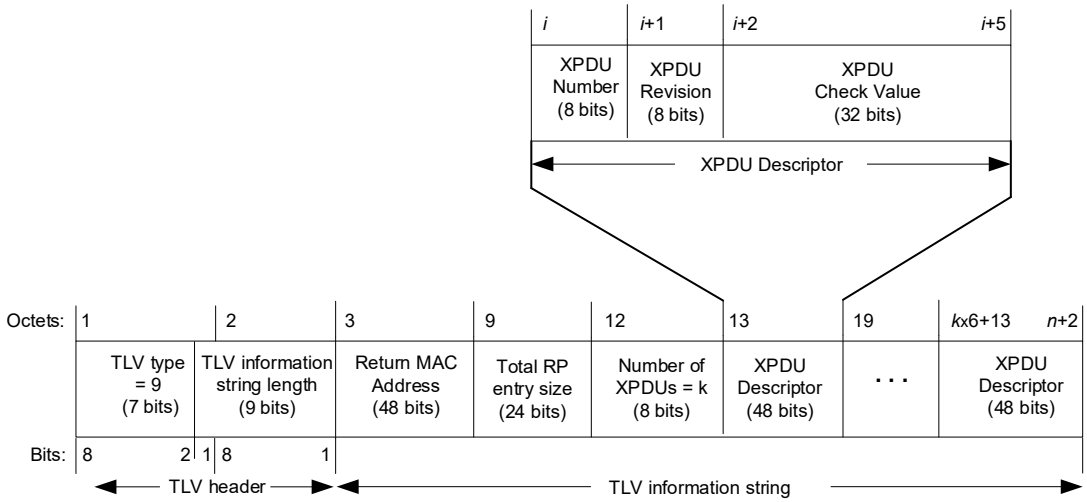


Figure 8-12—Manifest TLV format

8.6.1.1 TLV information string length

The TLV information string length field contains the number of octets in the TLV information string.

8.6.1.2 Return MAC Address

The individual MAC Address of the MSAP supporting the LLDP agent, to be used as the destination address for Extension Request LLDPDUs generated in response to this Manifest TLV.

8.6.1.3 Total information repository (RP) entry size

The total information repository entry size is the number of octets in all TLVs associated with this MSAP Identifier (regardless of the number of XPDUs used to convey those TLVs). It is the sum of the information string length fields in all TLVs, plus two times the number of TLVs, and includes the TLVs in the initial Normal LLDPDU. This allows the LLDP remote systems information repository manager to determine whether it expects to have space to store the information prior to requesting any XPDUs.

8.6.1.4 Number of XPDUs

The number of XPDUs field contains the count of the XPDU Descriptors in this TLV. Since the maximum size of the TLV information string is 511 octets, the minimum number of XPDU Descriptors is 1 and the maximum number is 83. When the number of XPDU Descriptors is k, the TLV information string length is at least $k \times 6 + 10$, but it need not be exactly $k \times 6 + 10$. This allows an implementation to use a fixed length TLV information string with a variable number of XPDU Descriptors.

8.6.1.5 XPDU Descriptor

An XPDU Descriptor comprises an 8-bit XPDU Number, an 8-bit XPDU Revision, and a 32-bit XPDU Check Value. The XPDU Number uniquely identifies each XPDU in a manifest. The XPDU Revision is assigned a random value at initialization, and is incremented (modulo 256) each time there is a change to one or more TLVs in the XPDU. The XPDU Check Value is the low order 32 bits of an MD5 hash calculated across all octets of the XPDU.

8.6.1.6 Manifest TLV usage rules

An LLDPDU does not contain more than one Manifest TLV. Each XPDU Descriptor associated with an MSAP identifier and Scope MAC Address have a unique XPDU Number.

8.6.2 Extension Request TLV

The Extension Request TLV allows an LLDP agent supporting XLLDP to request the neighbor to transmit specific Extension PDUs. The format for this TLV is shown in Figure 8-13.

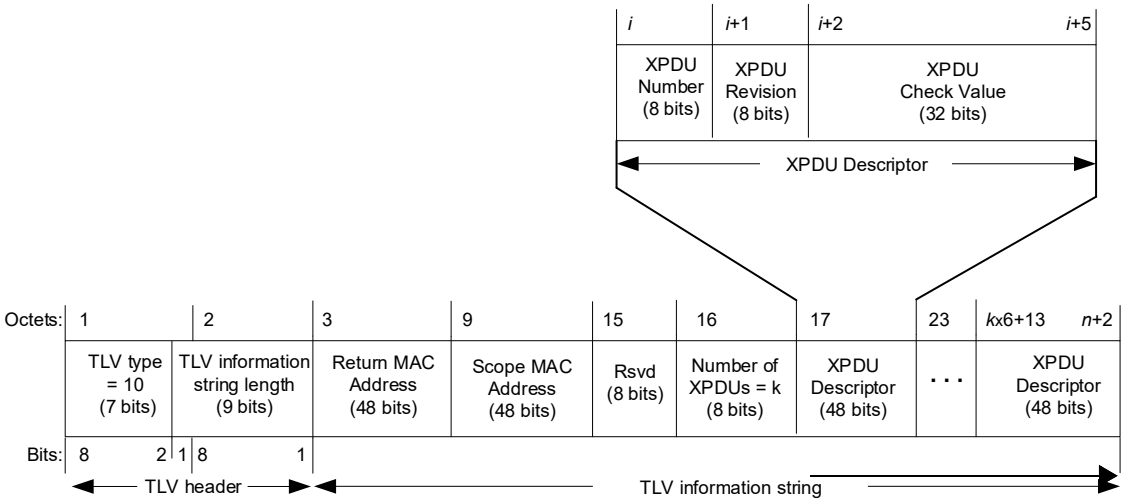


Figure 8-13—Extension Request TLV format

8.6.2.1 TLV information string length

The TLV information string length field contains the number of octets in the TLV information string.

8.6.2.2 Return MAC Address

The individual MAC address of the MSAP supporting the LLDP agent, to be used as the destination address for the requested Extension LLDPDUs.

8.6.2.3 Scope MAC Address

The MAC Address used by this LLDP agent as the destination address for Normal LLDPDUs.

8.6.2.4 Rsvd

The rsvd field is transmitted as zero and ignored on receipt.

8.6.2.5 Number of XPDUs

The number of XPDUs field contains the count of the XPDU Descriptors in this TLV. Since the maximum size of the TLV information string is 511 octets, the minimum number of XPDU Descriptors is 1 and the maximum number is 82. When the number of XPDU Descriptors is k, the TLV information string length is at least $k \times 6 + 14$, but it need not be exactly $k \times 6 + 14$. This allows an implementation to use a fixed length TLV information string with a variable number of XPDU Descriptors.

8.6.2.6 XPDU Descriptor

An XPDU Descriptor comprises an 8-bit XPDU Number, an 8-bit XPDU Revision, and a 32-bit XPDU Check, as specified in 8.6.1.5.

8.6.2.7 Extension Request TLV usage rules

An Extension Request LLDPDU contain an Extension Request TLV as the third TLV, and not contain more than one Extension Request TLV. An Extension Request TLV not be contained in a Normal LLDPDU or an Extension LLDPDU. Each XPDU Descriptor associated with a MSAP identifier and Scope MAC Address have a unique XPDU Number.

8.6.3 Extension Identifier TLV

The Extension Identifier TLV identifies an XPDU containing one or more TLVs. The format for this TLV is shown in Figure 8-11c.

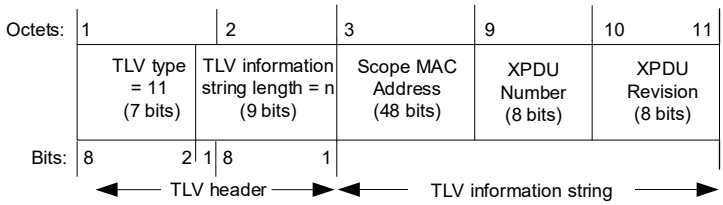


Figure 8-14—Extension Identifier TLV format

8.6.3.1 TLV information string length

The TLV information string length field contain the number of octets in the TLV information string.

8.6.3.2 Scope MAC Address

The MAC Address used by this LLDP agent as the destination address for Normal LLDPDUs.

8.6.3.3 XPDU Number

The XPDU Number portion of the XPDU Descriptor specified in 8.6.1.5.

8.6.3.4 XPDU Revision

The XPDU Revision portion of the XPDU Descriptor specified in 8.6.1.5.

8.6.3.5 Extension Identifier TLV usage rules

An Extension LLDPDU contains an Extension Identifier TLV as the third TLV, and does not contain more than one Extension Identifier TLV. An Extension Identifier TLV is not contained in a Normal LLDPDU or an Extension Request LLDPDU.

8.7 Organizationally Specific TLVs

This TLV category is provided to allow different organizations, such as IEEE 802.1, IEEE 802.3, IETF, as well as individual software and equipment vendors, to define TLVs that advertise information to remote entities attached to the same media, subject to the following restrictions:

a) Information transmitted in an Organizationally Specific TLV is intended to be a one way advertisement.

b) Information transmitted in an Organizationally Specific TLV is independent from information in a TLV received from a remote port.

c) Information transmitted in one Organizationally Specific TLV is not concatenated with information transmitted in another TLV on the same media in order to provide a means for sending messages that are larger than would fit within a single TLV.

d) Information received in an Organizationally Specific TLV is not explicitly forwarded to other ports in the system.

e) Organizationally Specific TLVs conform to 8.4 and 8.7.1.

Each set of Organizationally Specific TLVs includes associated LLDP MIB or YANG extensions and the associated TLV selection management variables and MIB/YANG to TLV cross reference tables. Systems that implement LLDP and that also support standard protocols for which Organizationally Specific TLV extension sets have been defined support all TLVs and the LLDP MIB or YANG extensions defined for that particular TLV set.

Annex D of IEEE Std 802.1Q-2022 and Clause 79 of IEEE Std 802.3-2022 contain Organizationally Specific TLVs for IEEE 802.1 standards and IEEE Std 802.3-2022, respectively, along with their associated LLDP MIB or YANG extensions and LLDP MIB or YANG to TLV cross reference tables. Organizations wishing to define TLVs for their use should use these annexes as examples.

8.7.1 Organizationally Specific TLV format

The format for Organizationally Specific TLVs is shown in Figure 8-15.

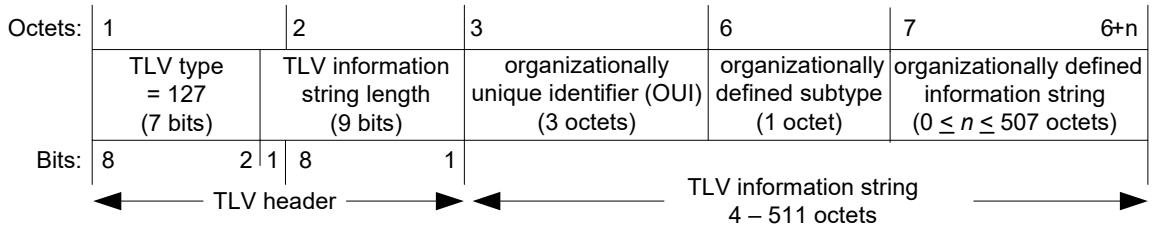


Figure 8-15—Format for Organizationally Specific TLVs

The following subclauses indicate how the individual fields are defined.

8.7.1.1 TLV type

The same Organizationally Specific TLV type value, 127, is used for all organizationally defined TLVs.

8.7.1.2 TLV information string length

The TLV information string length field contains the length of the information string, in octets.

8.7.1.3 organizationally unique identifier (OUI)

The organizationally unique identifier field contains the organization's OUI (see Clause 8 of IEEE Std 802-2024.

8.7.1.4 organizationally defined subtype

The organizationally defined subtype field contains a unique subtype value assigned by the defining organization.

NOTE—Defining organizations are responsible for maintaining listings of organizationally defined subtypes in order to assure uniqueness.

8.7.1.5 organizationally defined information string

The actual format of the organizationally defined information string field is organizationally specific and can

a) Contain either binary or alpha-numeric information that is instance specific for the particular TLV type and/or subtype. Alpha-numeric information should be encoded in UTF-8 (IETF RFC 3629).

b) Include one or more information fields with their associated field-type identifiers and field length designators similar to those in the Management Address TLV (see 8.5.9).

8.7.2 Organizationally Specific TLV usage rules

Organizations defining their own Organizationally Specific TLVs should include a subclause that defines any specific usage rules and/or specific conditions that affects how the receiving LLDP agent treats the TLV.

The Organizationally Specific TLV usage rules should include the following:

a) The number of Organizationally Specific TLVs that may be contained in an LLDPDU and any additional information field subtypes that would identify differences between two TLVs with the same OUI and organizationally defined subtype.

b) Any error conditions that are specific to the particular Organizationally Specific TLV and the action that is taken for each defined error condition.

9. LLDP agent operation

This clause describes the operation of the protocol from the point of view of a single LLDP agent (see Clause 6); i.e., it describes the operation of the protocol for the transmission and reception of LLDPDUs on a single Port. Where the use of more than one Scope MAC Address (7.1.1) is supported on a given Port, a separate instance of the LLDP agent is responsible for the operation of the protocol for each Scope MAC Address that is supported.

9.1 Overview

Each LLDP agent is responsible for causing the following tasks to be performed:

a) Maintaining current information in the LLDP local system information repository (RP).
b) Extracting and formatting LLDP local system information repository information for transmission to the remote port LLDP agent(s) at regular intervals or whenever there is a change in the system condition or status.
c) Recognizing and processing received LLDP frames.
d) Maintaining current values in the LLDP remote system information repository.
e) Using the procedure somethingChangedLocal() and the procedure somethingChangedRemote() to notify the PTOPO MIB manager and information repository managers of other optional MIBs and YANG modules whenever a value or status change has occurred in one or more objects in the LLDP local system LLDP remote systems information repository.

NOTE 1—A consequence of the support of multiple MAC addresses on a Port is that there are multiple copies of the LLDP local system information repository and remote system information repository, one copy for each address supported.

LLDP allows implementations that support three different operating modes: transmit only, receive only, or both transmit and receive. An LLDP agent shall conform to the specifications of each of the state machines indicated in Table 9-1 for the operating mode that it supports.

Table 9-1—Subclause/operating mode applicability

Subclause number	Subclause title	Transmit only	Receive only	Transmit and receive
9.2.10	Receive state machine	Mandatory if XLLDP is supported	Mandatory	Mandatory
9.2.9	Transmit state machine	Mandatory	—	Mandatory
9.2.11	Transmit timer state machine	Mandatory	—	Mandatory

Note 2—When XLLDP is supported and the operating mode is transmit only, the receive state machine is only enabled to recognize and respond to Extension Request LLDPDUs. Information advertised by remote LLDP agents is not stored in the LLDP remote systems information repository.

9.1.1 Frame transmission

An LLDPDU transmission cycle consists of the transmission of a Normal LLDPDU, potentially including a Manifest TLV if the implementation supports XLLDP, and the subsequent transmission of one or more Extension LLDPDUs upon receipt of Extension Request LLDPDUs. For an implementation not supporting XLLDP, the entire transmission cycle consists of the transmission of a single Normal LLDPDU.

The LLDP local system information repository contains the information needed for constructing individual TLVs and includes a capability that allows network managers to select the optional TLVs to be included in the Normal LLDPDU and, if XLLDP is supported, in one or more Extension LLDPDUs (see 10.2.2).

9.1.1.1 Normal LLDPDUs

The transmission of Normal LLDPDUs is performed by the transmit state machine, and the timing of transmission cycles is controlled by the transmit timer state machine. Transmissions occur as a result of a number of different local events, as follows:

a) A regular background transmission cycle occurs as a result of a transmission timer expiring. This prevents the receiving system from timing out the information received, as a result of the "time to live" associated with the last received LLDPDU expiring.

b) If a *new neighbor* is recognized (i.e., an LLDPDU is received by the receive state machine from a hitherto unknown remote LLDP agent), then a default of four LLDPDU transmission cycles are performed at shorter time intervals to quickly update the new neighbor with current information about its own neighbors. This rapid transmission behavior is suppressed if the remote systems information repository is not able to accommodate the new neighbor information without removing current information relating to an older neighbor, in order to prevent excessive PDU transmissions resulting from "churn" effects.

c) If a condition (status or value) change occurs in one or more objects in the LLDP local system information repository, then a transmission cycle is initiated immediately, in order to communicate the local changes as quickly as possible to any neighboring systems. The transmit timer state machine operates a simple credit-based transmission strategy that allows a number of transmission cycles in quick succession (the maximum number being determined by the maximum credit value), thus allowing rapid updating of information in situations where the local state is changing quickly. However, once the LLDP agent's transmission credit is exhausted, the rate of transmission is cut to an average of one transmission cycle per second.

d) The overall timing of regular transmission cycles is governed by a system "tick" that operates on a nominal timebase of 1 s. However, in shared media LANs, in order to avoid bunching of transmissions, the intervals between ticks include a random jitter component so that, while the average time interval between ticks is 1 s, the interval between adjacent pairs of ticks can vary (see 9.2.2).

In order to reduce synchronization effects, it is recommended that transmission cycle intervals for different LLDP agents on the same multi-port implementation are staggered.

When the transmit state machine (9.2.9) initiates a transmission cycle, the LLDP information repository manager extracts the selected information from the LLDP local system information repository and constructs a Normal LLDPDU containing the following:

a) The mandatory TLVs as specified in 8.2:
   1) Chassis ID TLV (see 8.5.2).
   2) Port ID TLV (see 8.5.3).
   3) Time To Live TLV, with the TTL value set equal to txTTL (see 8.5.4).

b) A Manifest TLV (see 8.6.1), if XLLDP is supported and any of the selected information is to be included in an Extension LLDPDU. The Manifest TLV includes a descriptor for each Extension LLDPDU, even if the information selected to be included in that Extension LLDPDU has not changed.

c) Additional optional TLVs, from the basic management set or from one or more organizationally specific sets, as allowed by LLDPDU length restrictions and as selected in the LLDP local system information repository by network management (see Table 8-1).

d) Optionally, an End Of LLDPDU TLV.

Optional TLVs from the basic management set, such as the Management Address TLV, with different TLV type and subtype combinations as well as optional TLVs with different organizationally unique identifier and organizationally defined subtype combinations from one or more organizationally specific sets can be included within the same LLDPDU.

9.1.1.2 Shutdown LLDPDUs

A special procedure exists for the case in which a LLDP agent knows an associated port is about to become non-operational (for example, the adminStatus for the port is transitioning to ‘disabled’). In the event a port, currently configured with LLDP frame transmission enabled, either becomes disabled for LLDP activity, or the interface is administratively disabled, the transmit state machine attempts to send a final LLDP shutdown LLDPDU with:

a) The Chassis ID TLV.
b) The Port ID TLV.
c) The Time To Live TLV with the TTL field set to zero.
d) Optionally, an End Of LLDPDU TLV.

The shutdown LLDPDU does not include any other optional TLVs and, if possible, should be transmitted before the interface is disabled.

NOTE—There is an inherent race condition between an interface knowing it is going down and its ability to send “one more frame.” If possible, the actual transition of the port to disabled is delayed until after this frame is transmitted. In the event where adminStatus is transitioning to the disabled state and the LLDP agent is shutting down, this shutdown procedure should be executed for all local ports.

9.1.1.3 Extension Request LLDPDUs

Extension Request LLDPDUs are transmitted in response to the reception of a Manifest LLDPDU. The transmission is controlled by the extended receive state machine. When XLLDP is supported, and a Manifest LLDPDU or an Extension LLDPDU is received, and the Manifest TLV includes XPDU Descriptors that do not match the XPDU Descriptors in the LLDP remote systems information repository entry for the MSAP Identifier received in the LLDPDU, the LLDP information repository manager constructs an Extension Request LLDPDU containing the following:

a) The mandatory TLVs as specified in 8.2:
    1) Chassis ID TLV (8.5.2).
    2) Port ID TLV (8.5.3).
    3) Extension Request TLV with a list of one or more XPDU descriptors (8.6.2).
b) Optionally, an End Of LLDPDU TLV.

9.1.1.4 Extension LLDPDUs

Extension LLDPDUs are transmitted in response to the reception of an Extension Request LLDPDU. The transmission is controlled by the receive state machine. When XLLDP is supported, and an Extension Request LLDPDU is received, then for each XPDU Number and XPDU Revision pair in the Extension Request TLV that matches an XPDU Number and XPDU Revision in the LLDP local system information repository, the LLDP information repository manager extracts the selected information from the LLDP local system information repository and constructs an Extension LLDPDU containing the following:

a) The mandatory TLVs as specified in 8.2:
    1) Chassis ID TLV (8.5.2).
    2) Port ID TLV (8.5.3).
    3) Extension Identifier TLV with the requested XPDU Number and XPDU Revision pair (8.6.3).

b) Additional optional TLVs, from the basic management set or from one or more organizationally specific sets, as allowed by LLDPDU length restrictions and as selected in the LLDP local system information repository by network management (Table 8-1).
c) Optionally, an End Of LLDPDU TLV.

9.1.2 Frame reception

LLDP frame reception consists of four phases: frame recognition, frame validation, TLV collection, and LLDP remote systems information repository updating.

9.1.2.1 Frame recognition

Frame recognition is performed at the LLDP/LSAP (see Figure 6-2), where the destination address and EtherType values determine whether the frame is an LLDP frame destined for this LLDP agent or not.

9.1.2.2 Frame validation

Frames that are recognized as LLDP frames are then validated to determine whether they are properly constructed and contain the correct set of mandatory TLVs. Frames that do not pass the validation criteria are discarded.

9.1.2.3 TLV collection

All received TLVs are validated by checking for format errors, and invalid TLVs are ignored. If the TLVs received in the Normal LLDPDU did not include a Manifest TLV (or the implementation does not support XLLDP and does not recognize the Manifest TLV), then all validated TLVs are passed to the LLDP remote systems information repository immediately. If the implementation supports XLLDP and the Normal LLDPDU includes a Manifest TLV, then this TLV is used to determine which Extension LLDPDUs need to be received before a complete set of TLVs can be passed to the LLDP remote systems information repository.

An implementation supporting XLLDP compares the XPDU descriptors in the Manifest TLV to those received in a previous Manifest TLV to determine which XPDUs (identified by the XPDU Number) contain new information. If no Manifest TLV has been received previously, then all of the XPDUs described in the Manifest TLV contain new information. If there are no XPDUs that have new information, then the validated TLVs received in the Normal LLDPDU are passed to the LLDP remote systems information repository. Otherwise these TLVs are retained until the XPDUs with new information have been received.

If there are XPDUs that have new information, then an Extension Request LLDPDU, requesting one or more of the XPDUs with new information, is transmitted to the LLDP agent that sourced the Normal LLDPDU. A timer is started when an Extension Request LLDPDU is transmitted, and the request is retried once if the timer expires before all requested XPDUs have been received. When all requested XPDUs have been received, another Extension Request LLDPDU is transmitted requesting one or more of the remaining XPDUs with new information. This process repeats until all XPDUs with new information have been received. If a new Manifest LLDPDU is received before all XPDUs have been received, any received XPDUs that have the same Revision and Checksum in the new Manifest TLV are retained, and any received XPDUs that have different Revision and Checksum are discarded. The extended receive state machine then continues to issue Extension Requests for any necessary XPDUs.

Received frames containing Extension LLDPDUs are recognized and validated following the same steps as frames containing Normal LLDPDUs. Once validated, the XPDU check value is calculated. This check value, together with the XPDU Number and XPDU Revision from the Extension Identifier TLV, is compared to the XPDU descriptors in the Manifest TLV. If there is a matching descriptor then all valid TLVs

received in this Extension LLDPDU are retained. When all XPDUs with new information have been received, all of the retained TLVs are passed to the LLDP remote systems information repository.

9.1.2.4 LLDP remote systems information repository updating

The collected TLVs are used to create, modify, or delete an entry in the LLDP remote systems information repository. The relevant entry in the LLDP remote systems information repository is determined by the originating MSAP identifier formed from the contents of the Chassis ID and Port ID TLVs. The LLDP remote systems information repository is updated as follows:

a) If the TTL value in the Time To Live TLV is greater than zero and an entry does not exist in the LLDP remote systems information repository for the originating MSAP identifier and there is sufficient space (or sufficient space can be created by the steps in 9.2) for the new information, then a new entry in the information repository is created.

b) If the TTL value in the Time To Live TLV is greater than zero and an entry already exists in the remote systems information repository for the originating MSAP identifier and there is sufficient space (or sufficient space can be created by the steps in 9.2) for the new information, then that entry is modified as follows:

   1) The TLVs received in the Normal LLDPDU are used to replace any information elements in the existing entry in the information repository that were previously received in a Normal LLDPDU. Note that if the Normal LLDPDU did not contain a Manifest TLV (or the implementation does not support XLLDP and does not recognize the Manifest TLV), then the new information contained in the LLDPDU is used to replace the whole of the existing entry in the information repository; i.e., if there are information elements in the existing entry for which there are corresponding elements in the received LLDPDU, then those elements are updated using the information received; any other information elements in the existing entry in the information repository are deleted.

   2) The TLVs received in an Extension LLDPDU are used to replace any information elements in the existing entry in the information repository that were previously received in an Extension LLDPDU with the same XPDU Number.

   3) Any information elements in the existing entry in the information repository that were previously received in an Extension LLDPDU with an XPDU Number that is not included in the received Manifest TLV are deleted.

c) If the TTL value in the Time To Live TLV is greater than zero and an entry exists in the LLDP remote systems information repository for the originating MSAP identifier and there is insufficient space for the new information, then the entry is deleted from the information repository.

d) If the TTL value in the Time To Live TLV is equal to zero and an entry exists in the LLDP remote systems information repository for the originating MSAP identifier, then the entry is deleted from the information repository.

When an entry in the LLDP remote systems information repository is created or modified (even if the modification is to replace all information elements with the same information), the rxInfoTTL timer associated with that entry is set to the TTL value from the Time To Live TLV. This determines how long the information associated with that MSAP identifier and destination MAC address is to be stored in the LLDP remote systems information repository before being aged out and deleted. The ageing out of old data purges the information repository of data that was originated by systems that are no longer neighbors as a result of topology changes, system failure, or system inactivation.

9.1.3 Too many neighbors

The amount of space needed in the LLDP remote systems information repository to accommodate the creation of several new information repository structures is beyond the scope of this standard. It may not always be possible to accommodate another new neighbor in implementations with limited memory. There are several possibilities for handling this case, including but not limited to the following examples:

a) Ignore and not process the new neighbor's information.
b) Delete the information from the oldest neighbor(s) until there is sufficient memory available to store the new neighbor's information.
c) Arbitrarily delete neighbors until there is sufficient memory available to store the new neighbor's information.

The method of handling the case where a new entry in the remote systems information repository can not be created is beyond the scope of this standard, however it is necessary for the implementation to keep track of this case by properly updating the variable tooManyNeighbors and the tooManyNeighborsTimer (9.2.5.16, 9.2.2). The variable tooManyNeighbors identifies when there is insufficient space in the LLDP remote systems information repository to store information from all active neighbors. The tooManyNeighborsTimer indicates the minimum time that this condition exists.

Further detail on the handling of the too many neighbors condition can be found in 9.2.8.5.1.

9.1.4 LLDP remote systems rxInfoTTL timer expiration

If the rxInfoTTL timer (9.2.2.1) associated with an MSAP identifier expires, all information associated with that MSAP identifier is deleted and the procedure somethingChangedRemote() notifies the managers of the PTOPO MIB and other optional MIB or YANG modules (see Figure 11-1) that something has changed in the LLDP remote systems information repository associated with that MSAP identifier.

9.1.5 LLDP local port/connection failure

If the local port or the connection to the remote system fails unexpectedly before a shutdown frame is received, the LLDP management entity does not delete objects in the LLDP remote systems information repository pertaining to information received from any MSAP identifier through that local port until the port is re-initialized or the associated rxInfoTTL timer (9.2.2.1) expires.

9.2 State machines

The operation of the protocol is represented by the following three state machines and associated timers to indicate when local system managed object values are to be transmitted to refresh the values in remote LLDP agent's LLDP remote systems information repository and when values in the local LLDP agent's remote systems information repository have become invalid:

a) Transmit state machine (9.2.9).
b) Receive state machine (9.2.10).
c) Transmit timer state machine (9.2.11).

9.2.1 Notational conventions used in state diagrams

State diagrams are used to represent the operation of a function as a group of connected, mutually exclusive states. Only one state of a function can be active at any given time.

Each state is represented in the state diagram as a rectangular box, divided into two parts by a horizontal line. The upper part contains the state identifier, written in uppercase letters. The lower part contains any procedures that are executed on entry to the state.

All permissible transitions between states are represented by arrows, the arrowhead denoting the direction of the possible transition. Labels attached to arrows denote the condition(s) that shall be met in order for the transition to take place. A transition that is global in nature (i.e., a transition that occurs from any of the possible states if the condition attached to the arrow is met) is denoted by an open arrow; i.e., no specific state is identified as the origin of the transition.

On entry to a state, the procedures defined for the state (if any) are executed exactly once, in the order that they appear on the page. Each action is deemed to be atomic; i.e., execution of a procedure completes before the next sequential procedure starts to execute. No procedures execute outside of a state block. On completion of all of the procedures within a state, all exit conditions for the state (including all conditions associated with global transitions) are evaluated continuously until such a time as one of the conditions is met. All exit conditions are regarded as Boolean expressions that evaluate to TRUE or FALSE; if a condition evaluates to TRUE, then the condition is met. When the condition associated with a global transition is met, it supersedes all other exit conditions, including UCT. The label UCT denotes an unconditional transition (i.e., UCT always evaluates to TRUE). The label ELSE denotes a transition that occurs if none of the other conditions for transitions from the state are met (i.e., ELSE evaluates to TRUE if all other possible exit conditions from the state evaluate to FALSE).

A variable that is set to a particular value in a state block retains this value until a subsequent state block executes a procedure that modifies the value, or until the value is modified by an external event or timer tick.

Where it is necessary to segment a state machine description across more than one diagram, a transition between two states that appear on different diagrams is represented by an exit arrow drawn with dashed lines, plus a reference to the diagram that contains the destination state. Similarly, dashed arrows and a dashed state box are used on the destination diagram to show the transition to the destination state. In a state machine that has been segmented in this way, any global transitions that can cause entry to states defined in one of the diagrams are deemed to be potential exit conditions for all of the states of the state machine, regardless of which diagram the state boxes appear in.

Should a conflict exist between the interpretation of a state diagram and either the corresponding global transition tables or the textual description associated with the state machine, the state diagram takes precedence.

The interpretation of the special symbols and operators used in the state diagrams is defined in Table 9-2; these symbols and operators are derived from the notation of the “C” programming language.

Table 9-2—State machine symbols

Symbol	Interpretation
( )	Used to force the precedence of operators in Boolean expressions and to delimit the argument(s) of actions within state boxes.
;	Used as a terminating delimiter for actions within state boxes. Where a state box contains multiple actions, the order of execution follows the normal English language conventions for reading text.

Table 9-2—State machine symbols *(continued)*

Symbol	Interpretation
=	Assignment action. The value of the expression to the right of the operator is assigned to the variable to the left of the operator. Where this operator is used to define multiple assignments, e.g., a = b = X the action causes the value of the expression following the right-most assignment operator to be assigned to all of the variables that appear to the left of the right-most assignment operator.
!	Logical NOT operator.
&&	Logical AND operator.
||	Logical OR operator.
if...then...	Conditional action. If the Boolean expression following the **if** evaluates to TRUE, then the action following the **then** is executed.
{statement 1, ... statement N}	Compound statement. Braces are used to group statements that are executed together as if they were a single statement.
!=	Inequality. Evaluates to TRUE if the expression to the left of the operator is not equal in value to the expression to the right.
==	Equality. Evaluates to TRUE if the expression to the left of the operator is equal in value to the expression to the right.
<	Less than. Evaluates to TRUE if the value of the expression to the left of the operator is less than the value of the expression to the right.
<=	Less than or equal to. Evaluates to TRUE if the value of the expression to the left of the operator is either less than or equal to the value of the expression to the right.
>	Greater than. Evaluates to TRUE if the value of the expression to the left of the operator is greater than the value of the expression to the right.
>=	Greater than or equal to. Evaluates to TRUE if the value of the expression to the left of the operator is either greater than or equal to the value of the expression to the right.
+	Arithmetic addition operator.
–	Arithmetic subtraction operator.

9.2.2 Timers

A set of timers is used by the LLDP state machines; these operate as countdown timers (i.e., they expire when their value reaches zero). These timers

a) Have a resolution of one tick.
b) Have an integer value n, where $0 < n < 65535$ tick intervals.
c) Are started by loading an initial integer value.
d) Are decremented once per timer tick as long as $n > 0$.
e) Represent the remaining time in the period.

For Ports connected to point-to-point LANs, the interval between timer ticks is 1 s. For Ports connected to shared media LANs, the interval between any pair of timer ticks is 0.8 s, plus a “jitter” component that is a random or pseudo-random value between 0.0 s and 0.4 s. Hence, the average interval between timer ticks is 1 s; however, the interval between any pair of ticks varies randomly. When a timer tick occurs, all instances of the variable txTick (9.2.5.22) for the Port are set TRUE.

NOTE—The random component of the tick time used in shared media LANs is intended to minimize the possibility of systems connected to the same LAN becoming synchronized in their transmission timings.

9.2.2.1 rxInfoTTL

An instance of this timer exists for each entry in the LLDP remote systems information repository that has been created by this instance of the receive state machine for a given MSAP identifier. The timer value indicates the number of seconds remaining until the information in all the objects in the LLDP remote systems information repository associated with the MSAP identifier is no longer valid and needs to be deleted.

9.2.2.2 tooManyNeighborsTimer

This timer indicates the number of seconds remaining during which it is known that an LLDP neighbor exists that may not be stored in the remote systems information repository.

9.2.2.3 txTTR

An instance of this timer exists for each Agent. The txTTR timer is used to determine the next LLDPDU transmission is due; it is initialized by the transmit timer state machine, with the value msgTxInterval (9.2.5.6) if txFast (9.2.5.19) is equal to zero, or with the value msgFastTx (9.2.5.4) if txFast is greater than zero.

9.2.2.4 txShutdownWhile

An instance of this timer exists for each Agent. The txShutdownWhile timer indicates the number of seconds remaining until LLDP re-initialization can occur.

9.2.2.5 rxTimer

An instance of this timer exists for each extended receive state machine to impose a time limit on each extension request, and on the overall TLV collection process.

9.2.3 Per-System variables

An instance of each of these variables exists per system; they are available for use by all of the state machines of all of the LLDP agents in the system.

9.2.3.1 BEGIN

This Boolean variable is controlled by the system initialization process. A value of TRUE causes all state machines to continuously execute their initial state. A value of FALSE allows all state machines to perform transitions out of their initial state, in accordance with the relevant state machine definitions.

9.2.4 Per-Port variables

An instance of each of these variables exists per Port; they are available to all of the state machines associated with a Port.

9.2.4.1 portEnabled

This variable is externally controlled. Its value reflects the operational state of the MAC service supporting the port. Its value is TRUE if the MAC service supporting the Port is in an operable condition; otherwise, it is FALSE.

9.2.5 Per-Agent variables

An instance of each of these variables exists per LLDP agent; they are available to all of the state machines associated with an LLDP agent.

9.2.5.1 adminStatus

This variable indicates whether or not the LLDP agent is enabled. The defined values for this variable are as follows:

a) enabledRxTx: The LLDP agent is enabled for reception and transmission of LLDPDUs. This is the recommended default value.
b) enabledTxOnly: The LLDP agent is enabled for transmission of LLDPDUs only.
c) enabledRxOnly: The LLDP agent is enabled for reception of LLDPDUs only.
d) disabled: The LLDP agent is disabled for both reception and transmission.

9.2.5.2 localChange

This variable is set TRUE by the somethingChangedLocal() procedure (see 9.2.8.8) when there is a change in the value of the LLDP local system information repository associated with this LLDP agent. The variable is set FALSE by the operation of the transmit timer state machine (see 9.2.11).

9.2.5.3 Scope MAC Address

The MAC address associated with this instance of the LLDP agent that defines the scope of propagation of Normal LLDPDUs in the network. It is used as the destination MAC address when the LLDP agent transmits Normal LLDPDUs, and the destination MAC address that, when present in a received LLDPDU, causes the LLDPDU to be passed to this LLDP agent instance for processing.

9.2.5.4 msgFastTx

This variable defines the time interval in timer ticks between transmissions during fast transmission periods (i.e., txFast is non-zero). The recommended default value of msgFastTx is 1; this value can be changed by management to any value in the range 1 through 3600.

9.2.5.5 msgTxHold

This variable is used, as a multiplier of msgTxInterval, to determine the value of txTTL that is carried in LLDP frames transmitted by the LLDP agent. The recommended default value of msgTxHold is 4; this value can be changed by management to any value in the range 2 through 10.

9.2.5.6 msgTxInterval

This variable defines the time interval in timer ticks between transmissions during normal transmission periods (i.e., txFast is zero). The recommended default value for msgTxInterval is 30 s; this value can be changed by management to any value in the range 1 through 3600.

9.2.5.7 newNeighbor

This variable is set TRUE by the rxProcessFrame() procedure (see 9.2.8.7) when information is received from a new neighbor. It is set FALSE by the Transmit timer state machine (9.2.11).

NOTE—The newNeighbor variable is used to force a more rapid regular transmission rate when a new neighbor appears, to ensure that the new neighbor also receives new information from the receiving system.

9.2.5.8 rcvFrame

This variable indicates that an LLDP frame has been recognized by the LLDP LSAP function and is ready to be processed by the LLDP receive state machine. If both of the following conditions are TRUE, the variable rcvFrame is set to TRUE and the frame is made available to the receive state machine for validation and processing:

a) The destination address value is the Scope MAC Address associated with this specific LLDP agent or the individual MAC address of the MSAP supporting the LLDP agent.

b) The EtherType value carried by the frame is the LLDP EtherType defined in Table 7-3.

NOTE—The model that is assumed for reception is that incoming LLDPDUs are presented to all LLDP agents associated with the reception Port, and if the Scope MAC Address is not one that is associated with a given LLDP agent, that LLDPDU is ignored by that LLDP agent. In practice, at most one LLDP agent processes any received LLDPDU, as there is only one LLDP agent associated with any given Scope MAC Address on a given reception Port.

9.2.5.9 reinitDelay

This parameter indicates the amount of delay from when adminStatus becomes ‘disabled’ until re-initialization is attempted. The recommended default value for reinitDelay is 2 s.

9.2.5.10 remoteChanges

A Boolean indication of whether the status/value of one or more selected objects in the LLDP remote systems information repository has changed or that all objects associated with a particular MSAP identifier have been deleted. This variable takes the value *(rxInfoAge OR (rxChanges && (rxTTL !=0)))*

9.2.5.11 rxChanges

This variable indicates that the incoming LLDPDU has been received with different TLV values from those currently in the LLDP remote systems information repository associated with the LLDPDU’s MSAP identifier.

9.2.5.12 rxInfoAge

This variable is set TRUE when a timer expiry event occurs for the rxInfoTTL timing counter associated with a particular MSAP identifier in the LLDP remote systems information repository (i.e., the counter transitions from non-zero to zero). It is set FALSE by the operation of the Receive state machine.

9.2.5.13 rxTTL

This variable indicates the time to live value associated with all TLVs received in the current frame.

9.2.5.14 rxType

This variable indicates the type of the most recently received LLDP frame. It is set by the rxProcessFrame() procedure to one of the following values:

a) xreq: The LLDP agent supports XLLDP and identifies the frame as containing a request for extended PDUs.
b) xpdu: The LLDP agent supports XLLDP and identifies the frame as containing an extended PDU.
c) shutdown: The frame contains a shutdown LLDPDU (which has an rxTTL value equal to zero).
d) manifest: The LLDP agent supports XLLDP and the frame is a Normal LLDPDU that has a rxTTL value greater than zero and contains a Manifest TLV.

e) normal: The frame contains a Normal LLDPDU that has an rxTTL value greater than zero, and either the LLDP agent does not support XLLDP or the frame does not contain a Manifest TLV.
f) invalid: The frame is improperly formatted for a valid LLDPDU, or the frame contains an extended PDU or a request for extended PDUs but the LLDP agent does not support XLLDP.

9.2.5.15 sentManifest

This Boolean variable is set to TRUE when the transmit state machine generates a Manifest LLDPDU, and is set to FALSE when the transmit state machine generates a Normal LLDPDU that does not contain a Manifest TLV. It is used to enable the receive state machine to recognize Extension Request LLDPDUs and respond with Extension LLDPDUs.

9.2.5.16 tooManyNeighbors

This Boolean variable indicates that there is insufficient space in the LLDP remote systems information repository to store information from all connected active remote ports (see 9.2.8.5.1).

9.2.5.17 txCredit

The number of consecutive LLDPDUs that can be transmitted at any time. This variable is incremented by the txAddCredit() procedure, up to the maximum value specified by the txCreditMax variable (see 9.2.5.18), and is decremented whenever an LLDPDU is transmitted by the transmit state machine.

9.2.5.18 txCreditMax

The maximum value of txCredit. The recommended default value is 5; this value can be changed by management to any value in the range 1 through 10.

9.2.5.19 txFast

This variable is used as a down counter to count the number of transmissions to be made during a fast transmission period. Fast transmission periods are initiated when a new neighbor is detected, and cause LLDPDU transmissions to take place at shorter time intervals than during normal (steady state) operation of the protocol. The fast transmission period ensures that more than one LLDPDU is transmitted when a new neighbor is detected. The first transmission is immediate; the subsequent transmissions occur at intervals determined by msgFastTx. The number of transmissions is determined by the value of txFastInit (see 9.2.5.20).

9.2.5.20 txFastInit

This variable is used as the initial value for the txFast variable (see 9.2.5.19). This value determines the number of LLDPDUs that are transmitted during a fast transmission period. The recommended default value of txFastInit is 4; this value can be changed by management to any value in the range 1 through 8.

9.2.5.21 txNow

This variable is used to signal to the transmit state machine that a transmission is required. It is set TRUE by the transmit timer state machine, and set FALSE by the transmit state machine when the transmission has been initiated.

9.2.5.22 txTick

This variable is set TRUE by the system timer tick at an average interval of one second (see 9.2.2). It is set FALSE by the operation of the transmit timer state machine.

9.2.5.23 txTTL

This variable indicates the time remaining before information in the outgoing LLDPDU is no longer valid. The TTL field in the Time To Live TLV is set to txTTL during LLDPDU construction (see 9.1.2). The value of txTTL depends on the following:

a) During normal operation, txTTL is set to whichever is the smaller of the values represented by Equation (1) and Equation (2):

$$(\text{msgTxInterval} \times \text{msgTxHold}) + 1 \quad (1)$$

$$65535 \quad (2)$$

where msgTxInterval is defined in 9.2.5.6 and msgTxHold is defined in 9.2.5.5.

b) If adminStatus is transitioning to ‘disabled’ or portEnabled is transitioning to FALSE, txTTL shall be set to zero.

9.2.6 Per-Neighbor variables

An instance of each of these variables exists per neighbor (identified by an MSAP) for each LLDP agent; they are available to all of the state machines associated with an LLDP agent.

9.2.6.1 createExtRx

This Boolean is used to initialize a per-neighbor extended receive state machine. It is set to TRUE when an extended receive state machine is created in response to the receipt of a Manifest LLDPDU from a neighbor for which an extended receive state machine does not already exist. It is set to FALSE by the extended receive state machine following initialization.

9.2.6.2 rxExtended

This Boolean is used to pass control between the per-agent receive state machine and the per-neighbor extended receive state machine. It is set to TRUE by the receive state machine when an LLDPDU containing a Manifest TLV or an Extension Identifier TLV is received, and is set to FALSE by the extended receive state machine.

9.2.6.3 manifestComplete

This Boolean indicates to the receive state machine whether the extended receive state machine has received all TLVs necessary to update the information for this MSAP in the LLDP remote systems information repository.

9.2.6.4 xreqComplete

This Boolean is used within the extended receive state machine to indicate that all XPDUs requested for this MSAP have been received.

9.2.6.5 rxTick

This Boolean is set TRUE by the system timer tick at an average interval of one second (see 9.2.2). It is set FALSE by the operation of the extended receive state machine.

9.2.6.6 rxTTLExpired

This Boolean is used to indicate that the extended receive state machine has been unable to collect all the TLVs within a time interval greater than the received rxTTL value. It is set to TRUE when the rxTimer is decremented to zero and either all Extension LLDPDUs for the most recently transmitted Extension Request LLDPDU have been received, or that Extension Request LLDPDU was a retry of a previous request.

9.2.7 Per-Agent statistical counters

An instance of each of these statistical counters exists per LLDP agent; they are used to count significant events in the transmit and receive state machines and are made available to the management functions described in Clause 10 and Clause 11. All counters are maintained as unsigned integer values, four octets in length, and are initialized to zero only on system initialization (i.e., when the BEGIN system variable is set TRUE).

9.2.7.1 statsAgeoutsTotal

This counter provides a count of the times that a neighbor's information has been deleted from the LLDP remote systems information repository because the rxInfoTTL timer associated with the neighbor's MSAP has expired.

9.2.7.2 statsFramesDiscardedTotal

This counter provides a count of all LLDPDUs received and then discarded for any of the following reasons:

a) One or more of the three mandatory TLVs at the beginning of the LLDPDU is missing, out of order, or contains an out of range information string length.
b) There is insufficient space in the remote systems information repository to store the LLDPDU.

9.2.7.3 statsFramesInErrorsTotal

This counter provides a count of all LLDPDUs received with one or more detectable errors.

9.2.7.4 statsFramesInTotal

This counter provides a count of all LLDP frames received.

9.2.7.5 statsFramesOutTotal

This counter provides a count of all LLDP frames transmitted by this instance of the transmit state machine. The counter shall be maintained as an unsigned integer value, four octets in length.

9.2.7.6 statsTLVsDiscardedTotal

This counter provides a count of all TLVs received and then discarded for any reason.

9.2.7.7 statsTLVsUnrecognizedTotal

This counter provides a count of all TLVs received on the port that are not recognized by the LLDP local agent.

9.2.7.8 lldpduLengthErrors

A count of the number of LLPDU length restriction errors detected by the rpConstrInfoLLDPDU() procedure (9.2.8.2).

9.2.8 State machine procedures

9.2.8.1 dec (variable-name)

If the state machine variable *variable-name* has a non-zero value, this procedure decrements the value of the variable by 1.

9.2.8.2 rpConstrInfoLLDPDU()

The rpConstrInfoLLDPDU() procedure constructs an information LLDPDU, structured according to the specification in 8.2, the associated basic TLV formats defined in 8.4, plus any optional Organizationally Specific TLVs as specified and their associated individual organizationally defined formats (see 8.7). The information LLDPDU contains the following:

a) The set of mandatory TLVs as specified in 8.2, with the value of txTTL set in accordance with the definition in 9.2.5.23.
b) Additional optional TLVs, from the basic management set or from one or more organizationally specific sets, as allowed by LLDPDU length restrictions and as selected in the LLDP local system information repository by network management (see Table 8-1).
c) Optionally, an End Of LLDPDU TLV.

If the set of TLVs that is selected in the LLDP local system information repository by network management would result in an LLDPDU that violates LLDPDU length restrictions and XLLDP is supported, each TLV selected for transmission is mapped to the Normal LLDPDU or to an Extension LLDPDU, as defined in 10.2.2.

If the set of TLVs that is selected in the LLDP local system information repository by network management would result in an LLDPDU that violates LLDPDU length restrictions and XXLDP is not supported, then the variable lldpduLengthErrors (9.2.8.2) is incremented by 1, and an LLDPDU is sent containing the mandatory TLVs plus as many of the optional TLVs in the set as will fit in the remaining LLPDU length.h.

9.2.8.3 rpConstrShutdownLLDPDU()

The rpConstrShutdownLLDPDU() procedure constructs a shutdown LLDPDU according to the LLDPDU and the associated TLV formats specified in 8.2 and 8.5. The shutdown LLDPDU contains only the following items:

a) The Chassis ID and Port ID TLVs.
b) The Time To Live TLV with the TTL field set to zero.
c) Optionally, an End Of LLDPDU TLV.

9.2.8.4 rpDeleteObjects()

The rpDeleteObjects() procedure deletes all information in the LLDP remote systems information repository associated with a given MSAP identifier (and destroys the associated extended receive state machine if one exists) if an LLDPDU is received with an rxTTL value of zero (see 9.2.8.7.1), the extended receive state machine sets rxTTLExpired to TRUE, or the timing counter rxInfoTTL expires (see 9.2.2.1).

9.2.8.5 rpUpdateObjects()

The rpUpdateObjects() procedure updates the information repository objects corresponding to the TLVs contained in the received Normal LLDPDU, and all TLVs contained in any received Extension LLDPDUs, for the LLDP remote system if information repository space is available, as follows:

a) Compare the MSAP identifier in the current LLDPDU with the MSAP identifiers in the LLDP remote systems information repository:
   1) If a match is found, but no difference exists between the information in the TLVs just received and the information in the LLDP remote systems information repository, then set the control variable rxChanges to FALSE, and set the timing counter rxInfoTTL associated with the MSAP identifier to rxTTL.
   2) If a match is found and sufficient space is not available in the LLDP remote systems information repository for the updated information in the TLVs just received, then follow the tooManyNeighbors procedure (9.2.8.5.1) to determine whether to discard the TLVs from a new neighbor or to delete information from an existing neighbor that is already in the LLDP remote systems information repository.
   3) If a match is found, and sufficient space is available (or has been made available) in the LLDP remote systems information repository for the updated information in the TLVs just received, then:
      i) Replace all information associated with the MSAP identifier in the LLDP remote systems information repository that was previously received in a Normal LLDPDU with the information in the just received Normal LLDPDU.
      ii) For each received Extension LLDPDU associated with the MSAP identifier, if any, replace all information in the LLDP remote systems information repository that was previously received in an Extension LLDPDU of the same XPDU Number with the information in the just received Extension LLDPDU. If a received Extension LLDPDU has an XPDU Number that has not been previously received, add the new information to the LLDP remote system information repository entry.
      iii) Set the control variable rxChanges to TRUE.
      iv) Set the timing counter rxInfoTTL associated with the MSAP identifier to rxTTL.
   4) If no match is found and sufficient space is not available in the LLDP remote systems information repository, then follow the tooManyNeighbors procedure (9.2.8.5.1) to determine whether to discard the TLVs from a new neighbor or to delete information from an existing neighbor that is already in the LLDP remote systems information repository.
   5) If no match is found and sufficient space is available (or has been made available), create a new information repository structure to receive information associated with the new MSAP identifier, set these information repository objects to the values indicated in their respective TLVs, set the control variable rxChanges to TRUE, and set the timing counter rxInfoTTL associated with the MSAP identifier to rxTTL.

If an incoming TLV is not recognized by the receiving LLDP agent, the TLV is stored in the LLDP remote systems information repository as follows:

b) If the TLV type value is in the range of the reserved TLV types in Table 8-1, the TLV can be from a later version of the basic management set and is stored according to the basic TLV format shown in Figure 8-2. These TLVs are indexed by their TLV type.

c) If the TLV type value is 127, the TLV is an Organizationally Specific TLV and is stored according to the basic format for Organizationally Specific TLVs shown in Figure 8-15. These TLVs are indexed by their organizationally unique identifier and organizationally defined TLV subtype.

9.2.8.5.1 Too many neighbors

When there is insufficient space in the LLDP remote systems information repository to store information from a new neighbor something needs to be discarded. Either the received LLDPDU is discarded or existing information within the current remote systems information repository is discarded in order to make space for the new information received in the LLDPDU. If the space required to store the information from the new neighbor is larger than the storage space available to the remote systems information repository then the received LLDPDU shall be discarded rather than discarding existing infromation in the remote systems information repository. The procedure is as follows:

a) Set the flag variable tooManyNeighbors to TRUE.
b) If the information selected to be discarded is the information in the current LLDPDU:
    1) Set the tooManyNeighborsTimer as specified in Equation (3).

$$\text{tooManyNeighborsTimer} = \max(\text{tooManyNeighborsTimer}, \text{rxTTL}) \quad (3)$$

    where rxTTL is the TTL value in the current LLDPDU
    2) Discard the current LLDPDU and increment the statsFramesDiscardedTotal counter.
    3) Set the control variable rxChanges to FALSE. Wait for the next LLDPDU.
c) If the information selected to be discarded is currently in the LLDP remote systems information repository:
    1) From the data in the remote information repository, select a neighbor.
    2) Delete all information associated with the selected neighbor's MSAP identifier from the LLDP remote systems information repository, and destroy the associated extended receive state machine if one exists.
    3) Set the tooManyNeighborsTimer as specified in Equation (4).

$$\text{tooManyNeighborsTimer} = \max(\text{tooManyNeighborsTimer}, \text{rxInfoTTL}) \quad (4)$$

    where rxInfoTTL = the selected neighbor's TTL value
    4) If sufficient space exists to store the information received in the current LLDPDU in the LLDP remote systems information repository, set control variable rxChanges to TRUE.
    5) If sufficient space still does not exist to store the information received in the current LLDPDU in the LLDP remote systems information repository, perform steps 1–4 again.

The variable tooManyNeighbors is automatically set to FALSE whenever the tooManyNeighborsTimer expires.

NOTE—The tooManyNeighborsTimer is not precise, as it is only cleared (and the tooManyNeighbors variable set FALSE) by timer expiration, and not by the removal of the too many neighbors condition.

9.2.8.6 rxInitializeLLDP()

The rxInitializeLLDP() procedure initializes the LLDP receive module. The rxInitializeLLDP() procedure waits till portEnabled is equal to TRUE and after the variable portEnabled is equal to TRUE, the LLDP receive module is initialized or re-initialized for frame reception. During this process, the local LLDP agent performs the following tasks:

a) The variable tooManyNeighbors is set to FALSE.
b) All information in the remote systems information repository associated with this LLDP agent is deleted.

9.2.8.7 rxProcessFrame()

The rxProcessFrame() procedure:

a) Validates the LLDPDU, sets the value of rxType, and validate the TLVs contained in the LLDPPU as specified in 9.2.8.7.1.

9.2.8.7.1 LLDPDU validation

The receive module processes each incoming LLDPDU as it is received. The statsFramesInTotal counter for the port is incremented and the LLDPDU is checked to verify the presence of the three mandatory TLVs at the beginning of the LLDPDU as defined in 8.2.

a) The first TLV is extracted:
   1) If the extracted TLV type value does not match the Chassis ID TLV type:
      i) The LLDPDU is discarded.
      ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
      iii) The rxType value is set equal to invalid.
      iv) The procedure rxProcessFrame**()** is terminated.
   2) If the extracted TLV type value matches the Chassis ID TLV type, and the chassis ID TLV information string length is not within the range $2 \leq n \leq 256$:
      i) The LLDPDU is discarded.
      ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
      iii) The rxType value is set equal to invalid.
      iv) The procedure rxProcessFrame() is terminated.
   3) If the extracted TLV type value matches the Chassis ID TLV type, and the chassis ID TLV information string length is within the range $2 \leq n \leq 256$, the chassis ID value is extracted to become the first part of the MSAP identifier.
b) The second TLV is extracted:
   1) If the extracted TLV type value does not match the Port ID TLV type:
      i) The LLDPDU is discarded.
      ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
      iii) The rxType value is set equal to invalid.
      iv) The procedure rxProcessFrame() is terminated.
   2) If the extracted TLV type value matches the Port ID TLV type, and the port ID TLV information string length is not within the range $2 \leq n \leq 256$:
      i) The LLDPDU is discarded.
      ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
      iii) The rxType value is set equal to invalid.
      iv) The procedure rxProcessFrame() is terminated.
   3) If the extracted TLV type value matches the Port ID TLV type, and the port ID TLV information string length is within the range $2 \leq n \leq 256$, the port ID value is extracted and appended to the chassis ID value to complete construction of the MSAP identifier.
c) The third TLV is extracted:

1) If the extracted TLV type value does not match the Time To Live, Extension Request, or Extension TLV type, the LLDPDU is invalid:
   i) The LLDPDU is discarded.
   ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
   iii) The rxType value is set equal to invalid.
   iv) The procedure rxProcessFrame() is terminated.
2) If the extracted TLV type value matches the Time To Live or Extension TLV type, and the adminStatus variable is enableTxOnly, the LLDPDU is discarded because the receive state machine is only enabled to receive Extension Request LLDPDUs.
   i) The LLDPDU is discarded.
   ii) The rxType value is set equal to invalid.
   iii) The procedure rxProcessFrame() is terminated.
3) If the extracted TLV type value matches the Time To Live TLV type, and the Time To Live TLV information string length is less than 2:
   i) The LLDPDU is discarded.
   ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
   iii) The rxType value is set equal to invalid.
   iv) The procedure rxProcessFrame() is terminated.
4) If the extracted TLV type value matches the Time To Live TLV type, and the Time To Live TLV information string length is greater than or equal to 2:
   i) The first two octets of the TLV information string is extracted and rxTTL is set to this value.
   ii) If rxTTL equals zero, the rxType value is set equal to shutdown and the procedure rxProcessFrame() is terminated.
   iii) If rxTTL is non-zero, the rxType value is set to normal.
5) If the extracted TLV type value matches the Extension Request TLV type, XLLDP is supported, and the received MSAP identifier and Scope MAC Address match the MSAP and Scope MAC Address of this LLDP agent:
   i) The rxType value is set equal to xreq.
   ii) The procedure rxProcessFrame() is terminated.
6) If the extracted TLV type value matches the Extension Request TLV type, and either XLLDP is not supported or the received MSAP identifier and Scope MAC Address combination does not match the MSAP and Scope MAC Address combination of this LLDP agent:
   i) The LLDPDU is discarded.
   ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
   iii) The rxType value is set equal to invalid.
   iv) The procedure rxProcessFrame() is terminated.
7) If the extracted TLV type value matches the Extension TLV type, XLLDP is supported, and an extended receive state machine exists for the neighbor identified by the received MSAP and Scope MAC Address:
   i) The rxType value is set equal to xpdu.
8) If the extracted TLV type value matches the Extension TLV type, and either XLLDP is not supported or an extended receive state machine does not exist for the neighbor identified by the received MSAP and Scope MAC Address:

i) The LLDPDU is discarded.

ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.

iii) The rxType value is set equal to invalid.

iv) The procedure rxProcessFrame() is terminated.

Each of the remaining TLV information elements are decoded in succession as required by their particular TLV format definitions:

d) If the TLV type value equals 0, the TLV is the End Of LLDPDU TLV, and the procedure rxProcessFrame() is terminated.

e) If the TLV_type_value is less than 127 and is recognized by the LLDP implementation, it is validated according to the general rules for all TLVs defined in 9.2.8.7.2 as well as any specific rules defined for the particular TLVs defined in 8.5.

f) If the TLV_type_value is less than 127 and is not recognized by the LLDP implementation, it may be a basic TLV from a later LLDP version. The statsTLVsUnrecognizedTotal counter is incremented, and the TLV is assumed to be validated.

g) If the implementation supports XLLDP and the TLV type value matches the Manifest TLV type:

1) Calculate the additional required space in the LLDP remote system information repository to store new or updated neighbor information:

i) If the MSAP identifier in the current LLDPDU does not match an MSAP identifier in the LLDP remote systems information repository, the total information repository size is extracted from the Manifest TLV and used as the additional required space.

ii) If the MSAP identifier in the current LLDPDU matches an MSAP identifier in the LLDP remote system, the additional required space is the difference between the total information repository entry size as extracted from the Manifest TLV and the current size of the data associated with the MSAP identifier in the LLDP remote system information repository.

2) If there is insufficient space in the LLDP remote systems information repository to store the neighbor information (based on the additional required space calculated in 1 above), and the policy is to discard the current LLDPDU rather than delete information for other MSAPs or if there would be insufficient space after deleting all possible information for other MSAPs from the remote system information repository then:

i) The flag variable tooManyNeighbors is set to TRUE.

ii) If the rxTTL value in the LLDPDU is greater than the value of the tooManyNeighborsTimer, the tooManyNeighborsTimer is set to the rxTTL value.

iii) The statsFrameDiscardedTotal counter is incremented.

iv) The rxType value is changed to invalid.

3) If there is sufficient space in the LLDP remote systems information repository or the policy allows deleting information from other MSAPs rather than discarding the current LLDPDU and sufficient space can be created (base on the space calculated in 1 above), then:

i) The rxType value is changed to manifest.

ii) If an extended receive state machine does not already exist for the MSAP identifier in the current LLDPDU, an extended receive state machine for that MSAP identifier is created and its createExtRx variable is set to TRUE.

h) If the TLV type value is 127, the TLV is an Organizationally Specific TLV:

1) If the TLV's organizationally unique identifier and organizationally defined subtype are recognized, the TLV is validated according to the general rules for all TLVs defined in 9.2.8.7.2 as well as the general rules for Organizationally Specific TLVs defined in 9.2.8.7.3 plus any specific rules defined for the particular TLV.

2) If the TLV's organizationally unique identifier and/or organizationally defined subtype are not recognized, the statsTLVsUnrecognizedTotal counter is incremented, and the TLV is assumed to be validated.

i) If the end of the LLDPDU has been reached the procedure rxProcessFrame**()** is terminated.

9.2.8.7.2 General validation rules for all TLVs

The value in the TLV information string length field is the value used to validate the TLV and to indicate the location of the next TLV in the LLDPDU.

All TLVs are subject to the general validation rules listed in this subclause as well as any specific usage rules defined for the particular TLV (for example, see systems capabilities TLV usage rules in 8.5.8.3):

a) If the LLDPDU contains more than one Chassis ID TLV, Port ID TLV, or Time To Live TLV:
   1) The LLDPDU is discarded.
   2) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
   3) The rxType value is set equal to invalid.
   4) The procedure rxProcessFrame() is terminated.

b) If the implementation supports XLLDP and the LLDPDU contains more than one Manifest TLV, Extension Request TLV, or Extension Identifier TLV:
   1) The LLDPDU is discarded.
   2) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
   3) The rxType value is set equal to invalid.
   4) The procedure rxProcessFrame() is terminated.

c) If the LLDPDU contains more than one out of the 3 TLV types Time to Live TLV, Extension Request TLV and Extension TLV:
   1) The LLDPDU is discarded.
   2) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
   3) The rxType value is set equal to invalid.
   4) The procedure rxProcessFrame() is terminated.

d) If the LLDPDU contains both a Manifest TLV and an Extension TLV:
   1) The LLDPDU is discarded.
   2) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
   3) The rxType value is set equal to invalid.
   4) The procedure rxProcessFrame() is terminated.

e) If the TLV information string length value is not greater than or equal to the sum of the lengths of all fields contained in the TLV information string:
   1) The LLDPDU is discarded.
   2) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
   3) The rxType value is set equal to invalid.
   4) The procedure rxProcessFrame() is terminated.

f) If any TLV contains an error condition specified for that particular TLV:
   1) The TLV is discarded.
   2) The statsTLVsDiscardedTotal counter is incremented and if the statsFramesinErrors has not been previously updated for this LLDPDU then the statsFramesInErrorsTotal counter is incremented.
   3) The location of the next TLV is based on the TLV information string length value of the current TLV.

g) With the exception of the TTL TLV for which specific rules apply (see 9.2.8.7.1), if the length of any field of a TLV is outside the length range specified for that particular TLV (for example, the

TLV information string length is greater than the maximum permitted length for the TLV information string as specified for that TLV):

1) The TLV is discarded.
2) The statsTLVsDiscardedTotal counter is incremented and if the statsFramesinErrors has not been previously updated for this LLDPDU then the statsFramesInErrorsTotal counter is incremented.
3) The location of the next TLV is based on the TLV information string length value of the current TLV.

h) If any TLV extends past the physical end of the frame:

1) The TLV is discarded.
2) The statsTLVsDiscardedTotal counter is incremented and if the statsFramesinErrors has not been previously updated for this LLDPDU then the statsFramesInErrorsTotal counter is incremented.

i) Any information following an End Of LLDPDU TLV is ignored.

NOTE—Usage rules for individual TLVs allow some TLVs to appear more than once in an LLDPDU. Duplicate TLVs result in any one of the values being placed in the information repository, can cause the discard stats to increment, and can cause the change marker for the information repository entry to change if any of the TLV copies change the value even if the value finally recorded is unchanged. The only thing guaranteed is that the information repository value is set to one (unspecified) of the TLV values, and if that value is different to what was previously in the information repository then the change marker is set.

9.2.8.7.3 General validation rules for all Organizationally Specific TLVs

If an Organizationally Specific TLV is not recognized by the receiving LLDP agent, the content of the TLV can be ignored but the TLV is stored in the LLDP remote systems information repository for possible retrieval by network management using the basic Organizationally Specific TLV format (see 8.7) and the StatsTLVsUnrecognizedTotal counter is incremented. If more than one unrecognized Organizationally Specific TLV is received with the same OUI and organizationally defined subtype, but with identifiable differences in the organizationally defined information strings, all copies are assigned a temporary identification index and stored.

9.2.8.8 somethingChangedLocal()

This procedure is called to indicate that the status/value of one or more of the selected objects in the LLDP local system information repository has changed, and that there is therefore a need to transmit new information. It is used to notify the managers of the PTOPO and other optional MIBs or YANG modules that something has changed in the LLDP local systems information repository.

This procedure also sets the value of the variable localChange (see 9.2.5.2) to signal the need for the LLDP agent(s) to transmit the new information, and copies the value of the variable txFastInit (see 9.2.5.20) to the variable txFast (see 9.2.5.19) to ensure that the new values will be seen by the neighbors on the port.

It is assumed that the system provides the ability for any processes that need to receive such indications to register with this procedure so that appropriate signals can be received by those processes when the procedure is called.

9.2.8.9 somethingChangedRemote()

This procedure is called after all the information associated with the particular MSAP identifier has been updated in the LLDP remote systems information repository. The procedure call serves as an indication to the managers of the PTOPO and other optional MIBs or YANG modules that one or more of the following conditions is true:

a) That the status/value of one or more objects in the LLDP remote systems information repository associated with the particular MSAP identifier has changed.
b) That an incoming LLDPDU contained an MSAP identifier requiring creation of a new information repository structure in the LLDP remote systems information repository to receive the information in that LLDPDU.
c) That information in the LLDP remote systems information repository associated with the MSAP identifier has been deleted.

It is assumed that the system provides the ability for any processes that need to receive such indications to register with this procedure so that appropriate signals can be received by those processes when the procedure is called.

9.2.8.10 txAddCredit()

This procedure increments the value of the txCredit variable (9.2.5.17) by one if the current value of the txCredit variable is less than the value of txCreditMax (9.2.5.18).

9.2.8.11 txFrame()

The txFrame() procedure makes use of the services of LLC to transmit the LLDPDU constructed by either the rpConstrInfoLLDPDU(), rpConstrShutdownLLDPDU(), constructXreqPDU(), or rxProcessXreq() procedures (9.2.8.2, 9.2.8.3, 9.2.8.16, 9.2.8.18). For Normal LLDPDUs, the destination MAC address used is the MAC address associated with the instance of the LLDP agent (see 7.1.1 and 9.2.5.3). For Extension Request and Extension LLDPDUs, the destination MAC address used is the Return MAC Address in the received Manifest TLV or Extension Request TLV respectively (see 8.6.1.2 and 8.6.2.2). The source MAC address used is the individual MAC address of the transmitting port. The EtherType used is the LLDP EtherType identified in Table 7-3.

If the frame conveys a Manifest LLDPDU, the sentManifest variable is set to TRUE; otherwise the sentManifest variable is set to FALSE.

After the frame has been transmitted, the statsFramesOutTotal counter (9.2.7.5) is incremented.

9.2.8.12 txInitializeLLDP()

The txInitializeLLDP() procedure initializes the LLDP transmit module. It performs the following tasks:

a) If applicable, either the non-volatile configuration for the LLDP local system information repository are retrieved or the appropriate default values are assigned to all LLDP configuration variables.
b) The internal (implementation specific) data structures are initialized with appropriate local physical topology information.
c) The setManifest variable is set to FALSE.
d) System default values are set for the following timing parameters:
    1) reinitDelay (9.2.5.9)
    2) msgTxHold (9.2.5.5)
    3) msgTxInterval (9.2.5.6)

4) msgFastTx (9.2.5.4)

NOTE—It is recommended to introduce a degree of stagger into starting the LLDP local agent initialization in multi-port implementations so that the timing of LLDP frame transmission cycles can be distributed among system ports, and distributed among supported MAC addresses on each port. The method of accomplishing the stagger is an implementation issue and is beyond the scope of this standard.

9.2.8.13 rxProcessManifest()

The rxProcessManifest() procedure performs the following tasks:

a) Form the MSAP identifier from the chassis ID and Port ID of the received LLDPDU.
b) All TLVs received in the Manifest LLDPDU and passed from the receive state machine are retained in temporary storage associated with the MSAP identifier calculated in step a), replacing any TLVs retained from a previous Manifest LLDPDU.
c) If there is no current entry in the LLDP remote systems information repository for the MSAP identifier calculated in step a), or if the current entry does not contain a Manifest TLV, all XPDU Descriptors in the new Manifest TLV are assigned a status of “update”.
d) If there is a current manifest in the LLDP remote systems information repository for the MSAP identifier calculated in step a), each XPDU Descriptor in the new manifest is compared to the XPDU Descriptor in the current manifest. Any XPDU Descriptors in the new manifest that match an XPDU Descriptor in the current manifest are assigned a status of “current”. Any XPDU Descriptors in the new manifest that do not have a matching XPDU Descriptor in the current manifest are assigned a status of “update”.
e) If there are any XPDUs in temporary storage associated with this MSAP identifier that match an XPDU Descriptor with a status of “update” in the new manifest, those XPDUs are retained and the status of the corresponding XPDU Descriptors is changed to “new”. Any XPDUs in temporary storage associated with the MSAP identifier calculated in step a) that do not match an XPDU descriptor received in the Manifest message with a status of update are discarded.

NOTE—Usage rules for the Manifest TLV specify that each XPDU Descriptor has a unique XPDU Number. There is no requirement for the LLDP agent receiving a Manifest TLV to check for duplicate XPDU Numbers, and the response of the receiving LLDP agent to duplicated XPDU Numbers is unspecified and implementation dependent.

9.2.8.14 rxProcessXPDU()

The rxProcessXPDU() procedure performs the following tasks:

a) The statsFramesInTotal counter for the port is incremented.
b) The 32-bit XPDU Check value of the XPDU is calculated. If the XPDU Number, Sequence, and Check do not match an XPDU Descriptor in the new manifest, the statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented and the procedure rxProcessXPDU() is terminated.
c) If the XPDU Number, Sequence, and Check match an XPDU Descriptor in the new manifest, the status of that XPDU Descriptor is changed to “new”.
d) Validates the TLVs contained in the LLDPDU as defined in 9.2.8.7.
e) Retains all validated TLVs in temporary local storage associated with this XPDU Descriptor.
f) If there are any XPDU Descriptors in the new manifest with a status of “requested” or “retried”, the variable xreqComplete is set to FALSE. Otherwise the variable xreqComplete is set to TRUE.

9.2.8.15 checkManifest()

The checkManifest() procedure reviews the status of all XPDU Descriptors in the new manifest. If the status of one or more XPDU Descriptors is “update”, “requested”, or “retried”, the manifestComplete variable is set to FALSE. Otherwise the manifestComplete variable is set to TRUE and rxTimer is set to rxTTL.

9.2.8.16 constructXreqPDU()

The constructXreqPDU() procedure performs the following tasks:

a) If there are any XPDU Descriptors with a status of “requested” in the new manifest, these XPDU Descriptors are selected to be included in the Extension Request TLV, the status of these XPDU Descriptors is changed to “retried”, and rxTimer is set to rxTTL.
b) If there are no XPDU Descriptors with a status of “requested” or “retried” in the new manifest, one or more XPDU Descriptors with a status of “update” are selected to be included in the request. The status of these XPDU Descriptors is changed to “requested”, and rxTimer is set to the value of rxTTL divided by 32 and rounded up to the nearest integer.
c) If any XPDUs have been selected in steps a) or b) then construct an Extension Request LLDPDU (8.2) including an Extension Request TLV, structured according to the specification in 8.6.2, containing the selected XPDU Descriptors..

The LLDP agent generating the Extension Request LLDPDU can pace the rate at which Extension LLDPDUs arrive by adjusting the number of XPDU Descriptors in a single Extension Request TLV and the timing of the Extension Request LLDPDU transmission. This pacing is implementation dependent and beyond the scope of this standard. An LLDP agent only transmits an Extension Request LLDPDU when it is prepared (e.g., has sufficient buffer resources) to receive all requested Extension LLDPDUs.

9.2.8.17 checkRxTimer()

The checkRxTimer() procedure reviews the status of all XPDU Descriptors in the new manifest. If the value of rxTimer is zero and there are any XPDU Descriptors with a status of “requested”, then this is a timeout for an Extension Request LLDPDU and the rxTTLExpired variable is set to FALSE. If the value of rxTimer is zero and there are no XPDU Descriptors with a status “requested”, then this is a timeout for the TLV collection process and rxTTLExpired is set to TRUE.

9.2.8.18 rxProcessXREQ()

If the MSAP identifier and Scope MAC Address in the received Extension Request LLDPDU match this LLDP agent, then for each XPDU Descriptor in the Extension Request TLV that matches one of the XPDU Descriptors in the most recent Manifest TLV created by the LLDP manager and transmitted in a Normal LLDPDU, an Extension LLDPDU (8.2) is constructed, including an Extension Identifier TLV structured according to the specification in 8.6.3, and the other TLVs selected by the LLDP manager, and transmits the frame using the txFrame() procedure.

NOTE—When the Extension Request TLV includes more than one XPDU Descriptor, the rate at which Extension LLDPDUs are transmitted is at the discretion of the transmitting agent and is implementation dependent.

9.2.9 Transmit state machine

The transmit state machine for an individual port and destination MAC address (see 7.1) shall implement the function specified by the state diagram contained in Figure 9-1 and the attendant definitions contained in 9.2.3 through 9.2.8.

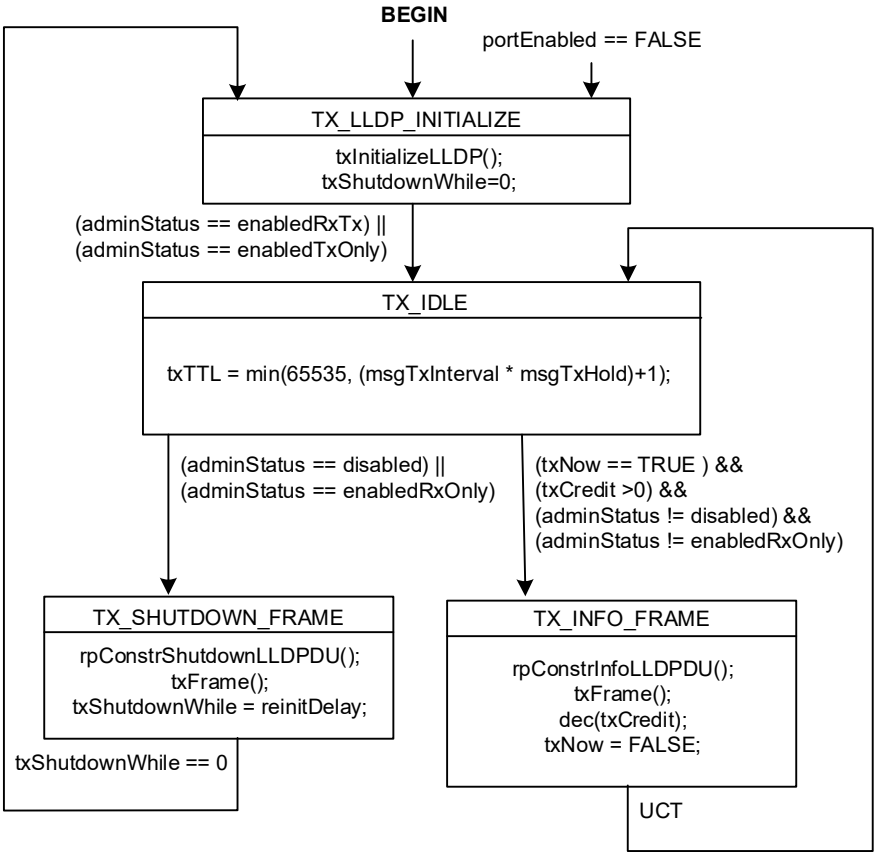


Figure 9-1—Transmit state machine

9.2.10 Receive state machine

The receive state machine for an individual port and destination MAC address (see 7.1) shall implement the function specified by the state diagram contained in Figure 9-2 and the attendant definitions contained in 9.2.2 through 9.2.8. Additionally, when an implementation supports Extended LLDP and a Manifest TLV is received in an LLDPDU, an extended receive state machine for the received chassis ID and port ID shall implement the function specified by the state diagram contained in Figure 9-3 and the attendant definitions contained in 9.2.2 through 9.2.8.

Because there can be multiple extended receive state machines, the receive state machine uses a notational convention to reference per-neighbor variables (9.2.6) associated with an extended receive state machine. The variable is prefaced by “MSAP.” to indicate a variable associated with the specific MSAP Identifier contained in the frame currently being processed by the receive state machine. The variable is prefaced with “any.” to indicate the logical OR of the variable from all extended state machines.

An extended receive state machine associated with a neighbor is dynamically instantiated, if it does not already exist, when a Normal LLDPDU is received that contains a Manifest TLV. It is destroyed, and any

TLVs held in temporary storage associated with this state machine are discarded, when a Normal LLDPDU is received that does not contain a Manifest TLV, or by the assertion of BEGIN.

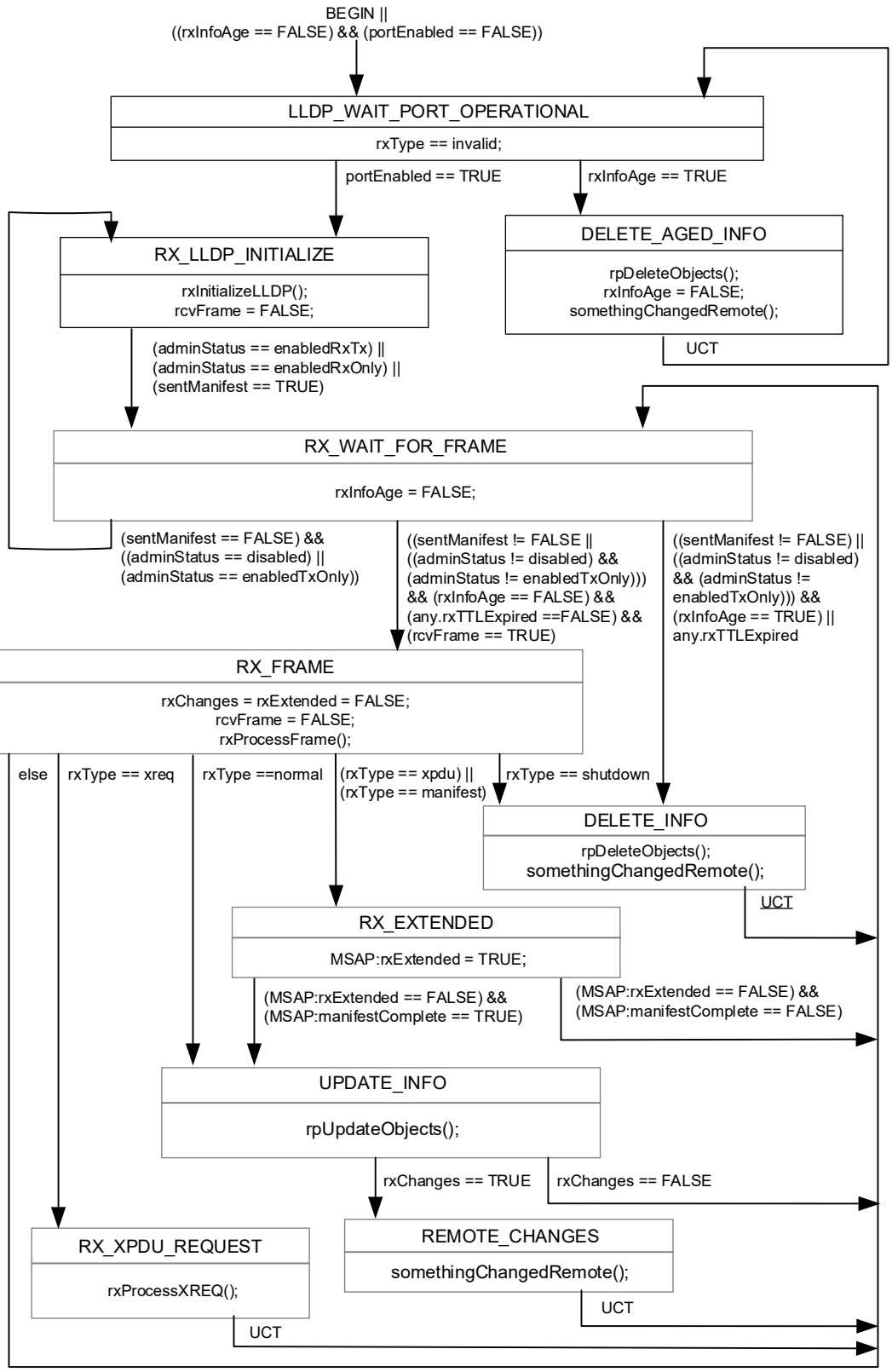


Figure 9-2— Receive state machine

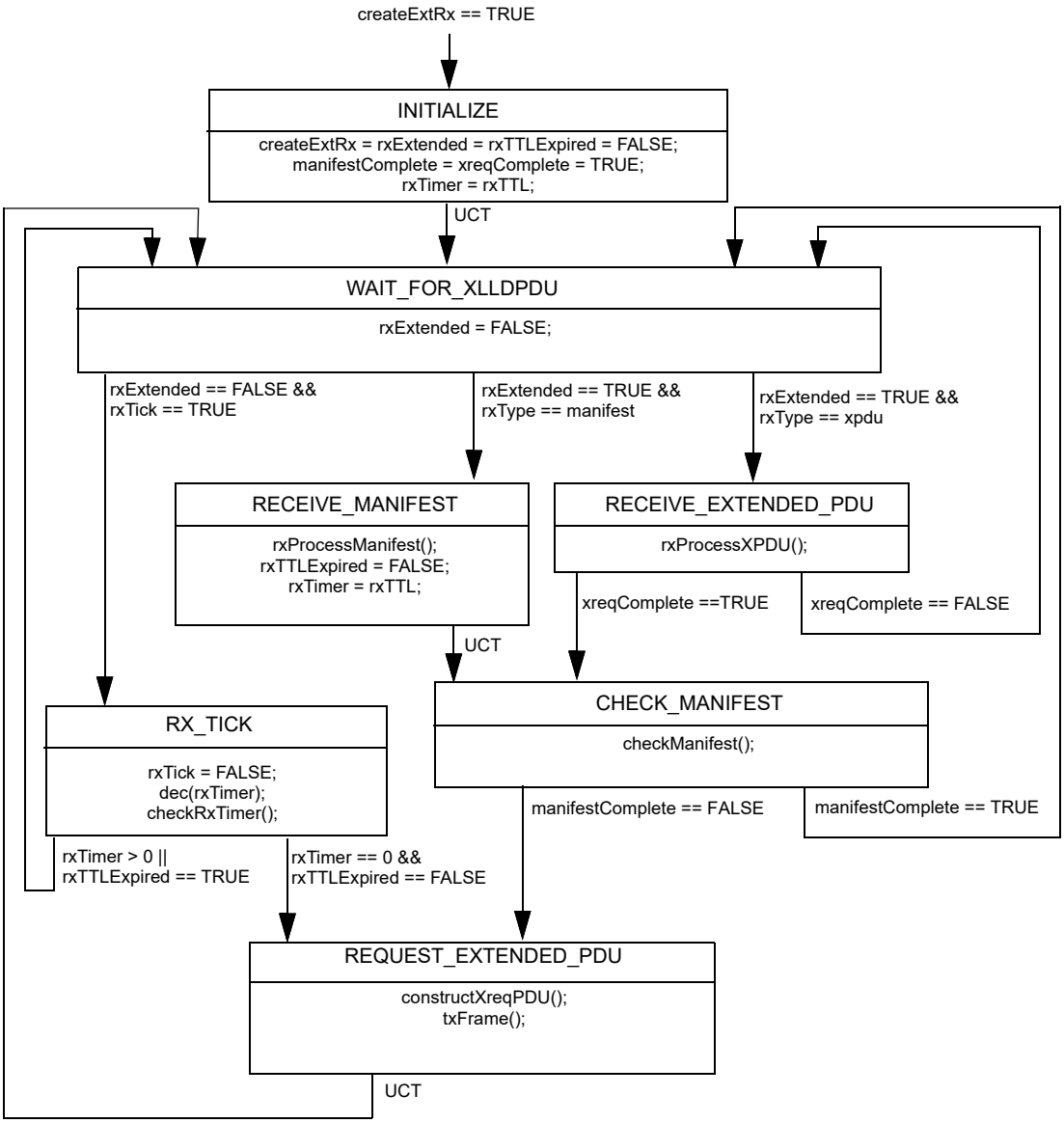


Figure 9-3— Extended receive state machine

9.2.11 Transmit timer state machine

The transmit timer state machine for an individual port and destination MAC address (see 7.1) shall implement the function specified by the state diagram contained in Figure 9-4 and the attendant definitions contained in 9.2.3 through 9.2.8.

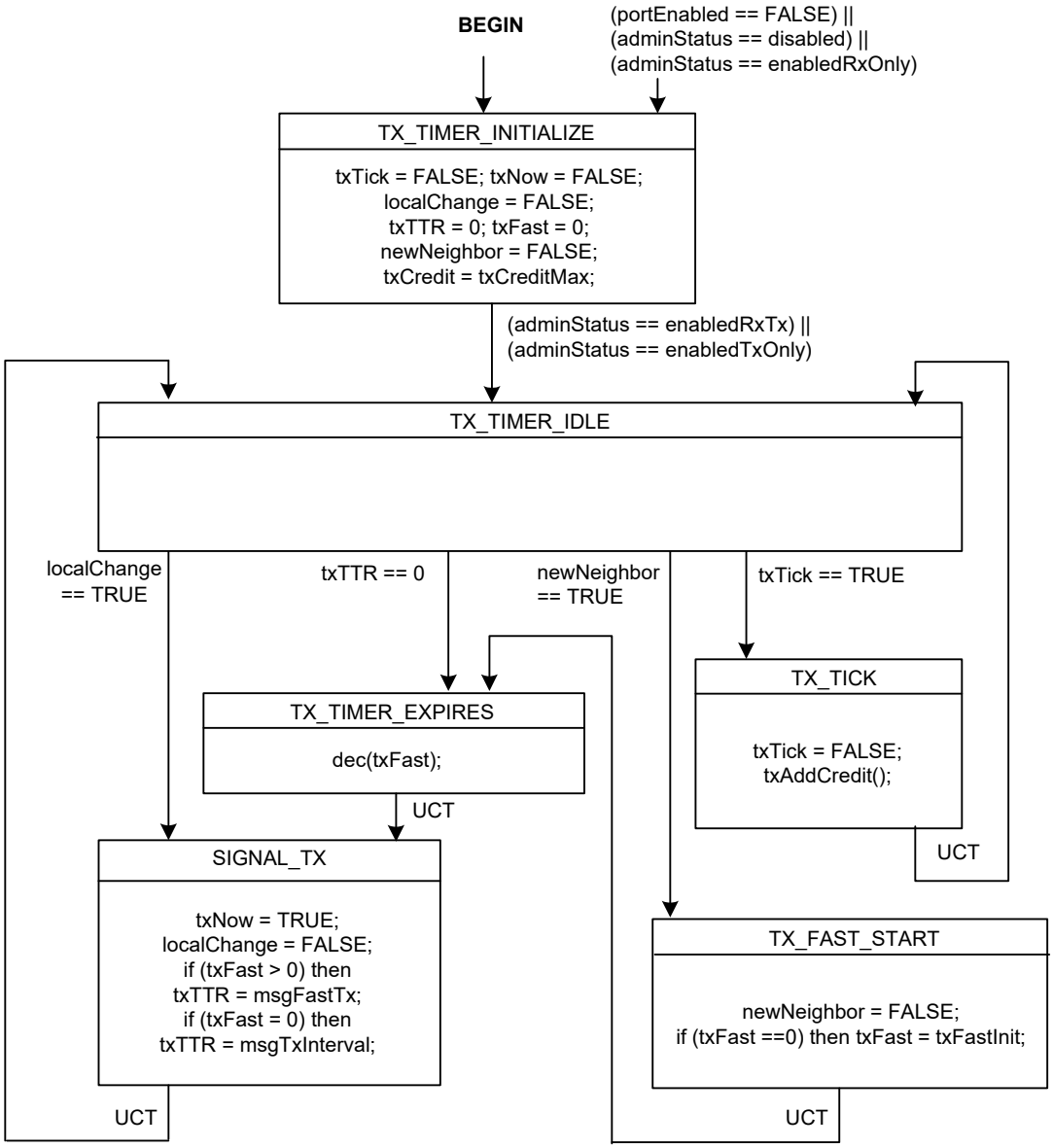


Figure 9-4—Transmit timer state machine

10. LLDP management

This clause defines the set of managed objects and their functionality that allow administrative configuration and monitoring of LLDP operation.

10.1 Data storage and retrieval

LLDP agents need to have a place to store both information about the local system and information they have received about remote systems. Data storage and retrieval capability to support the functionality defined in 7, 8, 9, and 10 shall be provided. No particular implementation is implied.

10.2 The LLDP management entity's responsibilities

The LLDP management entity has the following responsibilities:

a) Starting the transmit and receive state machines.
b) Providing a means for the network manager to select TLVs to be included in outgoing LLDPDUs.
c) Extracting the necessary information from the LLDP local system information repository and constructing the individual TLVs selected for insertion into an outgoing LLDPDU.
d) Prepending appropriate addressing and LLDP EtherType to the LLDPDU before submitting the MA_UNITDATA.request for frame transmission.
e) Updating the LLDP remote systems information repository and monitoring for information repository object adds, deletions and value changes.
f) Maintaining operational statistics.

10.2.1 Protocol initialization management

Protocol initialization consists of the following:

a) Retrieving non-volatile configuration values for the LLDP local system information repository.
b) Assigning default or management assigned values to LLDP configuration variables.
c) Loading physical topology information into their associated LLDP local system information repository objects.

10.2.2 TLV selection management

TLV selection management consists of providing the network manager with the means to select which specific TLVs are enabled for inclusion in an LLDPDU. The following LLDP variables cross reference to LLDP local systems configuration information repository tables indicating which specific TLVs are enabled for the particular port(s) on the system. The specific port(s) through which each TLV is enabled for transmission may be set (or reset) by the network manager.

a) **mibBasicTLVsTxEnable:** This variable lists the single-instance-use basic management TLVs, each with a bit map indicating the system ports through which the referenced TLV is enabled for transmission.
b) **mibMgmtAddrInstanceTxEnable:** This variable lists the different management addresses (by subtype) that are defined for the system, each with a bit map indicating the system ports through which the particular management address/subtype TLV is enabled for transmission.

NOTE—Implementers of new TLVs should be aware that provision needs to be made to allow similar network management selection of new TLVs and that doing so requires linkage to the LLDP information repository, regardless of whether or not the TLV is part of the basic management set or part of an Organizationally Specific TLV set.

When XLLDP is not supported, the LLDP agent can only transmit the TLVs that fit within a single Normal LLDPDU. If more TLVs are selected for inclusion in the LLDPDU than fit, some TLVs are not transmitted. Which TLVs are not transmitted is implementation dependent and not determined by this standard.

When XLLDP is supported, each TLV selected for transmission is mapped to the Normal LLDPDU or to an Extension LLDPDU. Each Extension LLDPDU is identified by an XPDU Number. The mapping of TLVs to XPDUs is implementation dependent. The LLDP management entity maintains a list of which TLVs are mapped to which XPDU Number, and uses this list to create a Manifest TLV for inclusion in a Manifest LLDPDU at the start of a transmission cycle. The addition, deletion, or modification of any TLV mapped to Extension LLDPDU since the last Manifest LLDPDU was transmitted cause the XPDU Revision for that Extension LLDPDU to be incremented prior to creating a new Manifest TLV. Maintaining a consistent mapping of TLVs to Extension LLDPDUs whenever possible reduces the number of Extension LLDPDUs that need to be requested and transmitted during a transmission cycle.

10.2.3 Transmission management

Transmission management consists of the following:

a) Determining when a new transmission cycle is required, as signaled by the somethingChangedLocal() procedure (see 9.2.8.8), or when an Extension LLDPDU has been requested by the rxProcessXREQ() procedure (see 9.2.8.18).

b) Extracting the appropriate LLDP local system information repository information for the three mandatory TLVs and inserting them at the beginning of the LLDPDU in proper format order.

c) Creating a Manifest TLV for inclusion in the Normal LLDPDU that starts a transmission cycle if XLLDP is supported and there are TLVs that are mapped to one or more Extension LLDPDUs.

d) Extracting the appropriate LLDP local system information repository information for the selected optional TLVs and inserting them into the LLDPDU.

e) Optionally, appending an End Of LLDPDU TLV after the last optional TLV in the LLDPDU.

f) Maintaining length control during LLDPDU construction so that the TLVs do not exceed the maximum length allowed for the LLDPDU.

g) Submitting the frame for transmission.

10.2.4 Reception management

Reception management consists of the following:

a) Receiving and parsing the incoming LLDPDUs.

b) Validating, and checking the types of the first three TLVs to ensure that they are the required type and in LLDPDU format order.

c) Validating optional TLVs and extracting information values.

d) Checking whether or not the current LLDPDU represents a new MSAP identifier.

e) Monitoring whether or not there is sufficient space available in the LLDP remote systems information repository for all active neighbors.

f) Arbitrating which information is to be discarded in cases where insufficient space is available in the LLDP remote systems information repository for all active neighbors.

g) Checking whether or not the received information represents a status or value change to the existing information repository object.

h) Updating remote systems information repository objects as necessary.

When XLLDP is supported, the LLDP management entity associates each TLV that is stored in the remote systems information repository with the XPDU Number of the Extension LLDPDU in which it was received. (For convenience an XPDU Number of zero can be used for TLVs received in a Normal LLDPDU,

since no valid Extension LLDPDU has an XPDU Number equal to zero.) This association is necessary so that the appropriate TLVs are updated or deleted when a new Extension LLDPDU with that XPDU Number is received.

10.2.5 Performance management

Performance management consists of the following:

a) Monitoring the LLDP local system information repository update activities for status or value changes to selected information repository objects and:
   1) Calling the somethingChangedLocal() procedure (see 9.2.8.8) whenever a status/value change occurs.
   2) Initiating a new transmit cycle if txCredit > 0.

b) Monitoring the txTTR timer for countdown expiration and initiating a new transmit cycle if txCredit > 0.

c) Monitoring the rxInfoTTL timer for countdown expiration and:
   1) Deleting the associated objects from the LLDP remote systems information repository whenever the timing counter expires.
   2) Performing the procedure somethingChangedRemote() associated with the MSAP identifier.

d) Monitoring rxTTL in the incoming LLDPDUs for shutdown indication and:
   1) Deleting the associated objects from the LLDP remote systems information repository whenever rxTTL = 0.
   2) Performing the procedure somethingChangedRemote() associated with the MSAP identifier.

e) Monitoring the “tooManyNeighbors” variable to determine whether an existing neighbor’s information needs to be deleted and:
   1) Deleting the objects associated with a selected MSAP identifier from the LLDP remote systems information repository whenever existing information is selected to be deleted.
   2) Performing the procedure somethingChangedRemote() associated with the MSAP identifier.

f) Monitoring the tooManyNeighborsTimer to determine when there is not sufficient space available to accommodate information from all active neighbors and setting the variable tooManyNeighbors associated with the port to FALSE when the tooManyNeighborsTimer expires.

g) Notifying the managers of the PTOPO MIB and other optional MIB and YANG modules (see Figure 11-1) whenever the procedures somethingChangedLocal() or somethingChangedRemote() are performed.

h) Maintaining operational statistics regarding the LLDP frames sent and received.

10.3 Managed objects

Managed objects model the semantics of management operations. Operations upon a managed object supply information concerning, or facilitate control over, the process or entity associated with that managed object.

Management of LLDP is described in terms of the managed resources that are associated with individual TLVs and that support frame transmission and reception.

10.4 Data types

This subclause specifies the semantics of operations independent of their encoding in management protocol. The data types of the parameters of operations are defined for that specification.

The following data types are used:

a) Boolean.
b) Enumerated, for a collection of named values.
c) Unsigned, for all parameters specified as “number of” some quantity.
d) MAC address.
e) Time interval, an Unsigned value representing a positive integral number of seconds for all time out parameters.
f) Counter, for all parameters specified as a “count” of some quantity (all counters increment and wrap with a modulus of 2 to the power 32).

10.5 LLDP variables

LLDP managed objects fall into the following categories:

a) Status/control variables necessary for operation of the protocol.
b) Variables that accumulate operational statistics.
c) Variables that are required by the particular TLVs.

10.5.1 LLDP operational status and control

a) Global parameters and variables:
    1) **adminStatus:** The authority that controls whether or not a local LLDP agent is enabled for transmit and receive, transmit only, or receive only; or is disabled (9.2.5.1).
b) Transmit state machine parameters and variables:
    1) **msgTxHold:** A multiplier on msgTxInterval used to compute the TTL value of txTTL (9.2.5.5, 9.2.5.23).
    2) **msgTxInterval:** The interval between successive transmit cycles in normal transmission mode (9.2.5.6).
    3) **reinitDelay:** The delay after adminStatus becomes ‘disabled’ before re-initialization is attempted (9.2.5.9).
    4) **txCreditMax:** The maximum value of transmission credit (9.2.5.18).
    5) **txFastInit:** The interval between successive LLDPDU transmissions in fast transmission mode (9.2.5.20).
c) Receive state machine parameters and variables:
    1) **remoteChanges:** An indication that the status/value of one or more selected objects in the LLDP remote systems information repository has changed or that all objects associated with a particular MSAP identifier have been deleted (9.2.5.10).
    2) **tooManyNeighbors:** An indication that there is insufficient space in the LLDP remote systems information repository to store information from all active neighbors (9.2.5.16).

10.5.2 LLDP operational statistics counters

The following counters provide operational statistics on a per port basis:

a) Transmission counters:
    1) **statsFramesOutTotal**: A count of all LLDP frames transmitted through the port (9.2.7.5).
    2) **statsLengthErrorsTotal:** A count of all LLDP length errors detected when constructing LLPDU frames for transmission through the port (9.2.8.2).
b) Reception counters:
    1) **statsAgeoutsTotal:** A count of the times that a neighbor’s information is deleted from the LLDP remote systems information repository because of rxInfoTTL timer expiration (9.2.7.1).

2) **statsFramesDiscardedTotal:** A count of all LLDPDUs received and then discarded (9.2.7.2).
3) **statsFramesInErrorsTotal:** A count of all LLDPDUs received at the port with one or more detectable errors 9.2.7.3).
4) **statsFramesInTotal:** A count of all LLDP frames received at the port (9.2.7.4).
5) **statsTlvsDiscardedTotal:** A count of all TLVs received at the port and discarded for any reason. (9.2.7.6)
6) **statsTLVsUnrecognizedTotal:** This counter provides a count of all TLVs not recognized by the receiving LLDP local agent (9.2.7.7).

10.5.3 TLV required variables

Variables in this category are defined by the requirements of the particular TLVs. They are maintained with reference to the local system in the LLDP local system information repository; and with reference to the remote system, in the LLDP remote systems information repository. TLV variables are outputs from the local LLDP agent and inputs to the remote LLDP agent.

Variables that pertain to basic set TLVs are listed in the following subclauses.

10.5.3.1 Chassis ID TLV objects

a) **chassis ID subtype:** The type of identifier used for the chassis (see 8.5.2.2).
b) **chassis ID:** The identification assigned to the chassis containing the port (see 8.5.2.3).

10.5.3.2 Port ID TLV objects

a) **port ID subtype:** The type of identifier used for the port (see 8.5.3.2).
b) **port ID:** The identification assigned to the port (see 8.5.3.3).

10.5.3.3 Port description TLV object

a) **port description:** The port’s description (see 8.5.5.2).

10.5.3.4 System name TLV object

a) **system name:** The system’s assigned name (see 8.5.6.2).

10.5.3.5 System description TLV object

a) **system description:** The system’s description (see 8.5.7.2).

10.5.3.6 System capabilities TLV objects

a) **system capabilities:** The primary capabilities of the system (see 8.5.8.1).
b) **enabled capabilities:** The system’s enabled capabilities (see 8.5.8.2).

10.5.3.7 Management address TLV objects

a) **management address length:** The length of the management address (see 8.5.9.2).
b) **management address subtype:** The management address type (see 8.5.9.3).
c) **management address:** The management address (see 8.5.9.4).
d) **interface numbering subtype:** The interface type (see 8.5.9.5).
e) **interface number:** The interface number (see 8.5.9.6).

f) **OID length:** The object identifier length (see 8.5.9.7).

g) **OID:** The object identifier (see 8.5.9.8).

11. LLDP MIB definitions

This clause defines the LLDP basic MIB for use with SNMP in TCP/IP based internets. In particular, it defines objects for managing the operation of LLDP based on the specifications of 7, 8, 9, and 10.

11.1 Internet Standard Management Framework

LLDP MIBs are designed to operate in a manner consistent with the principles of the Internet Standard Management Framework, which describes the separation of a data modeling language (for example, SMIv2) from content-specific data models (for example, the LLDP remote systems MIB), and from messages and protocol operations used to manipulate the data (for example, SNMPv3).

For a detailed overview of the documents that describe the current Internet Standard Management Framework, please refer to section 7 of IETF RFC 3410.

Managed objects are accessed via a virtual information store, termed the Management Information Base or MIB. MIB objects are generally accessed through the Simple Network Management Protocol (SNMP). Objects in the MIB are defined using the mechanisms defined in the Structure of Management Information (SMI). This clause specifies a MIB module that is compliant to the SMIv2, which is described in IETF RFC 2578 (STD 58) [B2], IETF RFC 2579 (STD 58) [B3], and IETF RFC 2580 (STD 58) [B4].

11.2 Structure of the LLDP MIB

The LLDP MIB consists of two types of MIB modules, the mandatory basic MIB defined in this clause and from zero to *n* optional organizationally specific MIB extensions such as the IEEE 802.1 MIB in Annex D of IEEE Std 802.1Q-2022 and the IEEE 802.3 MIB in Clause 79 of IEEE Std 802.3-2022.

Each MIB module is divided into two major sections as shown in Figure 11-1 to allow selective MIB support for the particular operating mode (transmit only, receive only, or both transmit and receive) being implemented.

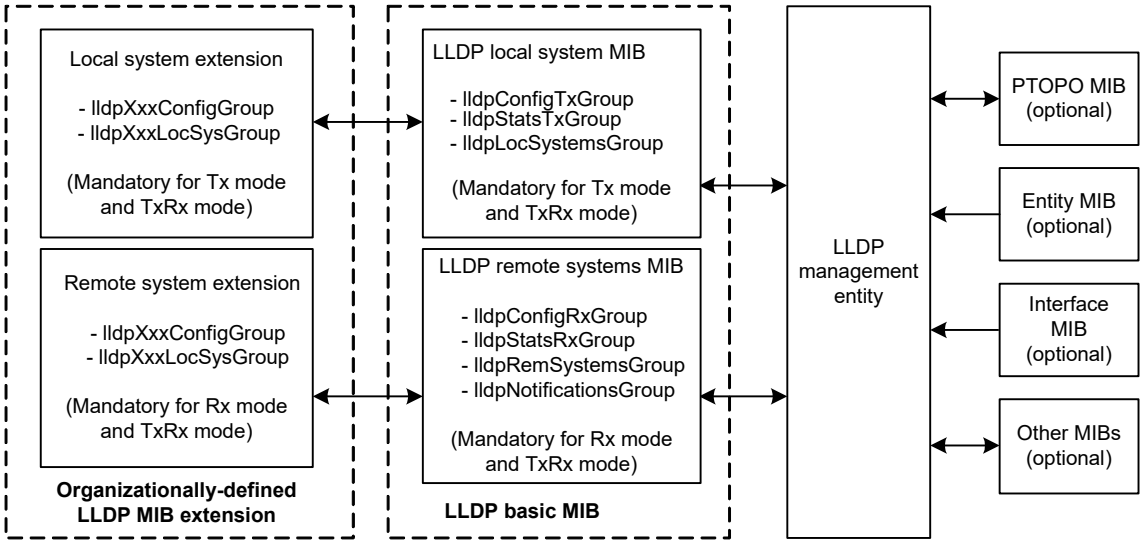


Figure 11-1—LLDP MIB block diagram

Table 11-1 summarizes the particular object groups that are required for each operating mode.The basic MIB shall comply with the MIB conformance section for the particular operating mode (Rx, Tx, or TxRx mode) being supported.

Table 11-1—MIB object groups and operating mode applicability

MIB group	Rx mode	Tx mode	TxRx mode
lldpV2ConfigGroup			
lldpV2ConfigRxGroup	M[a]	—	M
lldpV2ConfigTxGroup	—	M	M
lldpV2StatsRxGroup	M	—	M
lldpV2StatsTxGroup	—	M	M
lldpV2LocSysGroup	—	M	M
lldpV2RemSysGroup	M	—	M
lldpV2NotificationsGroup	M	—	M
ifGeneralInformationGroup	M	M	M

[a]M=Mandatory

Table 11-2 shows the structure of the MIB and the relationship of the MIB objects to the LLDP operational status/control variables, LLDP statistics variables, and TLV variables.

Table 11-2—LLDP MIB structure and object cross reference

MIB table	MIB object	LLDP reference
LLDP Configuration group		
	lldpV2MessageTxInterval	msgTxInterval, 9.2.5.6
	lldpV2MessageTxHoldMultiplier	msgTxHold, 9.2.5.5
	lldpV2ReinitDelay	reinitDelay, 9.2.5.9
	lldpV2NotificationInterval	msgTxInterval, 9.2.5.6
	lldpV2TxCreditMax	txCreditMax, 9.2.5.18
	lldpV2MessageFastTx	msgFastTx, 9.2.5.4
	lldpV2TxFastInit	txFastInit, 9.2.5.20

Table 11-2—LLDP MIB structure and object cross reference *(continued)*

MIB table	MIB object	LLDP reference
lldpV2PortConfigTableV2		
	lldpV2PortConfigIfIndexV2	(Table index)
	lldpV2PortConfigDestAddressIndexV2	(Table index)
	lldpV2PortConfigAdminStatusV2	adminStatus, 9.2.5.1
	lldpV2PortMessageTxInterval	msgTxInterval, 9.2.5.6
	lldpV2PortMessageTxHoldMultiplier	msgTxHold, 9.2.5.5
	lldpV2PortReinitDelay	reinitDelay, 9.2.5.9
	lldpV2PortNotificationInterval	msgTxInterval, 9.2.5.6
	lldpV2PortTxCreditMax	txCreditMax, 9.2.5.18
	lldpV2PortMessageFastTx	msgFastTx, 9.2.5.4
	lldpV2PortTxFastInit	txFastInit, 9.2.5.20
	lldpV2PortConfigNotificationEnableV2	—
	lldpV2PortConfigTLVsTxEnableV2	9.1.1.1
lldpV2DestAddressTable		
	lldpV2AddressTableIndex	(Table index)
	lldpV2DestMacAddress	(Table index)
lldpV2ManAddrConfigTxPortsTable		
	lldpV2ManAddrConfigIfIndex	(Table index)
	lldpV2ManAddrConfigDestAddressIndex	(Table index)
	lldpV2ManAddrConfigLocManAddrSubtype	8.5.9.3 (Table index)
	lldpV2ManAddrConfigLocManAddr	8.5.9.4 (Table index)
	lldpV2ManAddrConfigTxEnable	9.1.1.1
	lldpV2ManAddrConfigRowStatus	—
LLDP Statistics group		
	lldpV2StatsRemTablesLastChangeTime	—
	lldpV2StatsRemTablesInserts	—
	lldpV2StatsRemTablesDeletes	—
	lldpV2StatsRemTablesDrops	—
	lldpV2StatsRemTablesAgeouts	—

Table 11-2—LLDP MIB structure and object cross reference *(continued)*

MIB table	MIB object	LLDP reference
lldpV2StatsTxPortTable		
	lldpV2StatsTxIfIndex	(Table index)
	lldpV2StatsTxDestMACAddress	(Table index)
	lldpV2StatsTxPortFramesTotal	statsFramesOutTotal, 9.2.7.5
	lldpV2StatsTxLLDPDULengthErrors	lldpduLengthErrors, 9.2.7.8
lldpV2StatsRxPortTable		
	lldpV2StatsRxDestIfIndex	(Table index)
	lldpV2StatsRxDestMACAddress	(Table index)
	lldpV2StatsRxPortFramesDiscardedTotal	statsFramesDiscardedTotal, 9.2.7.2
	lldpV2StatsRxPortFramesErrors	statsFramesInErrorsTotal, 9.2.7.3
	lldpV2StatsRxPortFramesTotal	statsFramesInTotal, 9.2.7.4
	lldpV2StatsRxPortTLVsDiscardedTotal	statsTLVsDiscardedTotal, 9.2.7.6
	lldpV2StatsRxPortTLVsUnrecognizedTotal	statsTLVsUnrecognizedTotal, 9.2.7.7
	lldpV2StatsRxPortAgeoutsTotal	statsAgeoutsTotal, 9.2.7.1
Local System Data group		
	lldpV2LocChassisIdSubtype	chassis ID subtype, 8.5.2.2
	lldpV2LocChassisId	chassis ID, 8.5.2.3
	lldpV2LocSysName	system name, 8.5.6.2
	lldpV2LocSysDesc	system description, 8.5.7.2
	lldpV2LocSysCapSupported	system capabilities, 8.5.8.1
	lldpV2LocSysCapEnabled	enabled capabilities, 8.5.8.2
lldpV2LocPortTable		
	lldpV2LocPortIfIndex	(Table index)
	lldpV2LocPortIdSubtype	port ID subtype, 8.5.3.2
	lldpV2LocPortId	port ID, 8.5.3.3
	lldpV2LocPortDesc	port description, 8.5.5.2
lldpV2LocManAddrTable		
	lldpV2LocManAddrSubtype	management address subtype, 8.5.9.3 (Table index)
	lldpV2LocManAddr	management address, 8.5.9.4 (Table index)
	lldpV2LocManAddrLen	management address string length, 8.5.9.2
	lldpV2LocManAddrIfSubtype	interface numbering subtype, 8.5.9.5

Table 11-2—LLDP MIB structure and object cross reference *(continued)*

MIB table	MIB object	LLDP reference
	lldpV2LocManAddrIfId	interface number, 8.5.9.6
	lldpV2LocManAddrOID	object identifier, 8.5.9.8
Remote Systems Data group		
lldpV2RemTable		
	lldpV2RemTimeMark	(Table index)
	lldpV2RemLocalIfIndex	(Table index)
	lldpV2RemLocalDestMACAddress	(Table index)
	lldpV2RemIndex	(Table index)
	lldpV2RemChassisIdSubtype	chassis ID subtype, 8.5.2.2
	lldpV2RemChassisId	chassis ID, 8.5.2.3
	lldpV2RemPortIdSubtype	port ID sybtype, 8.5.3.2
	lldpV2RemPortId	port ID, 8.5.3.3
	lldpV2RemPortDesc	port description, 8.5.5.2
	lldpV2RemSysName	system name, 8.5.6.2
	lldpV2RemSysDesc	system description, 8.5.7.2
	lldpV2RemSysCapSupported	system capabilities, 8.5.8.1
	lldpV2RemSysCapEnabled	enabled capabilities, 8.5.8.2
	lldpV2RemRemoteChanges	remoteChanges, 9.2.5.10
	lldpV2RemTooManyNeighbors	tooManyNeighbors, 9.2.5.16
lldpV2RemManAddrTable		(Table index)
	lldpV2RemTimeMark	(Table index)
	lldpV2RemLocalIfIndex	(Table index)
	lldpV2RemLocalDestMACAddress	(Table index)
	lldpV2RemIndex	(Table index)
	lldpV2RemManAddrSubtype	management address subtype, 8.5.9.3 (Table index)
	lldpV2RemManAddr	management address, 8.5.9.4 (Table index)
	lldpV2RemManAddrIfSubtype	interface numbering subtype, 8.5.9.5
	lldpV2RemManAddrIfId	interface number, 8.5.9.6
	lldpV2RemManAddrOID	object identifier, 8.5.9.8

Table 11-2—LLDP MIB structure and object cross reference *(continued)*

MIB table	MIB object	LLDP reference
lldpV2RemUnknownTLVTable		
	lldpV2RemTimeMark	(Table index)
	lldpV2RemLocalIfIndex	(Table index)
	lldpV2RemLocalDestMACAddress	(Table index)
	lldpV2RemIndex	(Table index)
	lldpV2RemUnknownTLVType	LLDPDU validation, 9.2.8.7.1 (Table index)
	lldpV2RemUnknownTLVInfo	LLDPDU validation, 9.2.8.7.1
lldpV2RemOrgDefInfoTable		
	lldpV2RemTimeMark	(Table index)
	lldpV2RemLocalIfIndex	(Table index)
	lldpV2RemLocalDestMACAddress	(Table index)
	lldpV2RemIndex	(Table index)
	lldpV2RemOrgDefInfoOUI	organizationally unique identifier, 8.7.1.3 (Table index)
	lldpV2RemOrgDefInfoSubtype	organizationally defined subtype, 8.7.1.4 (Table index)
	lldpV2RemOrgDefInfoIndex	(Table index)
	lldpV2RemOrgDefInfo	organizationally defined information, 8.7.1.5
LLDP MIB Notifications		
	lldpV2RemTablesChange	

11.3 Relationship to other MIBs

This clause includes specifications for an LLDP Textual Conventions MIB module, an LLDP MIB module, and for IEEE 802.1 and IEEE 802.3 extension MIB modules that are compliant with the SMIv2 as defined in IETF RFC 2578 (STD 58) [B2], IETF RFC 2579 (STD 58) [B3], and IETF RFC 2580 (STD 58) [B4].

LLDP is designed to operate in conjunction with MIB modules defined by the IETF, IEEE 802, and others. The following subclauses discuss the relationship between LLDP and IETF MIB modules. LLDP agents automatically notify the managers of these MIB modules whenever there is a value or status change in an LLDP MIB object.

LLDP managed objects for the local system are stored in the LLDP local system MIB. Information received from a remote LLDP agent is stored in the local LLDP agent's LLDP remote system MIB.

NOTE—In this standard, managed objects for an LLDP MIB module are defined as they would be for an SNMP MIB. However, it is not required for LLDP implementations to support SNMP to store and retrieve system data. LLDP agents need to have a place to store both information about the local system and information they have received about remote systems. No particular implementation is implied.

11.3.1 Relationship to LLDP Version 1 MIB

The version 1 MIB module that was published in the initial 2005 publication of this standard has been superseded by the version 2 MIB module specified in 11.5.1, and support of the version 2 module is a requirement for conformance to the required or optional capabilities (Clause 5) in this revision of the standard. The version 2 MIB module reflects changes in indexation of the MIB objects that support the use of LLDP with multiple destination MAC addresses, as discussed in Clause 6.

In addition to the changes in indexation, the version 2 MIB module also supports changes to the management of the LLDP protocol that have resulted from changes to the LLDP protocol state machines. As these protocol changes affect only the timing and frequency of transmission of LLDPDUs, and not the content of the LLDPDUs, version 1 and version 2 implementations can successfully co-exist and interoperate.

From the perspective of a version 2 implementation, a version 1 neighbor behaves as a version 2 neighbor that is only using a single destination MAC address (the 01-80-C2-00-00-0E address specified for use in the IEEE 802.1AB-2005 version of the standard). From the perspective of a version 1 implementation, a version 2 neighbor that uses the 01-80-C2-00-00-0E destination MAC address behaves as a version 1 neighbor.

11.3.2 IETF Physical Topology MIB

The IETF Physical Topology (PTOPO) MIB (IETF RFC 2922 [B8]) allows a LLDP agent to expose learned physical topology information, using an IETF MIB. LLDP is intended to support the PTOPO MIB.

NOTE—The LLDP MIB module is a logical superset of the IETF PTOPO MIB; therefore, from a functional point of view, if the LLDP MIB module is implemented, it is likely not to be necessary to implement the PTOPO MIB defined in IETF RFC 2922 [B8]. However, for backward compatibility, this standard also supports the use of the PTOPO MIB.

11.3.3 IETF Entity MIB

The Entity MIB (IETF RFC 6933) allows the physical component inventory and hierarchy to be identified. Chassis IDs passed in the LLDPDU can identify entPhysicalTable entries. SNMP agents that implement the LLDP MIB should implement the entPhysicalAlias object from the Entity MIB version 2 or higher.

11.3.4 IETF Interfaces MIB

The Interfaces MIB (IETF RFC 2863) provides a standard mechanism for managing network ports. Port IDs passed in the LLDPDU can identify ifTable (or entPhysicalTable) entries. SNMP agents that implement the LLDP MIB shall also implement the ifTable and ifXTable for the ports that are represented in the Interfaces MIB.

11.4 Security considerations for LLDP base MIB module

There are a number of management objects defined in this MIB module with a MAX-ACCESS clause of read-write. [14] Such objects may be considered sensitive or vulnerable in some network environments. The support for SET operations in a non-secure environment without proper protection can have a negative effect on network operations.

a) Setting the following objects to incorrect values can result in an excessive number of LLDP packets being sent by the LLDP agent:
   1) lldpV2MessageTxInterval, lldpV2PortMessageTxInterval
   2) lldpV2TxCreditMax, lldpV2PortTxCreditMax

[14] In IETF MIB definitions, the MAX-ACCESS clause defines the type of access that is allowed for particular data elements in the MIB. An explanation of the MAX-ACCESS mappings is given in section 7.3 of IETF RFC 2578 [B2].

3) lldpV2MessageFastTx, lldpV2PortMessageFastTx
4) lldpV2TxFastInit, lldpV2PortTxFastInit

b) Setting the object, lldpV2MessageTxHoldMultiplier or lldpV2PortMessageTxHoldMultiplier, to incorrect values can cause the LLDP agent to transmit LLDPDUs with too-high TTL values, which affect the expiration time of objects grouped under lldpV2RemoteSystemsData identifier.

c) Setting the object, lldpV2ReinitDelay or lldpV2PortReinitDelay, to too low a value can cause the transmit state machine to attempt excessive re-initializations.

d) Setting incorrect bits in the object, lldpV2PortConfigTLVsTxEnableV2, can cause the LLDP agent to transmit LLDPDUs with an undesired optional TLV sequence.

e) Setting incorrect bits in the object, lldpV2ConfigManAddrPortsTxEnable, can cause the LLDP agent to advertise management addresses that were not meant to be disclosed and/or to omit addresses that were desired.

f) Setting the following objects to incorrect values can result in improper operation of the MIB notification process:

1) llldpV2NotificationInterval
2) llldpV2PortNotificationInterval
3) lldpV2PortConfigNotificationEnableV2

g) Setting the object, lldpV2PortConfigAdminStatusV2, to the incorrect value can result in enabling a non-desired operational mode.

The following readable objects in this MIB module may be considered to be sensitive or vulnerable in some network environments:

h) Objects that are associated with the transmit mode

1) lldpV2LocChassisIdSubtype
2) lldpV2LocChassisId
3) lldpV2LocPortIdSubtype
4) lldpV2LocPortId
5) lldpV2LocPortDesc
6) lldpV2LocSysName
7) lldpV2LocSysDesc
8) lldpV2LocSysCapSupported
9) lldpV2LocSysCapEnabled
10) lldpV2LocManAddrLen
11) lldpV2LocManAddrIfSubtype
12) lldpV2LocManAddrIfId
13) lldpV2LocManAddrOID

i) Objects that are associated with the receive mode

1) lldpV2NotificationInterval
2) lldpV2PortConfigNotificationEnable
3) lldpV2RemChassisIdSubtype
4) lldpV2RemChassisId
5) lldpV2RemPortIdSubtyp
6) lldpV2RemPortId
7) lldpV2RemPortDesc
8) lldpV2RemSysName
9) lldpV2RemSysDesc
10) lldpV2RemSysCapSupported
11) lldpV2RemSysCapEnabled
12) lldpV2RemManAddrIfSubtype
13) lldpV2RemManAddrIfId

14) lldpV2RemManAddrOID
15) lldpV2RemUnknownTLVInfo
16) lldpV2RemOrgDefInfo

This concern applies both to objects that describe the configuration of the local host, as well as for objects that describe information from the remote hosts, acquired via LLDP and displayed by the objects in this MIB module. It is thus important to control even GET and/or NOTIFY access to these objects and possibly to even encrypt the values of these objects when sending them over the network via SNMP.

SNMP versions prior to SNMPv3 did not include adequate security. Even if the network itself is secure (for example, by using IPSec), even then, there is no control as to who on the secure network is allowed to access and GET/SET (read/change/create/delete) the objects in this MIB module.

Implementers should consider the security features as provided by the SNMPv3 framework (see IETF RFC 3410, section 8), including full support for the SNMPv3 cryptographic mechanisms (for authentication and privacy).

Further, implementers should not deploy SNMP versions prior to SNMPv3. Instead**,** implementers should deploy SNMPv3 to enable cryptographic security. It is then a customer/operator responsibility to ensure that the SNMP entity giving access to an instance of this MIB module is properly configured to give access to the objects only to those principals (users) that have legitimate rights to indeed GET or SET (change/create/delete) them.

11.5 LLDP MIB modules [15, 16]

Two MIB modules are defined—a textual conventions MIB module (11.5.1) that contains the textual conventions used by the LLDP version 2 MIB module and by the version 2 extension MIB module in Annex D of IEEE Std 802.1Q-2022, and the LLDP version 2 MIB module itself (11.5.2). The textual conventions defined in the Textual-Convention MIB module are also available by extension MIB modules defined in other standards, such as the extension MIB modules defined in IEEE Std 802.1Q.

[15] *Copyright release for MIBs:* Users of this standard may freely reproduce the MIB contained in this subclause so that it can be used for its intended purpose.

[16] An ASCII version of this MIB module can be obtained by Web browser from the IEEE 802.1 Website at http://www.ieee802.org/1/pages/MIBS.html.

11.5.1 Definitions for the LLDP-V2-TC MIB module

In the following MIB module, should any discrepancy between the DESCRIPTION text and the corresponding definition in Clause 9 and Clause 10 occur, the definition in Clause 9 and Clause 10 shall take precedence.

```
LLDP-V2-TC-MIB DEFINITIONS ::= BEGIN
IMPORTS
    MODULE-IDENTITY,
    Unsigned32,
    org
        FROM SNMPv2-SMI
    TEXTUAL-CONVENTION
        FROM SNMPv2-TC;

lldpV2TcMIB MODULE-IDENTITY
    LAST-UPDATED "202508150000Z" -- August 15, 2025
ORGANIZATION "IEEE 802.1 Working Group"
    CONTACT-INFO
        "  WG-URL: http://www.ieee802.org/1/
         WG-EMail: stds-802-1-L@ieee.org

          Contact: IEEE 802.1 Working Group Chair
           Postal: C/O IEEE 802.1 Working Group
                   IEEE Standards Association
                   445 Hoes Lane
                   Piscataway
                   NJ 08854
                   USA
          E-mail: stds-802-1-L@ieee.org"
    DESCRIPTION
            "Textual conventions used throughout the IEEE Std 802.1AB
            version 2 and later MIB modules.

            Unless otherwise indicated, the references in this
            MIB module are to IEEE 802.1AB-2026.

            The TCs in this MIB are taken from the original LLDP-MIB,
            LLDP-EXT-DOT1-MIB, and LLDP-EXT-DOT3-MIB published in
            IEEE Std 802-1D-2004, with the addition of TCs to support
            the management address table. They have been made available
            as a separate TC MIB module to facilitate referencing from
            other MIB modules.

            Copyright (C) IEEE (2025). This version of this MIB module
            is published as 11.5.1 of IEEE Std 802.1AB-2026;
            see the standard itself for full legal notices."

       REVISION "202508150000Z" -- August 15, 2025

    DESCRIPTION
            "Published as part of IEEE Std 802.1AB-2026.
            This revision updated referencees."

    REVISION "201603110000Z" -- March 11, 2016

    DESCRIPTION
            "Published as part of IEEE Std 802.1AB-2016.
            This revision updated referencees."
```

```
    REVISION "200906080000Z" -- June 08, 2009

    DESCRIPTION
            "Published as part of IEEE Std 802.1AB-2009 revision."

   ::= { org ieee(111) standards-association-numbers-series-standards(2)
         lan-man-stds(802) ieee802dot1(1) 1 12 }

--
-- Definition of the root OID arc for IEEE 802.1 MIBs
--

ieee802dot1mibs OBJECT IDENTIFIER
    ::= { org ieee(111) standards-association-numbers-series-standards(2)
          lan-man-stds(802) ieee802dot1(1) 1 }

--
-- ************************************************************
--
-- Textual Conventions
--
-- ************************************************************

LldpV2ChassisIdSubtype ::= TEXTUAL-CONVENTION
    STATUS      current
    DESCRIPTION
            "This TC describes the source of a chassis identifier.

            The enumeration 'chassisComponent(1)' represents a chassis
            identifier based on the value of entPhysicalAlias object
            (defined in IETF RFC 6933) for a chassis component (i.e.,
            an entPhysicalClass value of 'chassis(3)').

            The enumeration 'interfaceAlias(2)' represents a chassis
            identifier based on the value of ifAlias object (defined in
            IETF RFC 2863) for an interface on the containing chassis.

            The enumeration 'portComponent(3)' represents a chassis
            identifier based on the value of entPhysicalAlias object
            (defined in IETF RFC 6933) for a port or backplane
            component (i.e., entPhysicalClass value of 'port(10)' or
            'backplane(4)'), within the containing chassis.

            The enumeration 'macAddress(4)' represents a chassis
            identifier based on the value of a unicast source address
            (encoded in network byte order and IEEE 802.3 canonical bit
            order), of a port on the containing chassis as defined in
            IEEE Std 802.

            The enumeration 'networkAddress(5)' represents a chassis
            identifier based on a network address, associated with
            a particular chassis. The encoded address is actually
            composed of two fields. The first field is a single octet,
            representing the IANA AddressFamilyNumbers value for the
            specific address type, and the second field is the network
            address value.
```

```
            The enumeration 'interfaceName(6)' represents a chassis
            identifier based on the value of ifName object (defined in
            IETF RFC 2863) for an interface on the containing chassis.

            The enumeration 'local(7)' represents a chassis identifier
            based on a locally defined value."
    SYNTAX  INTEGER {
            chassisComponent(1),
            interfaceAlias(2),
            portComponent(3),
            macAddress(4),
            networkAddress(5),
            interfaceName(6),
            local(7)
    }

LldpV2ChassisId ::= TEXTUAL-CONVENTION
    DISPLAY-HINT "1x:"
    STATUS      current
    DESCRIPTION
            "This TC describes the format of a chassis identifier string.
            Objects of this type are always used with an associated
            LldpChassisIdSubtype object, which identifies the format of
            the particular LldpChassisId object instance.

            If the associated LldpChassisIdSubtype object has a value of
            'chassisComponent(1)', then the octet string identifies
            a particular instance of the entPhysicalAlias object
            (defined in IETF RFC 6933) for a chassis component (i.e.,
            an entPhysicalClass value of 'chassis(3)').

            If the associated LldpChassisIdSubtype object has a value
            of 'interfaceAlias(2)', then the octet string identifies
            a particular instance of the ifAlias object (defined in
            IETF RFC 2863) for an interface on the containing chassis.
            If the particular ifAlias object does not contain any values,
            another chassis identifier type should be used.

            If the associated LldpChassisIdSubtype object has a value
            of 'portComponent(3)', then the octet string identifies a
            particular instance of the entPhysicalAlias object (defined
            in IETF RFC 6933) for a port or backplane component within
            the containing chassis.

            If the associated LldpChassisIdSubtype object has a value of
            'macAddress(4)', then this string identifies a particular
            unicast source address (encoded in network byte order and
            IEEE 802.3 canonical bit order), of a port on the containing
            chassis as defined in IEEE Std 802.

            If the associated LldpChassisIdSubtype object has a value of
            'networkAddress(5)', then this string identifies a particular
            network address, encoded in network byte order, associated
            with one or more ports on the containing chassis. The first
            octet contains the IANA Address Family Numbers enumeration
            value for the specific address type, and octets 2 through
            N contain the network address value in network byte order.

            If the associated LldpChassisIdSubtype object has a value
```

```
            of 'interfaceName(6)', then the octet string identifies
            a particular instance of the ifName object (defined in
            IETF RFC 2863) for an interface on the containing chassis.
            If the particular ifName object does not contain any values,
            another chassis identifier type should be used.

            If the associated LldpChassisIdSubtype object has a value of
            'local(7)', then this string identifies a locally assigned
            Chassis ID."
    SYNTAX      OCTET STRING (SIZE (1..255))

LldpV2PortIdSubtype ::= TEXTUAL-CONVENTION
    STATUS      current
    DESCRIPTION
            "This TC describes the source of a particular type of port
            identifier used in the LLDP MIB.

            The enumeration 'interfaceAlias(1)' represents a port
            identifier based on the ifAlias MIB object, defined in IETF
            RFC 2863.

            The enumeration 'portComponent(2)' represents a port
            identifier based on the value of entPhysicalAlias (defined in
            IETF RFC 6933) for a port component (i.e., entPhysicalClass
            value of 'port(10)'), within the containing chassis.

            The enumeration 'macAddress(3)' represents a port identifier
            based on a unicast source address (encoded in network
            byte order and IEEE 802.3 canonical bit order), which has
            been detected by the agent and associated with a particular
            port (IEEE Std 802).

            The enumeration 'networkAddress(4)' represents a port
            identifier based on a network address, detected by the agent
            and associated with a particular port.

            The enumeration 'interfaceName(5)' represents a port
            identifier based on the ifName MIB object, defined in IETF
            RFC 2863.

            The enumeration 'agentCircuitId(6)' represents a port
            identifier based on the agent-local identifier of the circuit
            (defined in IETF RFC 3046), detected by the agent and associated
            with a particular port.

            The enumeration 'local(7)' represents a port identifier
            based on a value locally assigned."

    SYNTAX  INTEGER {
            interfaceAlias(1),
            portComponent(2),
            macAddress(3),
            networkAddress(4),
            interfaceName(5),
            agentCircuitId(6),
            local(7)
    }

LldpV2PortId ::= TEXTUAL-CONVENTION
```

```
    DISPLAY-HINT "1x:"
    STATUS      current
    DESCRIPTION
            "This TC describes the format of a port identifier string.
            Objects of this type are always used with an associated
            LldpPortIdSubtype object, which identifies the format of the
            particular LldpPortId object instance.

            If the associated LldpPortIdSubtype object has a value of
            'interfaceAlias(1)', then the octet string identifies a
            particular instance of the ifAlias object (defined in IETF
            RFC 2863). If the particular ifAlias object does not contain
            any values, another port identifier type should be used.

            If the associated LldpPortIdSubtype object has a value of
            'portComponent(2)', then the octet string identifies a
            particular instance of the entPhysicalAlias object (defined
            in IETF RFC 6933) for a port or backplane component.

            If the associated LldpPortIdSubtype object has a value of
            'macAddress(3)', then this string identifies a particular
            unicast source address (encoded in network byte order
            and IEEE 802.3 canonical bit order) associated with the port
            (IEEE Std 802).

            If the associated LldpPortIdSubtype object has a value of
            'networkAddress(4)', then this string identifies a network
            address associated with the port. The first octet contains
            the IANA AddressFamilyNumbers enumeration value for the
            specific address type, and octets 2 through N contain the
            networkAddress address value in network byte order.

            If the associated LldpPortIdSubtype object has a value of
            'interfaceName(5)', then the octet string identifies a
            particular instance of the ifName object (defined in IETF
            RFC 2863). If the particular ifName object does not contain
            any values, another port identifier type should be used.

            If the associated LldpPortIdSubtype object has a value of
            'agentCircuitId(6)', then this string identifies a agent-local
            identifier of the circuit (defined in IETF RFC 3046).

            If the associated LldpPortIdSubtype object has a value of
            'local(7)', then this string identifies a locally
            assigned port ID."
    SYNTAX      OCTET STRING (SIZE (1..255))

LldpV2ManAddrIfSubtype ::= TEXTUAL-CONVENTION
    STATUS      current
    DESCRIPTION
            "This TC defines an enumeration value that identifies
            the interface numbering method used for defining the
            interface number associated with a management address.
            An object with this syntax defines the format of an
            interface number object.

            The enumeration 'unknown(1)' represents the case where the
            interface is not known. In this case, the corresponding
            interface number is of zero length.
```

```
            The enumeration 'ifIndex(2)' represents interface identifier
            based on the ifIndex MIB object.

            The enumeration 'systemPortNumber(3)' represents interface
            identifier based on the system port numbering convention."
    REFERENCE
            "8.5.9.5"

    SYNTAX  INTEGER {
            unknown(1),
            ifIndex(2),
            systemPortNumber(3)
    }

LldpV2ManAddress ::= TEXTUAL-CONVENTION
    DISPLAY-HINT "1x:"
    STATUS      current
    DESCRIPTION
            "The value of a management address associated with the LLDP
            agent that may be used to reach higher layer entities to
            assist discovery by network management.

            It should be noted that appropriate security credentials,
            such as SNMP engineId, may be required to access the LLDP
            agent using a management address. These necessary credentials
            should be known by the network management and the objects
            associated with the credentials are not included in the
            LLDP agent."
    SYNTAX      OCTET STRING (SIZE (1..31))

LldpV2SystemCapabilitiesMap ::= TEXTUAL-CONVENTION
    STATUS      current
    DESCRIPTION
            "This TC describes the system capabilities.

            The bit 'other(0)' indicates that the system has capabilities
            other than those listed below.

            The bit 'repeater(1)' indicates that the system has repeater
            capability.

            The bit 'bridge(2)' indicates that the system has bridge
            capability.

            The bit 'wlanAccessPoint(3)' indicates that the system has
            WLAN access point capability.

            The bit 'router(4)' indicates that the system has router
            capability.

            The bit 'telephone(5)' indicates that the system has telephone
            capability.

            The bit 'docsisCableDevice(6)' indicates that the system has
            DOCSIS Cable Device capability (IETF RFC 4639 & 2670).

            The bit 'stationOnly(7)' indicates that the system has only
            station capability and nothing else.
```

```
1
2               The bit 'cVLANComponent(8)' indicates that the system has
3               C-VLAN component functionality.
4
5               The bit 'sVLANComponent(8)' indicates that the system  has
6               S-VLAN component functionality.
7
8               The bit 'twoPortMACRelay(10)' indicates that the system has
9               Two-port MAC Relay (TPMR) functionality."
10      SYNTAX  BITS {
11              other(0),
12              repeater(1),
13              bridge(2),
14              wlanAccessPoint(3),
15              router(4),
16              telephone(5),
17              docsisCableDevice(6),
18              stationOnly(7),
19              cVLANComponent(8),
20              sVLANComponent(9),
21              twoPortMACRelay(10)
22
23      }
24
25
26  LldpV2DestAddressTableIndex ::= TEXTUAL-CONVENTION
27      DISPLAY-HINT "d"
28      STATUS    current
29      DESCRIPTION
30              "An index value, used as the index to the table of destination
31              MAC addresses used both as the destination addresses on
32              transmitted LLDPDUs and on received LLDPDUs. This index value
33              is also used as a secondary index value in tables indexed
34              by fields of type  ifIndex, in order to associate
35              a destination address with each row of the table."
36      SYNTAX  Unsigned32(1..4096)
37
38  LldpV2PowerPortClass ::= TEXTUAL-CONVENTION
39      STATUS      current
40      DESCRIPTION
41              "This TC describes the Power over Ethernet (PoE) port class."
42      SYNTAX  INTEGER {
43              pClassPSE(1),
44              pClassPD(2)
45      }
46
47  END
48
```

11.5.2 LLDP MIB module - version 2

In the following MIB module, should any discrepancy between the DESCRIPTION text and the corresponding definition in Clause 9 and Clause 10 occur, the definition in Clause 9 and Clause 10 shall take precedence.

```
LLDP-V2-MIB DEFINITIONS ::= BEGIN
IMPORTS
    MODULE-IDENTITY,
    OBJECT-TYPE,
    Unsigned32,
    Counter32,
    NOTIFICATION-TYPE
        FROM SNMPv2-SMI
    TimeStamp,
    TruthValue,
    MacAddress,
    RowStatus
        FROM SNMPv2-TC
    SnmpAdminString
        FROM SNMP-FRAMEWORK-MIB
    MODULE-COMPLIANCE,
    OBJECT-GROUP,
    NOTIFICATION-GROUP
        FROM SNMPv2-CONF
    TimeFilter,
    ZeroBasedCounter32
        FROM RMON2-MIB
    AddressFamilyNumbers
        FROM IANA-ADDRESS-FAMILY-NUMBERS-MIB
    ifGeneralInformationGroup,
    InterfaceIndex
        FROM IF-MIB
    LldpV2ChassisIdSubtype,
    LldpV2ChassisId,
    LldpV2PortIdSubtype,
    LldpV2PortId,
    LldpV2ManAddrIfSubtype,
    LldpV2ManAddress,
    LldpV2SystemCapabilitiesMap,
    LldpV2DestAddressTableIndex,
    ieee802dot1mibs
        FROM LLDP-V2-TC-MIB;

lldpV2MIB MODULE-IDENTITY
    LAST-UPDATED "202508150000Z" -- August 15, 2025
    ORGANIZATION "IEEE 802.1 Working Group"
    CONTACT-INFO
            "WG-URL: http://www.ieee802.org/1/
             WG-EMail: stds-802-1-L@ieee.org

            Contact: IEEE 802.1 Working Group Chair
            Postal:  IEEE Standards Board
                     445 Hoes Lane
                     Piscataway, NJ 08854
                     USA
            E-mail:  stds-802-1-L@ieee.org"
    DESCRIPTION
            "Management Information Base module for LLDP configuration,
```

```
            statistics, local system data and remote systems data
            components.

            This MIB module supports the architecture described in
            Clause 6, where multiple LLDP agents can be associated with
            a single Port, each supporting transmission by means of a
            different MAC address.

            Unless otherwise indicated, the references in this
            MIB module are to IEEE Std 802.1AB-2026.

            Copyright (C) IEEE (2025). This version of this MIB module
            is published as 11.5.2 of IEEE Std 802.1AB-2026;
            see the standard itself for full legal notices."

       REVISION "202508150000Z" -- August 15, 2025

    DESCRIPTION
            "Published as part of the 2026 revision of
            IEEE Std 802.1AB.
            This revision updates references to 802.1AB and
                    copyright dates."

    REVISION "201603110000Z" -- March 11, 2016

    DESCRIPTION
            "Published as part of the 2016 revision of
            IEEE Std 802.1AB.
            This revision incorporated changes to the MIB to
            address issues identified in maintenance item
            0121 - see http://www.ieee802.org/1/maint.html.
            The changes involved correcting the way that
            lldpV2MessageTxHoldMultiplier is calculated, in
            accordance with 9.2.5.23."


    REVISION "201502160000Z" -- February 16, 2015

    DESCRIPTION
            "Published as part of IEEE Std 802.1AB-2009 Cor-2.
            This corrigendum incorporated changes to the MIB to
            address issues identified in maintenance item
            0121 - see http://www.ieee802.org/1/maint.html."

    REVISION "200906080000Z" -- June 08, 2009

    DESCRIPTION
            "Published as part of IEEE Std 802.1AB-2009 revision.
            This revision incorporated changes to the MIB to
            support the use of LLDP with multiple destination MAC
            addresses."

   ::= { ieee802dot1mibs 13 }

lldpV2Notifications            OBJECT IDENTIFIER ::= { lldpV2MIB 0 }
lldpV2Objects                  OBJECT IDENTIFIER ::= { lldpV2MIB 1 }
lldpV2Conformance              OBJECT IDENTIFIER ::= { lldpV2MIB 2 }

--
```

```
-- LLDP MIB Objects
--

lldpV2Configuration            OBJECT IDENTIFIER ::= { lldpV2Objects 1 }
lldpV2Statistics               OBJECT IDENTIFIER ::= { lldpV2Objects 2 }
lldpV2LocalSystemData          OBJECT IDENTIFIER ::= { lldpV2Objects 3 }
lldpV2RemoteSystemsData        OBJECT IDENTIFIER ::= { lldpV2Objects 4 }
lldpV2Extensions               OBJECT IDENTIFIER ::= { lldpV2Objects 5 }

--
-- ************************************************************
--
--                  L L D P    C O N F I G
--
-- ************************************************************
--

lldpV2MessageTxInterval OBJECT-TYPE
    SYNTAX      Unsigned32(5..32768)
    UNITS       "seconds"
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
            "The interval at which LLDP frames are transmitted on
            behalf of this LLDP agent.

            The default value for lldpV2MessageTxInterval object is
            30 seconds.

            The value of this object is used as the initial value of
            the lldpV2PortMessageTxInterval object on row creation in
            the lldpV2PortConfigTableV2.

            The value of this object is restored from non-volatile
            storage after a re-initialization of the management system."
    REFERENCE
            "9.2.5.6"
    DEFVAL     { 30 }
    ::= { lldpV2Configuration 1 }

lldpV2MessageTxHoldMultiplier OBJECT-TYPE
    SYNTAX      Unsigned32(2..10)
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
            "The time to live value expressed as a multiple of the
            lldpV2MessageTxInterval object. The actual time to live
            value used in LLDP frames, transmitted on behalf of this
            LLDP agent, can be expressed by the following formula:
            TTL = min(65535,
            (lldpV2MessageTxInterval*lldpV2MessageTxHoldMultiplier)+1)
            For example, if the value of lldpV2MessageTxInterval is
            '30', and the value of lldpV2MessageTxHoldMultiplier
            is '4', then the value '121' is encoded in the
            TTL field in the LLDP header.

            The default value for lldpV2MessageTxHoldMultiplier
            object is 4.

```

```
            The value of this object is used as the initial value of
            the lldpV2PortMessageTxHoldMultiplier object on row creation in
            the lldpV2PortConfigTableV2.

            The value of this object is restored from non-volatile
            storage after a re-initialization of the management system."
    REFERENCE
            "9.2.5.5"
    DEFVAL      { 4 }
    ::= { lldpV2Configuration 2 }

lldpV2ReinitDelay OBJECT-TYPE
    SYNTAX      Unsigned32(1..10)
    UNITS       "seconds"
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
            "The lldpV2ReinitDelay indicates the delay (in units of
            seconds) from when lldpPortConfigAdminStatus object of a
            particular port becomes 'disabled' until re-initialization
            is attempted.

            The default value for lldpV2ReinitDelay is 2 s.

            The value of this object is used as the initial value of
            the lldpV2PortReinitDelay object on row creation in
            the lldpV2PortConfigTableV2.

            The value of this object is restored from non-volatile
            storage after a re-initialization of the management system."
    REFERENCE
            "9.2.5.9"
    DEFVAL      { 2 }
    ::= { lldpV2Configuration 3 }

lldpV2NotificationInterval OBJECT-TYPE
    SYNTAX      Unsigned32(5..3600)
    UNITS       "seconds"
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
            "This object controls the interval between transmission of
            LLDP notifications during normal transmission periods.

            The value of this object is used as the initial value of
            the lldpV2PortNotificationInterval object on row creation in
            the lldpV2PortConfigTableV2.

            The value of this object is restored from non-volatile
            storage after a re-initialization of the management system."
    DEFVAL { 30 }
    ::= { lldpV2Configuration 4 }

lldpV2TxCreditMax OBJECT-TYPE
    SYNTAX Unsigned32(1..100)
    UNITS "PDUs"
    MAX-ACCESS read-write
    STATUS current
    DESCRIPTION
```

```
            "The maximum number of consecutive LLDPDUs that can be
            transmitted at any time.

            The default value for lldpV2TxCreditMax object is 5 PDUs.

            The value of this object is used as the initial value of
            the lldpV2PortTxCreditMax object on row creation in
            the lldpV2PortConfigTableV2.

            The value of this object is restored from non-volatile
            storage after a re-initialization of the management system."
    REFERENCE
            "9.2.5.18"
    DEFVAL { 5 }
    ::= { lldpV2Configuration 5 }

lldpV2MessageFastTx OBJECT-TYPE
    SYNTAX Unsigned32(1..3600)
    UNITS "seconds"
    MAX-ACCESS read-write
    STATUS current
    DESCRIPTION
                "The interval at which LLDP frames are transmitted on
                behalf of this LLDP agent during fast transmission period
                (e.g., when a new neighbor is detected).

                The default value for lldpV2MessageFastTx object is
                1 second.

                The value of this object is used as the initial value of
                the lldpV2PortMessageFastTx object on row creation in
                the lldpV2PortConfigTableV2.

                The value of this object is restored from non-volatile
                storage after a re-initialization of the management system."
    REFERENCE
            "9.2.5.4"
    DEFVAL { 1 }
    ::= { lldpV2Configuration 6 }

lldpV2TxFastInit OBJECT-TYPE
    SYNTAX Unsigned32(1..8)
    MAX-ACCESS read-write
    STATUS current
    DESCRIPTION
                "The initial value used to initialize the txFast variable
                which determines the number of transmissions that are
                made in fast transmission mode.

                The default value for lldpV2TxFastInit object is 4.

                The value of this object is used as the initial value of
                the lldpV2PortTxFastInit object on row creation in
                the lldpV2PortConfigTableV2.

                The value of this object is restored from non-volatile
                storage after a re-initialization of the management system."
    REFERENCE
            "9.2.5.20"
```

```
    DEFVAL { 4 }
    ::= { lldpV2Configuration 7 }
--
-- lldpV2PortConfigTable: LLDP configuration indexed on a per port,
-- per destination address basis. The  ifIndex, coupled with an
-- index into the lldpDestAddressTable, is used to index per port
-- per destination MAC address.
-- ***This table and its associated objects are now deprecated
-- and replaced by lldpV2PortConfigTableV2.***
--

lldpV2PortConfigTable   OBJECT-TYPE
    SYNTAX      SEQUENCE OF LldpV2PortConfigEntry
    MAX-ACCESS  not-accessible
    STATUS      deprecated
    DESCRIPTION
            "The table that controls LLDP frame transmission on individual
            ports that uses particular destination MAC addresses."
    ::= { lldpV2Configuration 8 }

lldpV2PortConfigEntry   OBJECT-TYPE
    SYNTAX      LldpV2PortConfigEntry
    MAX-ACCESS  not-accessible
    STATUS      deprecated
    DESCRIPTION
            "LLDP configuration information for a particular port and
            destination MAC address.

            This configuration parameter controls the transmission and
            the reception of LLDP frames on those interface/address
            combinations whose rows are created in this table.

            Rows in this table can only be created for MAC addresses
            that can validly be used in association with the type of
            interface concerned, as defined by Table 7-2.

            The contents of this table is persistent across
            re-initializations or re-boots."
     INDEX  { lldpV2PortConfigIfIndex,
              lldpV2PortConfigDestAddressIndex  }
    ::= { lldpV2PortConfigTable 1 }

LldpV2PortConfigEntry ::= SEQUENCE {
      lldpV2PortConfigIfIndex             InterfaceIndex,
      lldpV2PortConfigDestAddressIndex    LldpV2DestAddressTableIndex,
      lldpV2PortConfigAdminStatus         INTEGER,
      lldpV2PortConfigNotificationEnable  TruthValue,
      lldpV2PortConfigTLVsTxEnable        BITS }

lldpV2PortConfigIfIndex   OBJECT-TYPE
    SYNTAX      InterfaceIndex
    MAX-ACCESS  not-accessible
    STATUS      deprecated
    DESCRIPTION
            "The interface index value used to identify the port
            associated with this entry. Its value is an index into
            the interfaces MIB.

            The value of this object is used as an index to the
```

```
            lldpV2PortConfigTable."
    ::= { lldpV2PortConfigEntry 1 }

lldpV2PortConfigDestAddressIndex   OBJECT-TYPE
    SYNTAX      LldpV2DestAddressTableIndex
    MAX-ACCESS  not-accessible
    STATUS      deprecated
    DESCRIPTION
            "The index value used to identify the destination
            MAC address associated with this entry. Its value identifies
            the row in the lldpV2DestAddressTable where the MAC address
            can be found.

            The value of this object is used as an index to the
            lldpV2PortConfigTable."
    ::= { lldpV2PortConfigEntry 2 }

lldpV2PortConfigAdminStatus  OBJECT-TYPE
    SYNTAX INTEGER {
       txOnly(1),
       rxOnly(2),
       txAndRx(3),
       disabled(4)
    }
    MAX-ACCESS read-write
    STATUS     deprecated
    DESCRIPTION
            "The administratively desired status of the local LLDP agent.

            If the associated lldpV2PortConfigAdminStatus object is
            set to a value of 'txOnly(1)', then LLDP agent transmits
            LLDPframes on this port and it does not store any
            information about the remote systems connected.

            If the associated lldpV2PortConfigAdminStatus object is
            set to a value of 'rxOnly(2)', then the LLDP agent
            receives, but it does not transmit LLDP frames on this port.

            If the associated lldpV2PortConfigAdminStatus object is set
            to a value of 'txAndRx(3)', then the LLDP agent transmits
            and receives LLDP frames on this port.

            If the associated lldpV2PortConfigAdminStatus object is set
            to a value of 'disabled(4)', then LLDP agent does not
            transmit or receive LLDP frames on this port. If there is
            remote systems information that is received on this port
            and stored in other tables, before the port's
            lldpV2PortConfigAdminStatus becomes disabled, then that
            information is deleted."
    REFERENCE
            "9.2.5.1"
    DEFVAL  { txAndRx }
   ::= { lldpV2PortConfigEntry 3 }

lldpV2PortConfigNotificationEnable OBJECT-TYPE
    SYNTAX     TruthValue
    MAX-ACCESS read-write
    STATUS     deprecated
    DESCRIPTION
```

```
            "The lldpV2PortConfigNotificationEnable controls, on a per
            agent basis, whether or not notifications from the agent
            are enabled. The value true(1) means that notifications are
            enabled; the value false(2) means that they are not."
    DEFVAL  { false }
   ::= { lldpV2PortConfigEntry 4 }

lldpV2PortConfigTLVsTxEnable OBJECT-TYPE
    SYNTAX      BITS {
            portDesc(0),
            sysName(1),
            sysDesc(2),
            sysCap(3)
    }
    MAX-ACCESS  read-write
    STATUS      deprecated
    DESCRIPTION
            "The lldpV2PortConfigTLVsTxEnable, defined as a bitmap,
            includes the basic set of LLDP TLVs whose transmission is
            allowed on the local LLDP agent by the network management.
            Each bit in the bitmap corresponds to a TLV type associated
            with a specific optional TLV.

            It should be noted that the organizationally-specific TLVs
            are excluded from the lldpV2PortConfigTLVsTxEnable bitmap.

            LLDP Organization Specific Information Extension MIBs should
            have similar configuration objects to control transmission
            of their organizationally defined TLVs.

            The bit 'portDesc(0)' indicates that LLDP agent should
            transmit 'Port Description TLV'.

            The bit 'sysName(1)' indicates that LLDP agent should transmit
            'System Name TLV'.

            The bit 'sysDesc(2)' indicates that LLDP agent should transmit
            'System Description TLV'.

            The bit 'sysCap(3)' indicates that LLDP agent should transmit
            'System Capabilities TLV'.

            There is no bit reserved for the management address TLV type
            since transmission of management address TLVs are controlled
            by another object, lldpV2ConfigManAddrTable.

            The default value for lldpV2PortConfigTLVsTxEnable object is
            empty set, which means no enumerated values are set.

            The value of this object is restored from non-volatile
            storage after a re-initialization of the management system."
    REFERENCE
            "9.1.2.1"
    DEFVAL  { { } }
    ::= { lldpV2PortConfigEntry 5 }

--
-- lldpV2PortConfigTableV2: LLDP configuration indexed on a per port,
-- per destination address basis. The  ifIndex, coupled with an
```

```
-- index into the lldpDestAddressTable, is used to index per port
-- per destination MAC address.
--
-- V2 extends the original table definition to include per-port
-- per-MAC address parameters msgTxInterval, msgTxHold, reinitDelay,
-- notificationInterval, txCreditMax, msgFastTx, and txFastInit.
--

lldpV2PortConfigTableV2   OBJECT-TYPE
    SYNTAX      SEQUENCE OF LldpV2PortConfigEntryV2
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The table that controls LLDP frame transmission on individual
            ports and using particular destination MAC addresses."
    ::= { lldpV2Configuration 11 }

lldpV2PortConfigEntryV2   OBJECT-TYPE
    SYNTAX      LldpV2PortConfigEntryV2
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "LLDP configuration information for a particular port and
            destination MAC address.

            This configuration parameter controls the transmission and
            the reception of LLDP frames on those interface/address
            combinations whose rows are created in this table.

            Rows in this table can only be created for MAC addresses
            that can validly be used in association with the type of
            interface concerned, as defined by Table 7-2.

            The contents of this table is persistent across
            re-initializations or re-boots."
     INDEX  { lldpV2PortConfigIfIndexV2,
              lldpV2PortConfigDestAddressIndexV2  }
    ::= { lldpV2PortConfigTableV2 1 }

LldpV2PortConfigEntryV2 ::= SEQUENCE {
      lldpV2PortConfigIfIndexV2             InterfaceIndex,
      lldpV2PortConfigDestAddressIndexV2    LldpV2DestAddressTableIndex,
      lldpV2PortConfigAdminStatusV2         INTEGER,
      lldpV2PortMessageTxInterval           Unsigned32,
      lldpV2PortMessageTxHoldMultiplier     Unsigned32,
      lldpV2PortReinitDelay                 Unsigned32,
      lldpV2PortNotificationInterval        Unsigned32,
      lldpV2PortTxCreditMax                 Unsigned32,
      lldpV2PortMessageFastTx               Unsigned32,
      lldpV2PortTxFastInit                  Unsigned32,
      lldpV2PortConfigNotificationEnableV2  TruthValue,
      lldpV2PortConfigTLVsTxEnableV2        BITS }

lldpV2PortConfigIfIndexV2   OBJECT-TYPE
    SYNTAX      InterfaceIndex
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The interface index value used to identify the port
```

```
            associated with this entry. Its value is an index into
            the interfaces MIB.

            The value of this object is used as an index to the
            lldpV2PortConfigTable."
    ::= { lldpV2PortConfigEntryV2 1 }

lldpV2PortConfigDestAddressIndexV2   OBJECT-TYPE
    SYNTAX      LldpV2DestAddressTableIndex
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The index value used to identify the destination
            MAC address associated with this entry. Its value identifies
            the row in the lldpV2DestAddressTable where the MAC address
            can be found.

            The value of this object is used as an index to the
            lldpV2PortConfigTable."
    ::= { lldpV2PortConfigEntryV2 2 }

lldpV2PortConfigAdminStatusV2  OBJECT-TYPE
    SYNTAX INTEGER {
       txOnly(1),
       rxOnly(2),
       txAndRx(3),
       disabled(4)
    }
    MAX-ACCESS read-write
    STATUS     current
    DESCRIPTION
            "The administratively desired status of the local LLDP agent.

            If the associated lldpV2PortConfigAdminStatus object is
            set to a value of 'txOnly(1)', then LLDP agent transmits
            LLDPframes on this port and it does not store any
            information about the remote systems connected.

            If the associated lldpV2PortConfigAdminStatus object is
            set to a value of 'rxOnly(2)', then the LLDP agent
            receives, but it does not transmit, LLDP frames on this port.

            If the associated lldpV2PortConfigAdminStatus object is set
            to a value of 'txAndRx(3)', then the LLDP agent transmits
            and receives LLDP frames on this port.

            If the associated lldpV2PortConfigAdminStatus object is set
            to a value of 'disabled(4)', then LLDP agent does not
            transmit or receive LLDP frames on this port. If there is
            remote systems information that is received on this port
            and stored in other tables, before the port's
            lldpV2PortConfigAdminStatus becomes disabled, then that
            information is deleted."
    REFERENCE
            "9.2.5.1"
    DEFVAL  { txAndRx }
   ::= { lldpV2PortConfigEntryV2 3 }

lldpV2PortMessageTxInterval OBJECT-TYPE
```

```
    SYNTAX      Unsigned32(5..32768)
    UNITS       "seconds"
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
            "The interval at which LLDP frames are transmitted on
            behalf of this LLDP agent.

            This object takes its initial value from the
            lldpV2MessageTxInterval object on table row creation.

            The value of this object is restored from non-volatile
            storage after a re-initialization of the management system."
    REFERENCE
            "9.2.5.6"
    DEFVAL     { 30 }
    ::= { lldpV2PortConfigEntryV2 4 }

lldpV2PortMessageTxHoldMultiplier OBJECT-TYPE
    SYNTAX      Unsigned32(2..10)
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
            "The time to live value expressed as a multiple of the
            lldpV2MessageTxInterval object. The actual time to live
            value used in LLDP frames, transmitted on behalf of this
            LLDP agent, can be expressed by the following formula:
            TTL = min(65535,
            (lldpV2MessageTxInterval*lldpV2MessageTxHoldMultiplier)+1)
            For example, if the value of lldpV2MessageTxInterval is
            '30', and the value of lldpV2MessageTxHoldMultiplier
            is '4', then the value '121' is encoded in the
            TTL field in the LLDP header.

            This object takes its initial value from the
            lldpV2PortMessageTxHoldMultiplier object on table row creation.

            The value of this object is restored from non-volatile
            storage after a re-initialization of the management system."
    REFERENCE
            "9.2.5.5"
    DEFVAL      { 4 }
    ::= { lldpV2PortConfigEntryV2 5 }

lldpV2PortReinitDelay OBJECT-TYPE
    SYNTAX      Unsigned32(1..10)
    UNITS       "seconds"
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
            "The lldpV2ReinitDelay indicates the delay (in units of
            seconds) from when lldpPortConfigAdminStatus object of a
            particular port becomes 'disabled' until re-initialization
            is attempted.

            This object takes its initial value from the
            lldpV2PortReinitDelay object on table row creation.

            The value of this object is restored from non-volatile
```

```
            storage after a re-initialization of the management system."
    REFERENCE
            "9.2.5.9"
    DEFVAL      { 2 }
    ::= { lldpV2PortConfigEntryV2 6 }

lldpV2PortNotificationInterval OBJECT-TYPE
    SYNTAX      Unsigned32(5..3600)
    UNITS       "seconds"
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
            "This object controls the interval between transmission of
            LLDP notifications during normal transmission periods.

            This object takes its initial value from the
            lldpV2PortNotificationInterval object on table row creation.

            The value of this object is restored from non-volatile
            storage after a re-initialization of the management system."
    DEFVAL { 30 }
    ::= { lldpV2PortConfigEntryV2 7 }

lldpV2PortTxCreditMax OBJECT-TYPE
    SYNTAX Unsigned32(1..100)
    UNITS "PDUs"
    MAX-ACCESS read-write
    STATUS current
    DESCRIPTION
            "The maximum number of consecutive LLDPDUs that can be
            transmitted at any time.

            This object takes its initial value from the
            lldpV2PortTxCreditMax object on table row creation.

            The value of this object is restored from non-volatile
            storage after a re-initialization of the management system."
    REFERENCE
            "9.2.5.18"
    DEFVAL { 5 }
    ::= { lldpV2PortConfigEntryV2 8 }

lldpV2PortMessageFastTx OBJECT-TYPE
    SYNTAX Unsigned32(1..3600)
    UNITS "seconds"
    MAX-ACCESS read-write
    STATUS current
    DESCRIPTION
                "The interval at which LLDP frames are transmitted on
                behalf of this LLDP agent during fast transmission period
                (e.g., when a new neighbor is detected).

                This object takes its initial value from the
                lldpV2PortMessageFastTx object on table row creation.

                The value of this object is restored from non-volatile
                storage after a re-initialization of the management system."
    REFERENCE
            "9.2.5.4"
```

```
    DEFVAL { 1 }
    ::= { lldpV2PortConfigEntryV2 9 }

lldpV2PortTxFastInit OBJECT-TYPE
    SYNTAX Unsigned32(1..8)
    MAX-ACCESS read-write
    STATUS current
    DESCRIPTION
                "The initial value used to initialize the txFast variable
                which determines the number of transmissions that are
                made in fast transmission mode.

                This object takes its initial value from the
                lldpV2PortTxFastInit object on table row creation.

                The value of this object is restored from non-volatile
                storage after a re-initialization of the management system."
    REFERENCE
            "9.2.5.20"
    DEFVAL { 4 }
    ::= { lldpV2PortConfigEntryV2 10 }


lldpV2PortConfigNotificationEnableV2 OBJECT-TYPE
    SYNTAX     TruthValue
    MAX-ACCESS read-write
    STATUS     current
    DESCRIPTION
            "The lldpV2PortConfigNotificationEnableV2 controls, on a per
            agent basis, whether or not notifications from the agent
            are enabled. The value true(1) means that notifications are
            enabled; the value false(2) means that they are not."
    DEFVAL  { false }
   ::= { lldpV2PortConfigEntryV2 11 }

lldpV2PortConfigTLVsTxEnableV2 OBJECT-TYPE
    SYNTAX      BITS {
            portDesc(0),
            sysName(1),
            sysDesc(2),
            sysCap(3)
    }
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION
            "The lldpV2PortConfigTLVsTxEnableV2, defined as a bitmap,
            includes the basic set of LLDP TLVs whose transmission is
            allowed on the local LLDP agent by the network management.
            Each bit in the bitmap corresponds to a TLV type associated
            with a specific optional TLV.

            It should be noted that the organizationally-specific TLVs
            are excluded from the lldpV2PortConfigTLVsTxEnable bitmap.

            LLDP Organization Specific Information Extension MIBs should
            have similar configuration objects to control transmission
            of their organizationally defined TLVs.

            The bit 'portDesc(0)' indicates that LLDP agent should
```

```
            transmit 'Port Description TLV'.

            The bit 'sysName(1)' indicates that LLDP agent should transmit
            'System Name TLV'.

            The bit 'sysDesc(2)' indicates that LLDP agent should transmit
            'System Description TLV'.

            The bit 'sysCap(3)' indicates that LLDP agent should transmit
            'System Capabilities TLV'.

            There is no bit reserved for the management address TLV type
            since transmission of management address TLVs are controlled
            by another object, lldpV2ConfigManAddrTable.

            The default value for lldpV2PortConfigTLVsTxEnable object is
            empty set, which means no enumerated values are set.

            The value of this object is restored from non-volatile
            storage after a re-initialization of the management system."
    REFERENCE
            "9.1.2.1"
    DEFVAL  { { } }
    ::= { lldpV2PortConfigEntryV2 12 }


--
-- lldpV2DestAddressTable: Destination MAC addresses used by LLDP
--

lldpV2DestAddressTable   OBJECT-TYPE
    SYNTAX      SEQUENCE OF LldpV2DestAddressTableEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The table that contains the set of MAC addresses used
            by LLDP for transmission and reception of LLDPDUs."
    ::= { lldpV2Configuration 9 }

lldpV2DestAddressTableEntry   OBJECT-TYPE
    SYNTAX      LldpV2DestAddressTableEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "Destination MAC address information for LLDP.

            This configuration parameter identifies a MAC address
            corresponding to a LldpV2DestAddressTableIndex value.

            Rows in this table are created as necessary, to support
            MAC addresses needed by other tables in the MIB that
            are indexed by MAC address.

            A given row in this table cannot be deleted if the MAC
            address table index value is in use in any other table
            in the MIB.

            The contents of this table are persistent across
            re-initializations or re-boots."
```

```
     INDEX  { lldpV2AddressTableIndex  }
    ::= { lldpV2DestAddressTable 1 }

LldpV2DestAddressTableEntry ::= SEQUENCE {
      lldpV2AddressTableIndex      LldpV2DestAddressTableIndex,
      lldpV2DestMacAddress         MacAddress  }

lldpV2AddressTableIndex   OBJECT-TYPE
    SYNTAX      LldpV2DestAddressTableIndex
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The index value used to identify the destination
            MAC address associated with this entry.

            The value of this object is used as an index to the
            lldpV2DestAddressTable."
    ::= { lldpV2DestAddressTableEntry 1 }

lldpV2DestMacAddress   OBJECT-TYPE
    SYNTAX      MacAddress
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The MAC address associated with this entry.

            The octet string identifies an individual or a group
            MAC address that is in use by LLDP as a destination
            MAC address.

            The MAC address is encoded in the octet string in
            canonical format (see IEEE Std 802)."
    ::= { lldpV2DestAddressTableEntry 2 }

--
-- lldpV2ManAddrConfigTxPortsTable : selection of management addresses
-- to be transmitted on a specified set of port/destination
-- address pairs.
--

lldpV2ManAddrConfigTxPortsTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF LldpV2ManAddrConfigTxPortsEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The table that controls selection of LLDP management address
            TLV instances to be transmitted on individual port/
            destination address pairs."
    ::= { lldpV2Configuration 10 }

lldpV2ManAddrConfigTxPortsEntry  OBJECT-TYPE
    SYNTAX      LldpV2ManAddrConfigTxPortsEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "LLDP configuration information that specifies the set
            of port/destination address pairs on which the local
            system management address instance is transmitted.

```

```
            Each active lldpManAddrConfigTxPortsTableV2Entry is
            restored from non-volatile storage and re-created
            after a re-initialization of the management system."
    INDEX {
            lldpV2ManAddrConfigIfIndex,
            lldpV2ManAddrConfigDestAddressIndex,
            lldpV2ManAddrConfigLocManAddrSubtype,
            lldpV2ManAddrConfigLocManAddr }
    ::= { lldpV2ManAddrConfigTxPortsTable 1 }

LldpV2ManAddrConfigTxPortsEntry  ::= SEQUENCE {
            lldpV2ManAddrConfigIfIndex            InterfaceIndex,
            lldpV2ManAddrConfigDestAddressIndex   LldpV2DestAddressTableIndex,
            lldpV2ManAddrConfigLocManAddrSubtype  AddressFamilyNumbers,
            lldpV2ManAddrConfigLocManAddr         LldpV2ManAddress,
            lldpV2ManAddrConfigTxEnable           TruthValue,
            lldpV2ManAddrConfigRowStatus          RowStatus
}

lldpV2ManAddrConfigIfIndex   OBJECT-TYPE
    SYNTAX      InterfaceIndex
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The interface index value used to identify the port
            associated with this entry. Its value is an index into
            the interfaces MIB.

            The value of this object is used as an index to the
            lldpV2PortConfigTable.

            The value in this column of the table MUST match
            the IfIndex value specified in the BridgePort table."
    ::= { lldpV2ManAddrConfigTxPortsEntry 1 }

lldpV2ManAddrConfigDestAddressIndex   OBJECT-TYPE
    SYNTAX      LldpV2DestAddressTableIndex
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The index value used to identify the destination
            MAC address associated with this entry. Its value identifies
            the row in the lldpV2DestAddressTable where the MAC address
            can be found.

            The value of this object is used as an index to the
            lldpV2PortConfigTable."
    ::= { lldpV2ManAddrConfigTxPortsEntry 2 }


lldpV2ManAddrConfigLocManAddrSubtype  OBJECT-TYPE
    SYNTAX      AddressFamilyNumbers
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The type of management address identifier encoding used in
            the associated 'lldpLocManagmentAddr' object.

            It should be noted that only a subset of the possible
```

```
            address encodings enumerated in AddressFamilyNumbers
            are appropriate for use as a LLDP management
            address, either because some are just not applicable or
            because the maximum size of a LldpV2ManAddress octet string
            would prevent the use of some address identifier encodings."
    REFERENCE
            "8.5.9.3"
    ::= { lldpV2ManAddrConfigTxPortsEntry 3 }

lldpV2ManAddrConfigLocManAddr  OBJECT-TYPE
    SYNTAX      LldpV2ManAddress
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The string value used to identify the management address
            component associated with the local system. The purpose of
            this address is to contact the management entity."
    REFERENCE
            "8.5.9.4"
    ::= { lldpV2ManAddrConfigTxPortsEntry 4 }



lldpV2ManAddrConfigTxEnable  OBJECT-TYPE
    SYNTAX        TruthValue
    MAX-ACCESS    read-create
    STATUS        current
    DESCRIPTION
            "A Boolean controlling the transmission of system
             management address instance for the specified port,
             destination, subtype, and MAN address used to index
             this table. If set to the default value of false,
             no transmission occurs. If set to true, the
             appropriate information is transmitted out of the
             port specified in the row's index."
    REFERENCE
            "9.1.2.1"
    DEFVAL  { false }     -- not transmitted
    ::= { lldpV2ManAddrConfigTxPortsEntry 5 }

lldpV2ManAddrConfigRowStatus OBJECT-TYPE
    SYNTAX RowStatus
    MAX-ACCESS read-create
    STATUS current
    DESCRIPTION
        "Indicates the status of an entry in this table, and is used
        to create/delete entries.
        The corresponding instances of the following objects
        must be set before this object can be made active(1):
           lldpV2ManAddrConfigDestAddressIndex
           lldpV2ManAddrConfigLocManAddrSubtype
           lldpV2ManAddrConfigLocManAddr
           lldpV2ManAddrConfigTxEnable

        The corresponding instances of the following objects
        may not be changed while this object is active(1):
           lldpV2ManAddrConfigDestAddressIndex
           lldpV2ManAddrConfigLocManAddrSubtype
           lldpV2ManAddrConfigLocManAddr "
```

```
::= { lldpV2ManAddrConfigTxPortsEntry 6 }

--
--  ********************************************************
--
--                   L L D P    S T A T S
--
--  ********************************************************
--
-- LLDP Stats Group

lldpV2StatsRemTablesLastChangeTime OBJECT-TYPE
    SYNTAX      TimeStamp
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The value of sysUpTime object (defined in IETF RFC 3418)
            at the time an entry is created, modified, or deleted in the
            in tables associated with the lldpV2RemoteSystemsData objects
            and all LLDP extension objects associated with remote systems.

            An NMS can use this object to reduce polling of the
            lldpV2RemoteSystemsData objects."
    ::= { lldpV2Statistics 1 }

lldpV2StatsRemTablesInserts OBJECT-TYPE
    SYNTAX      ZeroBasedCounter32
    UNITS       "table entries"
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The number of times the complete set of information
            advertised by a particular MSAP has been inserted into tables
            contained in lldpV2RemoteSystemsData and lldpV2Extensions objects.

            The complete set of information received from a particular
            MSAP should be inserted into related tables. If partial
            information cannot be inserted for a reason such as lack
            of resources, all of the complete set of information should
            be removed.

            This counter should be incremented only once after the
            complete set of information is successfully recorded
            in all related tables. Any failures during inserting
            information set that result in deletion of previously
            inserted information should not trigger any changes in
            lldpV2StatsRemTablesInserts since the insert is not completed
            yet or in lldpStatsRemTablesDeletes since the deletion
            would only be a partial deletion. If the failure was the
            result of lack of resources, the lldpStatsRemTablesDrops
            counter should be incremented once."
    ::= { lldpV2Statistics 2 }

lldpV2StatsRemTablesDeletes   OBJECT-TYPE
    SYNTAX      ZeroBasedCounter32
    UNITS       "table entries"
    MAX-ACCESS  read-only
    STATUS      current

```

```
    DESCRIPTION
            "The number of times the complete set of information
            advertised by a particular MSAP has been deleted from
            tables contained in lldpV2RemoteSystemsData and lldpV2Extensions
            objects.

            This counter should be incremented only once when the
            complete set of information is completely deleted from all
            related tables. Partial deletions, such as deletion of
            rows associated with a particular MSAP from some tables,
            but not from all tables are not allowed, thus should not
            change the value of this counter."
    ::= { lldpV2Statistics 3 }

lldpV2StatsRemTablesDrops  OBJECT-TYPE
    SYNTAX      ZeroBasedCounter32
    UNITS       "table entries"
    MAX-ACCESS  read-only

    STATUS      current
    DESCRIPTION
            "The number of times the complete set of information
            advertised by a particular MSAP could not be entered into
            tables contained in lldpV2RemoteSystemsData and lldpV2Extensions
            objects because of insufficient resources."
    ::= { lldpV2Statistics 4 }

lldpV2StatsRemTablesAgeouts   OBJECT-TYPE
    SYNTAX      ZeroBasedCounter32
    UNITS       "table entries"
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The number of times the complete set of information
            advertised by a particular MSAP has been deleted from tables
            contained in lldpV2RemoteSystemsData and lldpV2Extensions objects
            because the information timeliness interval has expired.

            This counter should be incremented only once when the complete
            set of information is completely invalidated (aged out)
            from all related tables. Partial ageing, similar to deletion
            case, is not allowed, and thus, should not change the value
            of this counter."
    ::= { lldpV2Statistics 5 }

--
-- TX statistics
-- Indexed by port (via ifIndex) and
-- destination MAC address.
--

lldpV2StatsTxPortTable  OBJECT-TYPE
    SYNTAX      SEQUENCE OF LldpV2StatsTxPortEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "A table containing LLDP transmission statistics for
            individual port/destination address combinations.
            Entries are not required to exist in
```

```
            this table while the lldpPortConfigEntry object is equal to
            'disabled(4)'."
    ::= { lldpV2Statistics 6 }

lldpV2StatsTxPortEntry   OBJECT-TYPE
     SYNTAX      LldpV2StatsTxPortEntry
     MAX-ACCESS  not-accessible
     STATUS      current
     DESCRIPTION
             "LLDP frame transmission statistics for a particular port
             and destination MAC address.
             The port is contained in the same chassis as the
             LLDP agent.

             All counter values in a particular entry shall be
             maintained on a continuing basis and shall not be deleted
             upon expiration of rxInfoTTL timing counters in the LLDP
             remote systems MIB of the receipt of a shutdown frame from
             a remote LLDP agent.

             All statistical counters associated with a particular
             port on the local LLDP agent become frozen whenever the
             adminStatus is disabled for the same port.

             Rows in this table can only be created for MAC addresses
             that can validly be used in association with the type of
             interface concerned, as defined by Table 7-2."
     INDEX  { lldpV2StatsTxIfIndex,
              lldpV2StatsTxDestMACAddress }
     ::= { lldpV2StatsTxPortTable 1 }

LldpV2StatsTxPortEntry ::= SEQUENCE {
      lldpV2StatsTxIfIndex               InterfaceIndex,
      lldpV2StatsTxDestMACAddress        LldpV2DestAddressTableIndex,
      lldpV2StatsTxPortFramesTotal       Counter32,
      lldpV2StatsTxLLDPDULengthErrors    Counter32  }

lldpV2StatsTxIfIndex   OBJECT-TYPE
    SYNTAX      InterfaceIndex
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The interface index value used to identify the port
            associated with this entry. Its value is an index
            into the interfaces MIB.

            The value of this object is used as an index to the
            lldpV2StatsTxPortTable."
    ::= { lldpV2StatsTxPortEntry 1 }

lldpV2StatsTxDestMACAddress   OBJECT-TYPE
    SYNTAX      LldpV2DestAddressTableIndex
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The index value used to identify the destination
            MAC address associated with this entry. Its value identifies
            the row in the lldpV2DestAddressTable where the MAC address
            can be found.
```

```
            The value of this object is used as an index to the
            lldpV2StatsTxPortTable."
    ::= { lldpV2StatsTxPortEntry 2 }

lldpV2StatsTxPortFramesTotal  OBJECT-TYPE
    SYNTAX        Counter32
    UNITS         "LLDP frames"
    MAX-ACCESS    read-only
    STATUS        current
    DESCRIPTION
            "The number of LLDP frames transmitted by this LLDP agent
            on the indicated port to the destination MAC address
            associated with this row of the table."
    REFERENCE
            "9.2.7.5"
    ::= { lldpV2StatsTxPortEntry 3 }

lldpV2StatsTxLLDPDULengthErrors  OBJECT-TYPE
    SYNTAX        Counter32
    UNITS         "LLDP frames"
    MAX-ACCESS    read-only
    STATUS        current
    DESCRIPTION
            "The number of LLDPDU Length Errors recorded for the Port."
    REFERENCE
            "9.2.7.8"
    ::= { lldpV2StatsTxPortEntry 4 }

--
-- lldpV2StatsRxPortTable - RX statistics
-- This table is indexed by ifIndex and destination MAC address.
--

lldpV2StatsRxPortTable  OBJECT-TYPE
    SYNTAX      SEQUENCE OF LldpV2StatsRxPortEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "A table containing LLDP reception statistics for individual
            ports and destination MAC addresses.
            Entries are not required to exist in this table while
            the lldpPortConfigEntry object is equal to 'disabled(4)'."
    ::= { lldpV2Statistics 7 }

lldpV2StatsRxPortEntry   OBJECT-TYPE
     SYNTAX      LldpV2StatsRxPortEntry
     MAX-ACCESS  not-accessible
     STATUS      current
     DESCRIPTION
             "LLDP frame reception statistics for a particular port.
             The port is contained in the same chassis as the
             LLDP agent.

             All counter values in a particular entry shall be
             maintained on a continuing basis and shall not be deleted
             upon expiration of rxInfoTTL timing counters in the LLDP
             remote systems MIB of the receipt of a shutdown frame from
             a remote LLDP agent.
```

```
1
2               All statistical counters associated with a particular
3               port on the local LLDP agent become frozen whenever the
4               adminStatus is disabled for the same port.
5
6               Rows in this table can only be created for MAC addresses
7               that can validly be used in association with the type of
8               interface concerned, as defined by Table 7-2.
9
10              The contents of this table is persistent across
11              re-initializations or re-boots."
12      INDEX  { lldpV2StatsRxDestIfIndex,
13               lldpV2StatsRxDestMACAddress }
14      ::= { lldpV2StatsRxPortTable 1 }
15
16 LldpV2StatsRxPortEntry ::= SEQUENCE {
17       lldpV2StatsRxDestIfIndex               InterfaceIndex,
18       lldpV2StatsRxDestMACAddress            LldpV2DestAddressTableIndex,
19       lldpV2StatsRxPortFramesDiscardedTotal  Counter32,
20       lldpV2StatsRxPortFramesErrors          Counter32,
21       lldpV2StatsRxPortFramesTotal           Counter32,
22       lldpV2StatsRxPortTLVsDiscardedTotal    Counter32,
23       lldpV2StatsRxPortTLVsUnrecognizedTotal Counter32,
24       lldpV2StatsRxPortAgeoutsTotal          ZeroBasedCounter32
25 }
26
27 lldpV2StatsRxDestIfIndex   OBJECT-TYPE
28     SYNTAX      InterfaceIndex
29     MAX-ACCESS  not-accessible
30     STATUS      current
31     DESCRIPTION
32             "The interface index value used to identify the port
33             associated with this entry. Its value is an index
34             into the interfaces MIB.
35
36             The value of this object is used as an index to the
37             lldpStatsRxPortV2Table."
38     ::= { lldpV2StatsRxPortEntry 1 }
39
40 lldpV2StatsRxDestMACAddress   OBJECT-TYPE
41     SYNTAX      LldpV2DestAddressTableIndex
42     MAX-ACCESS  not-accessible
43     STATUS      current
44     DESCRIPTION
45             "The index value used to identify the destination
46             MAC address associated with this entry. Its value identifies
47             the row in the lldpV2DestAddressTable where the MAC address
48             can be found.
49
50             The value of this object is used as an index to the
51             lldpStatsRxPortV2Table."
52     ::= { lldpV2StatsRxPortEntry 2 }
53
54
55 lldpV2StatsRxPortFramesDiscardedTotal OBJECT-TYPE
56     SYNTAX        Counter32
57     UNITS         "LLDP frames"
58     MAX-ACCESS read-only
59     STATUS     current
```

```
    DESCRIPTION
            "The number of LLDP frames received by this LLDP agent on
            the indicated port, and then discarded for any reason.
            This counter can provide an indication that LLDP header
            formatting problems may exist with the local LLDP agent in
            the sending system or that LLDPDU validation problems may
            exist with the local LLDP agent in the receiving system."
   REFERENCE
            "9.2.7.2"
   ::= { lldpV2StatsRxPortEntry 3 }

lldpV2StatsRxPortFramesErrors  OBJECT-TYPE
    SYNTAX        Counter32
    UNITS         "LLDP frames"
    MAX-ACCESS    read-only
    STATUS        current
    DESCRIPTION
            "The number of invalid LLDP frames received by this LLDP
            agent on the indicated port, while this LLDP agent is enabled."
    REFERENCE
            "9.2.7.3"
    ::= { lldpV2StatsRxPortEntry 4 }

lldpV2StatsRxPortFramesTotal OBJECT-TYPE
    SYNTAX        Counter32
    UNITS         "LLDP frames"
    MAX-ACCESS    read-only
    STATUS        current
    DESCRIPTION
            "The number of valid LLDP frames received by this LLDP agent
            on the indicated port, while this LLDP agent is enabled."
    REFERENCE
            "9.2.7.4"
    ::= { lldpV2StatsRxPortEntry 5 }

lldpV2StatsRxPortTLVsDiscardedTotal OBJECT-TYPE
    SYNTAX        Counter32
    UNITS         "TLVs"
    MAX-ACCESS    read-only
    STATUS        current
    DESCRIPTION
            "The number of LLDP TLVs discarded for any reason by this LLDP
            agent on the indicated port."
    REFERENCE
            "9.2.7.6"
    ::= { lldpV2StatsRxPortEntry 6 }

lldpV2StatsRxPortTLVsUnrecognizedTotal  OBJECT-TYPE
    SYNTAX        Counter32
    UNITS         "TLVs"
    MAX-ACCESS    read-only
    STATUS        current
    DESCRIPTION
            "The number of LLDP TLVs received on the given port that
            are not recognized by this LLDP agent on the indicated port.

            An unrecognized TLV is referred to as the TLV whose type value
            is in the range of reserved TLV types (000 1100 - 111 1110)
            in Table 8-1 of IEEE Std 802.1AB. An unrecognized
```

```
            TLV may be a basic management TLV from a later LLDP version."
    REFERENCE
            "9.2.7.7"
    ::= { lldpV2StatsRxPortEntry 7 }

lldpV2StatsRxPortAgeoutsTotal   OBJECT-TYPE
    SYNTAX      ZeroBasedCounter32
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The counter that represents the number of age-outs that
            occurred on a given port. An age-out is the number of
            times the complete set of information advertised by a
            particular MSAP has been deleted from tables contained in
            lldpV2RemoteSystemsData and lldpV2Extensions objects because
            the information timeliness interval has expired.

            This counter is similar to lldpV2StatsRemTablesAgeouts, except
            that the counter is on a per port basis. This enables NMS to
            poll tables associated with the lldpV2RemoteSystemsData objects
            and all LLDP extension objects associated with remote systems
            on the indicated port only.

            This counter is set to zero during agent initialization
            and its value should not be saved in non-volatile storage.

            This counter is incremented only once when the
            complete set of information is invalidated (aged out) from
            all related tables on a particular port. Partial ageing
            is not allowed."
    REFERENCE
            "9.2.7.1"
    ::= { lldpV2StatsRxPortEntry 8 }


--  ********************************************************
--
--          L O C A L    S Y S T E M    D A T A
--
--  ********************************************************

lldpV2LocChassisIdSubtype  OBJECT-TYPE
    SYNTAX      LldpV2ChassisIdSubtype
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The type of encoding used to identify the chassis
            associated with the local system."
    REFERENCE
            "8.5.2.2"
    ::= { lldpV2LocalSystemData 1 }

lldpV2LocChassisId  OBJECT-TYPE
    SYNTAX      LldpV2ChassisId
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The string value used to identify the chassis component
            associated with the local system."
```

```
    REFERENCE
            "8.5.2.3"
    ::= { lldpV2LocalSystemData 2 }

lldpV2LocSysName  OBJECT-TYPE
    SYNTAX      SnmpAdminString (SIZE(0..255))
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The string value used to identify the system name of the
            local system. If the local agent supports IETF RFC 3418,
            lldpLocSysName object should have the same value as sysName
            object."
    REFERENCE
            "8.5.6.2"
    ::= { lldpV2LocalSystemData 3 }

lldpV2LocSysDesc  OBJECT-TYPE
    SYNTAX      SnmpAdminString (SIZE(0..255))
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The string value used to identify the system description
            of the local system. If the local agent supports IETF RFC 3418,
            lldpLocSysDesc object should have the same value as sysDesc
            object."
    REFERENCE
            "8.5.7.2"
    ::= { lldpV2LocalSystemData 4 }

lldpV2LocSysCapSupported OBJECT-TYPE
    SYNTAX      LldpV2SystemCapabilitiesMap
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The bitmap value used to identify which system capabilities
            are supported on the local system."
    REFERENCE
            "8.5.8.1"
    ::= { lldpV2LocalSystemData 5 }

lldpV2LocSysCapEnabled  OBJECT-TYPE
    SYNTAX      LldpV2SystemCapabilitiesMap
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The bitmap value used to identify which system capabilities
            are enabled on the local system."
    REFERENCE
            "8.5.8.2"
    ::= { lldpV2LocalSystemData 6 }


--
-- lldpV2LocPortTable : Port specific Local system data
-- Indexed by  ifIndex.
--

lldpV2LocPortTable OBJECT-TYPE
```

```
    SYNTAX      SEQUENCE OF LldpV2LocPortEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "This table contains one row per port of information
            associated with the local system known to this agent."
    ::= { lldpV2LocalSystemData 7 }

lldpV2LocPortEntry OBJECT-TYPE
    SYNTAX      LldpV2LocPortEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "Information about a particular port component.

            Entries may be created and deleted in this table by the
            agent.

            Rows in this table can only be created for MAC addresses
            that can validly be used in association with the type of
            interface concerned, as defined by Table 7-2.

            The contents of this table is persistent across
            re-initializations or re-boots."
    INDEX   { lldpV2LocPortIfIndex }
    ::= { lldpV2LocPortTable 1 }

LldpV2LocPortEntry ::= SEQUENCE {
      lldpV2LocPortIfIndex            InterfaceIndex,
      lldpV2LocPortIdSubtype          LldpV2PortIdSubtype,
      lldpV2LocPortId                 LldpV2PortId,
      lldpV2LocPortDesc               SnmpAdminString
}

lldpV2LocPortIfIndex   OBJECT-TYPE
    SYNTAX      InterfaceIndex
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The interface index value used to identify the port
            associated with this entry. Its value is an index
            into the interfaces MIB.

            The value of this object is used as an index to the
            lldpV2LocPortTable."
    ::= { lldpV2LocPortEntry 1 }

lldpV2LocPortIdSubtype  OBJECT-TYPE
    SYNTAX      LldpV2PortIdSubtype
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The type of port identifier encoding used in the associated
            'lldpLocPortId' object."
    REFERENCE
            "8.5.3.2"
    ::= { lldpV2LocPortEntry 2 }

lldpV2LocPortId   OBJECT-TYPE
```

```
    SYNTAX      LldpV2PortId
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The string value used to identify the port component
            associated with a given port in the local system."
    REFERENCE
            "8.5.3.3"
    ::= { lldpV2LocPortEntry 3 }

lldpV2LocPortDesc   OBJECT-TYPE
    SYNTAX      SnmpAdminString (SIZE(0..255))
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The string value used to identify the IEEE 802 LAN station's port
            description associated with the local system. If the local
            agent supports IETF RFC 2863, lldpLocPortDesc object should
            have the same value of ifDescr object."
    REFERENCE
            "8.5.5.2"
    ::= { lldpV2LocPortEntry 4 }

--
-- lldpV2LocManAddrTable : Management addresses of the local system
--

lldpV2LocManAddrTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF LldpV2LocManAddrEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "This table contains management address information on the
            local system known to this agent."
    ::= { lldpV2LocalSystemData 8 }

lldpV2LocManAddrEntry OBJECT-TYPE
    SYNTAX      LldpV2LocManAddrEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "Management address information about a particular chassis
            component. There may be multiple management addresses
            configured on the system identified by a particular
            lldpLocChassisId. Each management address should have
            distinct 'management address type' (lldpV2LocManAddrSubtype) and
            'management address' (lldpLocManAddr.)

            Entries may be created and deleted in this table by the
            agent.

            Since a variable length octetstring is used as an index
            in a table, the address length is encoded as part of the OID
            (as per IETF RFC 2578)."

    INDEX   { lldpV2LocManAddrSubtype,
              lldpV2LocManAddr }
    ::= { lldpV2LocManAddrTable 1 }
```

```
LldpV2LocManAddrEntry ::= SEQUENCE {
      lldpV2LocManAddrSubtype    AddressFamilyNumbers,
      lldpV2LocManAddr           LldpV2ManAddress,
      lldpV2LocManAddrLen        Unsigned32,
      lldpV2LocManAddrIfSubtype  LldpV2ManAddrIfSubtype,
      lldpV2LocManAddrIfId       Unsigned32,
      lldpV2LocManAddrOID        OBJECT IDENTIFIER
}

lldpV2LocManAddrSubtype  OBJECT-TYPE
    SYNTAX      AddressFamilyNumbers
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The type of management address identifier encoding used in
            the associated 'lldpLocManagmentAddr' object.

            It should be noted that only a subset of the possible
            address encodings enumerated in AddressFamilyNumbers
            are appropriate for use as a LLDP management
            address, either because some are just not applicable or
            because the maximum size of a LldpV2ManAddress octet string
            would prevent the use of some address identifier encodings."
    REFERENCE
            "8.5.9.3"
    ::= { lldpV2LocManAddrEntry 1 }

lldpV2LocManAddr  OBJECT-TYPE
    SYNTAX      LldpV2ManAddress
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The string value used to identify the management address
            component associated with the local system. The purpose of
            this address is to contact the management entity."
    REFERENCE
            "8.5.9.4"
    ::= { lldpV2LocManAddrEntry 2 }

lldpV2LocManAddrLen  OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The total length of the management address subtype and the
            management address fields in LLDPDUs transmitted by the
            local LLDP agent.

            The management address length field is needed so that the
            receiving systems that do not implement SNMP are not
            required to implement an Internet Assigned Numbers
            Authority (IANA) family numbers/address length equivalency
            table in order to decode the management address."
    REFERENCE
            "8.5.9.2"
    ::= { lldpV2LocManAddrEntry 3 }

lldpV2LocManAddrIfSubtype  OBJECT-TYPE
    SYNTAX      LldpV2ManAddrIfSubtype
```

```
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The enumeration value that identifies the interface numbering
            method used for defining the interface number
            (lldpV2LocManAddrIfId), associated with the local system."
    REFERENCE
            "8.5.9.5"
    ::= { lldpV2LocManAddrEntry 4 }

lldpV2LocManAddrIfId  OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The integer value used to identify the interface number
            regarding the management address component associated with
            the local system."
    REFERENCE
            "8.5.9.6"
    ::= { lldpV2LocManAddrEntry 5 }

lldpV2LocManAddrOID  OBJECT-TYPE
    SYNTAX      OBJECT IDENTIFIER
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The OID value used to identify the type of hardware component
            or protocol entity associated with the management address
            advertised by the local system agent."
    REFERENCE
            "8.5.9.8"
    ::= { lldpV2LocManAddrEntry 6 }


--  ********************************************************
--
--          R E M O T E    S Y S T E M S    D A T A
--
--  ********************************************************


--
-- lldpV2RemTable
-- Indexed by ifIndex and destination MAC address.
--

lldpV2RemTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF LldpV2RemEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "This table contains one or more rows per physical network
            connection known to this agent. The agent may wish to ensure
            that only one lldpRemEntry is present for each local port
            and destination MAC address,
            or it may choose to maintain multiple lldpRemEntries for
            the same local port and destination MAC address.

```

```
            The following procedure may be used to retrieve remote
            systems information updates from an LLDP agent:

               1. NMS polls all tables associated with remote systems
                  and keeps a local copy of the information retrieved.
                  NMS polls periodically the values of the following
                  objects:
                     a. lldpV2StatsRemTablesInserts
                     b. lldpV2StatsRemTablesDeletes
                     c. lldpV2StatsRemTablesDrops
                     d. lldpV2StatsRemTablesAgeouts
                     e. lldpV2StatsRxPortAgeoutsTotal for all ports.

               2. LLDP agent updates remote systems MIB objects, and
                  sends out notifications to a list of notification
                  destinations.

               3. NMS receives the notifications and compares the new
                  values of objects listed in step 1.

                  Periodically, NMS should poll the object
                  lldpV2StatsRemTablesLastChangeTime to find out if anything
                  has changed since the last poll. If something has
                  changed, NMS polls the objects listed in step 1 to
                  figure out what kind of changes occurred in the tables.

                  If value of lldpV2StatsRemTablesInserts has changed,
                  then NMS walks all tables by employing TimeFilter
                  with the last-polled time value. This request
                  returns new objects or objects whose values have been
                  updated since the last poll.

                  If value of lldpV2StatsRemTablesAgeouts has changed,
                  then NMS walks the lldpStatsRxPortAgeoutsTotal and
                  compares the new values with previously recorded ones.
                  For ports whose lldpStatsRxPortAgeoutsTotal value is
                  greater than the recorded value, NMS can
                  retrieve objects associated with those ports from
                  table(s) without employing a TimeFilter (which is
                  performed by specifying 0 for the TimeFilter).

                  lldpV2StatsRemTablesDeletes and lldpV2StatsRemTablesDrops
                  objects are provided for informational purposes."
    ::= { lldpV2RemoteSystemsData  1 }

lldpV2RemEntry OBJECT-TYPE
    SYNTAX      LldpV2RemEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "Information about a particular physical network connection.
            Entries may be created and deleted in this table by the agent,
            if a physical topology discovery process is active.

            Rows in this table can only be created for MAC addresses
            that can validly be used in association with the type of
            interface concerned, as defined by Table 7-2.

            The contents of this table is persistent across
```

```
            re-initializations or re-boots."
    INDEX   {
           lldpV2RemTimeMark,
           lldpV2RemLocalIfIndex,
           lldpV2RemLocalDestMACAddress,
           lldpV2RemIndex
    }
    ::= { lldpV2RemTable 1 }

LldpV2RemEntry ::= SEQUENCE {
      lldpV2RemTimeMark              TimeFilter,
      lldpV2RemLocalIfIndex          InterfaceIndex,
      lldpV2RemLocalDestMACAddress   LldpV2DestAddressTableIndex,
      lldpV2RemIndex                 Unsigned32,
      lldpV2RemChassisIdSubtype      LldpV2ChassisIdSubtype,
      lldpV2RemChassisId             LldpV2ChassisId,
      lldpV2RemPortIdSubtype         LldpV2PortIdSubtype,
      lldpV2RemPortId                LldpV2PortId,
      lldpV2RemPortDesc              SnmpAdminString,
      lldpV2RemSysName               SnmpAdminString,
      lldpV2RemSysDesc               SnmpAdminString,
      lldpV2RemSysCapSupported       LldpV2SystemCapabilitiesMap,
      lldpV2RemSysCapEnabled         LldpV2SystemCapabilitiesMap,
      lldpV2RemRemoteChanges        TruthValue,
      lldpV2RemTooManyNeighbors      TruthValue
}

lldpV2RemTimeMark  OBJECT-TYPE
    SYNTAX      TimeFilter
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "A TimeFilter for this entry. See the TimeFilter textual
            convention in IETF RFC 4502 and
            http://www.ietf.org/IESG/Implementations/RFC2021-Implementation.txt
            to see how TimeFilter works."
    REFERENCE
            "IETF RFC 4502 section 6"
    ::= { lldpV2RemEntry 1 }


lldpV2RemLocalIfIndex   OBJECT-TYPE
    SYNTAX      InterfaceIndex
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The interface index value used to identify the port
            associated with this entry. Its value is an index
            into the interfaces MIB

            The value of this object is used as an index to the
            lldpV2RemTable."
    ::= { lldpV2RemEntry 2 }

lldpV2RemLocalDestMACAddress   OBJECT-TYPE
    SYNTAX      LldpV2DestAddressTableIndex
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
```

```
            "The index value used to identify the destination
            MAC address associated with this entry. Its value identifies
            the row in the lldpV2DestAddressTable where the MAC address
            can be found.

            The value of this object is used as an index to the
            lldpV2RemTable."
    ::= { lldpV2RemEntry 3 }


lldpV2RemIndex   OBJECT-TYPE
    SYNTAX      Unsigned32(1..2147483647)
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "This object represents an arbitrary local integer value used
            by this agent to identify a particular connection instance,
            unique only for the indicated remote system.

            An agent is encouraged to assign monotonically increasing
            index values to new entries, starting with one, after each
            reboot. It is considered unlikely that the lldpRemIndex
            can wrap between reboots."
    ::= { lldpV2RemEntry 4 }

lldpV2RemChassisIdSubtype   OBJECT-TYPE
    SYNTAX      LldpV2ChassisIdSubtype
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The type of encoding used to identify the chassis associated
            with the remote system."
    REFERENCE
            "8.5.2.2"
    ::= { lldpV2RemEntry 5 }

lldpV2RemChassisId   OBJECT-TYPE
    SYNTAX      LldpV2ChassisId
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The string value used to identify the chassis component
            associated with the remote system."
    REFERENCE
            "8.5.2.3"
    ::= { lldpV2RemEntry 6 }

lldpV2RemPortIdSubtype   OBJECT-TYPE
    SYNTAX      LldpV2PortIdSubtype
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The type of port identifier encoding used in the associated
            'lldpRemPortId' object."
    REFERENCE
            "8.5.3.2"
    ::= { lldpV2RemEntry 7 }

lldpV2RemPortId   OBJECT-TYPE
```

```
    SYNTAX      LldpV2PortId
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The string value used to identify the port component
            associated with the remote system."
    REFERENCE
            "8.5.3.3"
    ::= { lldpV2RemEntry 8 }

lldpV2RemPortDesc   OBJECT-TYPE
    SYNTAX      SnmpAdminString (SIZE(0..255))
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The string value used to identify the description of
            the given port associated with the remote system."
    REFERENCE
            "8.5.5.2"
    ::= { lldpV2RemEntry 9 }

lldpV2RemSysName   OBJECT-TYPE
    SYNTAX      SnmpAdminString (SIZE(0..255))
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The string value used to identify the system name of the
            remote system."
    REFERENCE
            "8.5.6.2"
    ::= { lldpV2RemEntry 10 }

lldpV2RemSysDesc   OBJECT-TYPE
    SYNTAX      SnmpAdminString (SIZE(0..255))
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The string value used to identify the system description
            of the remote system."
    REFERENCE
            "8.5.7.2"
    ::= { lldpV2RemEntry 11 }

lldpV2RemSysCapSupported  OBJECT-TYPE
    SYNTAX      LldpV2SystemCapabilitiesMap
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The bitmap value used to identify which system capabilities
            are supported on the remote system."
    REFERENCE
            "8.5.8.1"
    ::= { lldpV2RemEntry 12 }

lldpV2RemSysCapEnabled   OBJECT-TYPE
    SYNTAX      LldpV2SystemCapabilitiesMap
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
```

```
            "The bitmap value used to identify which system capabilities
            are enabled on the remote system."
    REFERENCE
            "8.5.8.2"
    ::= { lldpV2RemEntry 13 }

lldpV2RemRemoteChanges   OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "Indicates that there are changes in the remote systems
            MIB, as determined by the variable remoteChanges."
    REFERENCE
            "9.2.5.10"
    ::= { lldpV2RemEntry 14 }

lldpV2RemTooManyNeighbors   OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "Indicates that there are too many neighbors
             as determined by the variable tooManyNeighbors."
    REFERENCE
            "9.2.5.16"
    ::= { lldpV2RemEntry 15 }

--
-- lldpV2RemManAddrTable : Management addresses of the remote system
-- Version 2 includes additional index values for ifIndex and
-- destination MAC address.
--

lldpV2RemManAddrTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF LldpV2RemManAddrEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "This table contains one or more rows per management address
            information on the remote system learned on a particular port
            contained in the local chassis known to this agent."
    ::= { lldpV2RemoteSystemsData 2 }

lldpV2RemManAddrEntry OBJECT-TYPE
    SYNTAX      LldpV2RemManAddrEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "Management address information about a particular chassis
            component. There may be multiple management addresses
            configured on the remote system identified by a particular
            lldpRemIndex whose information is received on
            an interface of the local system and a given destination
            MAC address. Each management
            address should have distinct 'management address
            type' (lldpRemManAddrSubtype) and 'management address'
            (lldpRemManAddr).
```

```
            Entries may be created and deleted in this table by the
            agent.
            Since a variable length octetstring is used as an index
            in a table, theaddress length is encoded as part of the OID
            (as per IETF RFC 2578)."
    INDEX   { lldpV2RemTimeMark,
              lldpV2RemLocalIfIndex,
              lldpV2RemLocalDestMACAddress,
              lldpV2RemIndex,
              lldpV2RemManAddrSubtype,
              lldpV2RemManAddr
 }
    ::= { lldpV2RemManAddrTable 1 }


LldpV2RemManAddrEntry ::= SEQUENCE {
      lldpV2RemManAddrSubtype      AddressFamilyNumbers,
      lldpV2RemManAddr             LldpV2ManAddress,
      lldpV2RemManAddrIfSubtype    LldpV2ManAddrIfSubtype,
      lldpV2RemManAddrIfId         Unsigned32,
      lldpV2RemManAddrOID          OBJECT IDENTIFIER
}

lldpV2RemManAddrSubtype  OBJECT-TYPE
    SYNTAX      AddressFamilyNumbers
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The type of management address identifier encoding used in
            the associated 'lldpRemManagmentAddr' object.

            It should be noted that only a subset of the possible
            address encodings enumerated in AddressFamilyNumbers
            are appropriate for use as a LLDP management
            address, either because some are just not apliccable or
            because the maximum size of a LldpV2ManAddress octet string
            would prevent the use of some address identifier encodings."
    REFERENCE
            "8.5.9.3"
    ::= { lldpV2RemManAddrEntry 1 }

lldpV2RemManAddr  OBJECT-TYPE
    SYNTAX      LldpV2ManAddress
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The string value used to identify the management address
            component associated with the remote system. The purpose
            of this address is to contact the management entity."
    REFERENCE
            "8.5.9.4"
    ::= { lldpV2RemManAddrEntry 2 }

lldpV2RemManAddrIfSubtype  OBJECT-TYPE
    SYNTAX      LldpV2ManAddrIfSubtype
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The enumeration value that identifies the interface numbering
```

```
            method used for defining the interface number, associated
            with the remote system."
    REFERENCE
            "8.5.9.5"
    ::= { lldpV2RemManAddrEntry 3 }

lldpV2RemManAddrIfId  OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The integer value used to identify the interface number
            regarding the management address component associated with
            the remote system. The value depends upon the value of the
            lldpV2RemManAddrIfSubtype for the table row."
    REFERENCE
            "8.5.9.6"
    ::= { lldpV2RemManAddrEntry 4 }

lldpV2RemManAddrOID  OBJECT-TYPE
    SYNTAX      OBJECT IDENTIFIER
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The OID value used to identify the type of hardware component
            or protocol entity associated with the management address
            advertised by the remote system agent."
    REFERENCE
            "8.5.9.8"
    ::= { lldpV2RemManAddrEntry 5 }


--
-- lldpV2RemUnknownTLVTable : Unrecognized TLV information
-- This version has additional indexes for
-- ifIndex and destination MAC address
--

lldpV2RemUnknownTLVTable  OBJECT-TYPE
    SYNTAX      SEQUENCE OF LldpV2RemUnknownTLVEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "This table contains information about an incoming TLV that
            is not recognized by the receiving LLDP agent. The TLV may
            be from a later version of the basic management set.

            This table should only contain TLVs that are found in
            a single LLDP frame. Entries in this table, associated
            with an MAC service access point (MSAP, the access point
            for MAC services provided to the LCC sublayer, defined
            in IEEE Standards Dictionary Online, which is also
            identified with a particular lldpRemLocalPortNum,
            lldpRemIndex pair) are overwritten with most recently
            received unrecognized TLV from the same MSAP, or they
            naturally age out when the rxInfoTTL timer
            (associated with the MSAP) expires."
    REFERENCE
            "9.2.8.7.1"
```

```
    ::= { lldpV2RemoteSystemsData 3 }

lldpV2RemUnknownTLVEntry OBJECT-TYPE
    SYNTAX      LldpV2RemUnknownTLVEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "Information about an unrecognized TLV received from a
            physical network connection. Entries may be created and
            deleted in this table by the agent, if a physical topology
            discovery process is active."
    INDEX   {
           lldpV2RemTimeMark,
           lldpV2RemLocalIfIndex,
           lldpV2RemLocalDestMACAddress,
           lldpV2RemIndex,
           lldpV2RemUnknownTLVType
    }
    ::= { lldpV2RemUnknownTLVTable 1 }

LldpV2RemUnknownTLVEntry ::= SEQUENCE {
      lldpV2RemUnknownTLVType      Unsigned32,
      lldpV2RemUnknownTLVInfo      OCTET STRING
}

lldpV2RemUnknownTLVType OBJECT-TYPE
    SYNTAX      Unsigned32(9..126)
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "This object represents the value extracted from the type
            field of the TLV."
    REFERENCE
            "9.2.8.7.1"
    ::= { lldpV2RemUnknownTLVEntry 1 }

lldpV2RemUnknownTLVInfo OBJECT-TYPE
    SYNTAX      OCTET STRING (SIZE(0..511))
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "This object represents the value extracted from the value
            field of the TLV."
    REFERENCE
            "9.2.8.7.1"
    ::= { lldpV2RemUnknownTLVEntry 2 }

-- ------------------------------------------------------------------------
-- Remote Systems Extension Table - Organizationally Defined Information
-- ------------------------------------------------------------------------
--
-- lldpV2RemOrgDefInfoTable - indexed by ifIndex and destination
-- MAC address.
--

lldpV2RemOrgDefInfoTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF LldpV2RemOrgDefInfoEntry
    MAX-ACCESS  not-accessible
    STATUS      current
```

```
    DESCRIPTION
            "This table contains one or more rows per physical network
            connection which advertises the organizationally defined
            information.

            Note that this table contains one or more rows of
            organizationally defined information that is not recognized
            by the local agent.

            If the local system is capable of recognizing any
            organizationally defined information, appropriate extension
            MIBs from the organization should be used for information
            retrieval."
    ::= { lldpV2RemoteSystemsData 4 }

lldpV2RemOrgDefInfoEntry OBJECT-TYPE
    SYNTAX      LldpV2RemOrgDefInfoEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "Information about the unrecognized organizationally
            defined information advertised by the remote system.
            The lldpRemTimeMark, lldpRemLocalPortNum, lldpRemIndex,
            lldpRemOrgDefInfoOUI, lldpRemOrgDefInfoSubtype, and
            lldpRemOrgDefInfoIndex are indexes to this table. If there is
            an lldpRemOrgDefInfoEntry associated with a particular remote
            system identified by the lldpRemLocalPortNum and lldpRemIndex,
            then there is an lldpRemEntry associated with the same
            instance (i.e., using same indexes.) When the lldpRemEntry
            for the same index is removed from the lldpRemTable, the
            associated lldpRemOrgDefInfoEntry is removed from
            the lldpRemOrgDefInfoTable.

            Entries may be created and deleted in this table by the
            agent."
    INDEX   { lldpV2RemTimeMark,
              lldpV2RemLocalIfIndex,
              lldpV2RemLocalDestMACAddress,
              lldpV2RemIndex,
              lldpV2RemOrgDefInfoOUI,
              lldpV2RemOrgDefInfoSubtype,
              lldpV2RemOrgDefInfoIndex }
    ::= { lldpV2RemOrgDefInfoTable 1 }

LldpV2RemOrgDefInfoEntry ::= SEQUENCE {
      lldpV2RemOrgDefInfoOUI         OCTET STRING,
      lldpV2RemOrgDefInfoSubtype     Unsigned32,
      lldpV2RemOrgDefInfoIndex       Unsigned32,
      lldpV2RemOrgDefInfo            OCTET STRING
}

lldpV2RemOrgDefInfoOUI  OBJECT-TYPE
    SYNTAX      OCTET STRING (SIZE(3))
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The Organizationally Unique Identifier (OUI), as defined
            in IEEE Std 802, is a 24 bit (three octets) globally
            unique assigned number referenced by various standards,
```

```
            of the information received from the remote system."
    REFERENCE
            "8.7.1.3"
    ::= { lldpV2RemOrgDefInfoEntry 1 }

lldpV2RemOrgDefInfoSubtype  OBJECT-TYPE
    SYNTAX      Unsigned32(1..255)
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "The integer value used to identify the subtype of the
            organizationally defined information received from the
            remote system.

            The subtype value is required to identify different instances
            of organizationally defined information that could not be
            retrieved without a unique identifier that indicates the
            particular type of information contained in the information
            string."
    REFERENCE
            "8.7.1.4"
    ::= { lldpV2RemOrgDefInfoEntry 2 }

lldpV2RemOrgDefInfoIndex  OBJECT-TYPE
    SYNTAX      Unsigned32(1..2147483647)
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
            "This object represents an arbitrary local integer value
            used by this agent to identify a particular unrecognized
            organizationally defined information instance, unique only
            for the lldpRemOrgDefInfoOUI and lldpRemOrgDefInfoSubtype
            from the same remote system.

            An agent is encouraged to assign monotonically increasing
            index values to new entries, starting with one, after each
            reboot. It is considered unlikely that the
            lldpRemOrgDefInfoIndex can wrap between reboots."
    ::= { lldpV2RemOrgDefInfoEntry 3 }

lldpV2RemOrgDefInfo  OBJECT-TYPE
    SYNTAX      OCTET STRING(SIZE(0..507))
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
            "The string value used to identify the organizationally
            defined information of the remote system. The encoding for
            this object should be as defined for SnmpAdminString TC."
    REFERENCE
            "8.7.1.5"
    ::= { lldpV2RemOrgDefInfoEntry 4 }


--
-- ********************************************************
--
--        L L D P   M I B   N O T I F I C A T I O N S
--
-- ********************************************************
```

```
--

lldpV2NotificationPrefix OBJECT IDENTIFIER ::= { lldpV2Notifications 0 }

lldpV2RemTablesChange NOTIFICATION-TYPE
    OBJECTS {
        lldpV2StatsRemTablesInserts,
        lldpV2StatsRemTablesDeletes,
        lldpV2StatsRemTablesDrops,
        lldpV2StatsRemTablesAgeouts
    }
    STATUS        current
    DESCRIPTION
            "A lldpV2RemTablesChange notification is sent when the value
            of lldpV2StatsRemTablesLastChangeTime changes. It can be
            utilized by an NMS to trigger LLDP remote systems table
            maintenance polls.

            Note that transmission of lldpV2RemTablesChange
            notifications are throttled by the agent, as specified by the
            'lldpV2NotificationInterval' object."
   ::= { lldpV2NotificationPrefix 1 }


--
-- ********************************************************
--
--           L L D P   M I B   C O N F O R M A N C E
--
-- ********************************************************
--

lldpV2Compliances OBJECT IDENTIFIER ::= { lldpV2Conformance 1 }
lldpV2Groups      OBJECT IDENTIFIER ::= { lldpV2Conformance 2 }

-- compliance statements

lldpV2TxRxCompliance MODULE-COMPLIANCE
                        --V2 to add ifGeneralInformationGroup
                        --and support re-indexed tables
    STATUS  current
    DESCRIPTION
            "A compliance statement for all SNMP entities that
            implement the LLDP MIB as either a transmitter or
            a receiver of LLDPDUs.

            This version defines compliance requirements for
            V2 of the LLDP MIB module."
    MODULE  -- this module
        MANDATORY-GROUPS { lldpV2ConfigGroup,
                           ifGeneralInformationGroup
        }

    ::= { lldpV2Compliances 1 }

lldpV2TxCompliance MODULE-COMPLIANCE
                    --V2 requirements for transmitters of LLDPDUs
                       --and support re-indexed tables
    STATUS  current
```

```
    DESCRIPTION
            "A compliance statement for SNMP entities that implement
            the LLDP MIB and have the capability of transmitting
            LLDP frames.

            This version defines compliance requirements for
            V2 of the LLDP MIB module."
    MODULE  -- this module
        MANDATORY-GROUPS { lldpV2ConfigTxGroup,
                           lldpV2StatsTxGroup,
                           lldpV2LocSysGroup
        }

    ::= { lldpV2Compliances 2 }

lldpV2RxCompliance MODULE-COMPLIANCE
                      --V2 requirements for receivers of LLDPDUs
                      --and support re-indexed tables
    STATUS  current
    DESCRIPTION
            "The compliance statement for SNMP entities that implement
            the LLDP MIB and have the capability of receiving
            LLDP frames.

            This version defines compliance requirements for
            V2 of the LLDP MIB module."
    MODULE  -- this module
        MANDATORY-GROUPS { lldpV2ConfigRxGroup,
                           lldpV2StatsRxGroup,
                           lldpV2RemSysGroup,
                           lldpV2NotificationsGroup
        }

     ::= { lldpV2Compliances 3 }

-- MIB groupings


lldpV2ConfigGroup    OBJECT-GROUP
    OBJECTS {
        lldpV2PortConfigAdminStatusV2
    }
    STATUS  current
    DESCRIPTION
            "The collection of objects that are used to configure the
            LLDP implementation behavior."
    ::= { lldpV2Groups 1 }


lldpV2ConfigRxGroup    OBJECT-GROUP
    OBJECTS {
        lldpV2NotificationInterval,
        lldpV2PortConfigNotificationEnableV2
    }
    STATUS  current
    DESCRIPTION
            "The collection of objects that are used to configure the
            LLDP reception implementation behavior."
    ::= { lldpV2Groups 2 }
```

```
lldpV2ConfigTxGroup    OBJECT-GROUP
    OBJECTS {
        lldpV2MessageTxInterval,
        lldpV2MessageTxHoldMultiplier,
        lldpV2ReinitDelay,
        lldpV2PortConfigTLVsTxEnableV2,
        lldpV2ManAddrConfigTxEnable,
        lldpV2ManAddrConfigRowStatus,
        lldpV2TxCreditMax,
        lldpV2MessageFastTx,
        lldpV2TxFastInit,
        lldpV2DestMacAddress,
        lldpV2PortMessageTxInterval,
        lldpV2PortMessageTxHoldMultiplier,
        lldpV2PortReinitDelay,
        lldpV2PortNotificationInterval,
        lldpV2PortTxCreditMax,
        lldpV2PortMessageFastTx,
        lldpV2PortTxFastInit
    }
    STATUS  current
    DESCRIPTION
            "The collection of objects that are used to configure the
            LLDP transmission implementation behavior."
    ::= { lldpV2Groups 3 }


lldpV2StatsRxGroup    OBJECT-GROUP
    OBJECTS {
        lldpV2StatsRemTablesLastChangeTime,
        lldpV2StatsRemTablesInserts,
        lldpV2StatsRemTablesDeletes,
        lldpV2StatsRemTablesDrops,
        lldpV2StatsRemTablesAgeouts,
        lldpV2StatsRxPortFramesDiscardedTotal,
        lldpV2StatsRxPortFramesErrors,
        lldpV2StatsRxPortFramesTotal,
        lldpV2StatsRxPortTLVsDiscardedTotal,
        lldpV2StatsRxPortTLVsUnrecognizedTotal,
        lldpV2StatsRxPortAgeoutsTotal
    }
    STATUS  current
    DESCRIPTION
            "The collection of objects that are used to represent LLDP
            reception statistics."
    ::= { lldpV2Groups 4 }


lldpV2StatsTxGroup    OBJECT-GROUP
    OBJECTS {
        lldpV2StatsTxPortFramesTotal,
        lldpV2StatsTxLLDPDULengthErrors
    }
    STATUS  current
    DESCRIPTION
            "The collection of objects that are used to represent LLDP
            transmission statistics."
```

```
    ::= { lldpV2Groups 5 }


lldpV2LocSysGroup  OBJECT-GROUP
    OBJECTS {
        lldpV2LocChassisIdSubtype,
        lldpV2LocChassisId,
        lldpV2LocPortIdSubtype,
        lldpV2LocPortId,
        lldpV2LocPortDesc,
        lldpV2LocSysDesc,
        lldpV2LocSysName,
        lldpV2LocSysCapSupported,
        lldpV2LocSysCapEnabled,
        lldpV2LocManAddrLen,
        lldpV2LocManAddrIfSubtype,
        lldpV2LocManAddrIfId,
        lldpV2LocManAddrOID
    }
    STATUS  current
    DESCRIPTION
            "The collection of objects that are used to represent LLDP
            Local System Information."
    ::= { lldpV2Groups 6 }

lldpV2RemSysGroup  OBJECT-GROUP
    OBJECTS {
        lldpV2RemChassisIdSubtype,
        lldpV2RemChassisId,
        lldpV2RemPortIdSubtype,
        lldpV2RemPortId,
        lldpV2RemPortDesc,
        lldpV2RemSysName,
        lldpV2RemSysDesc,
        lldpV2RemSysCapSupported,
        lldpV2RemSysCapEnabled,
        lldpV2RemRemoteChanges,
        lldpV2RemTooManyNeighbors,
        lldpV2RemManAddrIfSubtype,
        lldpV2RemManAddrIfId,
        lldpV2RemManAddrOID,
        lldpV2RemUnknownTLVInfo,
        lldpV2RemOrgDefInfo
    }
    STATUS  current
    DESCRIPTION
            "The collection of objects that are used to represent
            LLDP Remote Systems Information. The objects represent the
            information associated with the basic TLV set. Please note
            that even if the agent does not implement some of the optional
            TLVs, it shall recognize all the optional TLV information
            that the remote system may advertise."
    ::= { lldpV2Groups 7 }

lldpV2NotificationsGroup  NOTIFICATION-GROUP
    NOTIFICATIONS {
        lldpV2RemTablesChange
    }
    STATUS  current
```

```
    DESCRIPTION
            "The collection of notifications used to indicate LLDP MIB
            data consistency and general status information."
    ::= { lldpV2Groups 8 }

END
```

12. LLDP YANG definitions

<<Editor's Note: All of this clause was added by IEEE Std 802.1ABcu-2021.>>

This clause specifies YANG modules that provide control and status monitoring of systems and system components that implement functionality specified in this standard.

This clause:

a) Introduces the YANG framework that governs the naming and hierarchy of configuration and operational data structures in the data models, and the modeling of network interfaces (12.1).
b) Describes the information data model and its relationship to the operational processes and managed objects specified in the other clauses of this standard, and provides a Unified Modeling Language (UML) representation of each data model (12.2).
c) Describes the structure of the data models, each of which comprises or makes use of one or more YANG modules (12.3).
d) Includes a relationship description of other modules imported in YANG modules (12.4).
e) Reviews security considerations applicable to each of the modules, with specific reference to data nodes in the YANG modules that compose the model (12.5).
f) Includes each of the YANG modules and its data schema (12.6).

12.1 Internet Standard Management Framework

This YANG module uses the YANG 1.1 Data Modeling Language as specified in IETF RFC 7950.

The YANG framework applies hierarchy in the following areas:

a) The uniform resource name (URN), as specified in IEEE Std 802.
b) The YANG objects form a hierarchy of configuration and operational data structures that define the YANG model.

12.2 Information model for LLDP management

The YANG objects are based on the TLVs detailed in 8.5 and 8.7, and the agent operation variables detailed in 9.1 and 9.2. A UML-like representation of the management model is provided in the following subclauses.

The purpose of a UML-like [17] diagram is to express the model design on a single piece of paper. The structure of the UML-like representation shows the name of the object followed by a list of properties for the object. The properties indicate its type and accessibility. It should be noted that the UML-like representation is meant to express simplified semantics for the properties. It is not meant to provide the specific datatype as used to encode the object in either MIB or YANG. In the UML-like representation, a box with a white background represents information that comes from sources outside of the IEEE. A box with a gray background represents objects that are defined by this IEEE Standard.

The YANG hierarchical structure that incorporates the LLDP YANG modules supported by this standard is represented by Figure 12-1. In the figures in this clause, items that are shaded gray are described in this document, items with no background shading are defined elsewhere. The YANG data model is realized in two YANG modules. One module *ieee802-dot1ab-types* provides data types that are needed by the LLDP configuration and monitoring objects. The *ieee802-dot1ab-lldp* module provides the LLDP configuration

[17] A description of the UML-like diagrams used in this clause is provided at https://1.ieee802.org/uml-like-diagrams

and monitoring objects. The ability to augment the port container to support extension TLVs is also shown. The LLDP capabilities are not only applicable to IEEE Std 802.1Q bridges, but also (for example) end stations, and routers.

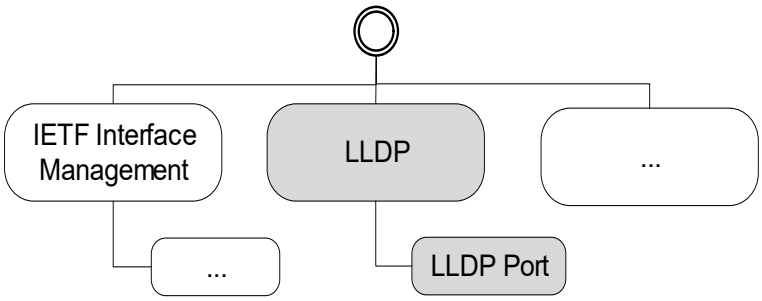


Figure 12-1—YANG root hierarchy with LLDP YANG modules

12.2.1 LLDP UML

The LLDP configuration and monitoring objects in Figure 12-2 show the objects that are applicable on a per-agent or chassis associated with the LLDP agent.

lldp		
uint32	message-fast-tx;	// (9.2.5.5) r-w
uint32	message-tx-hold-multiplier;	// (9.2.5.6) r-w
uint32	message-tx-interval;	// (9.2.5.7) r-w
uint32	reinit-delay;	// (9.2.5.10) r-w
uint32	tx-credit-max;	// (9.2.5.17) r-w
uint32	tx-fast-init;	// (9.2.5.19) r-w
uint32	notification-interval;	// (9.2.5.7) r-w
remote-statistics		
timestamp	last-change-time;	// r
zero-based-counter32	remote-inserts;	// r
zero-based-counter32	remote-deletes;	// r
zero-based-counter32	remote-drops;	// r
zero-based-counter32	remote-ageouts;	// r
local-system-data		
enum	chassis-id-subtype;	// (8.5.2.2) r
string	chassis-id;	// (8.5.2.3) r
string	system-name;	// (8.5.6.2) r
string	system-description;	// (8.5.7.2) r
bits	system-capabilities-supported	// (8.5.8.1) r
bits	system-capabilities-enabled	// (8.5.8.2) r

Figure 12-2—LLDP configuration and monitoring objects

12.2.2 LLDP Port UML

The LLDP port configuration and monitoring objects in Figure 12-3 show the objects that are applicable to an LLDP port.

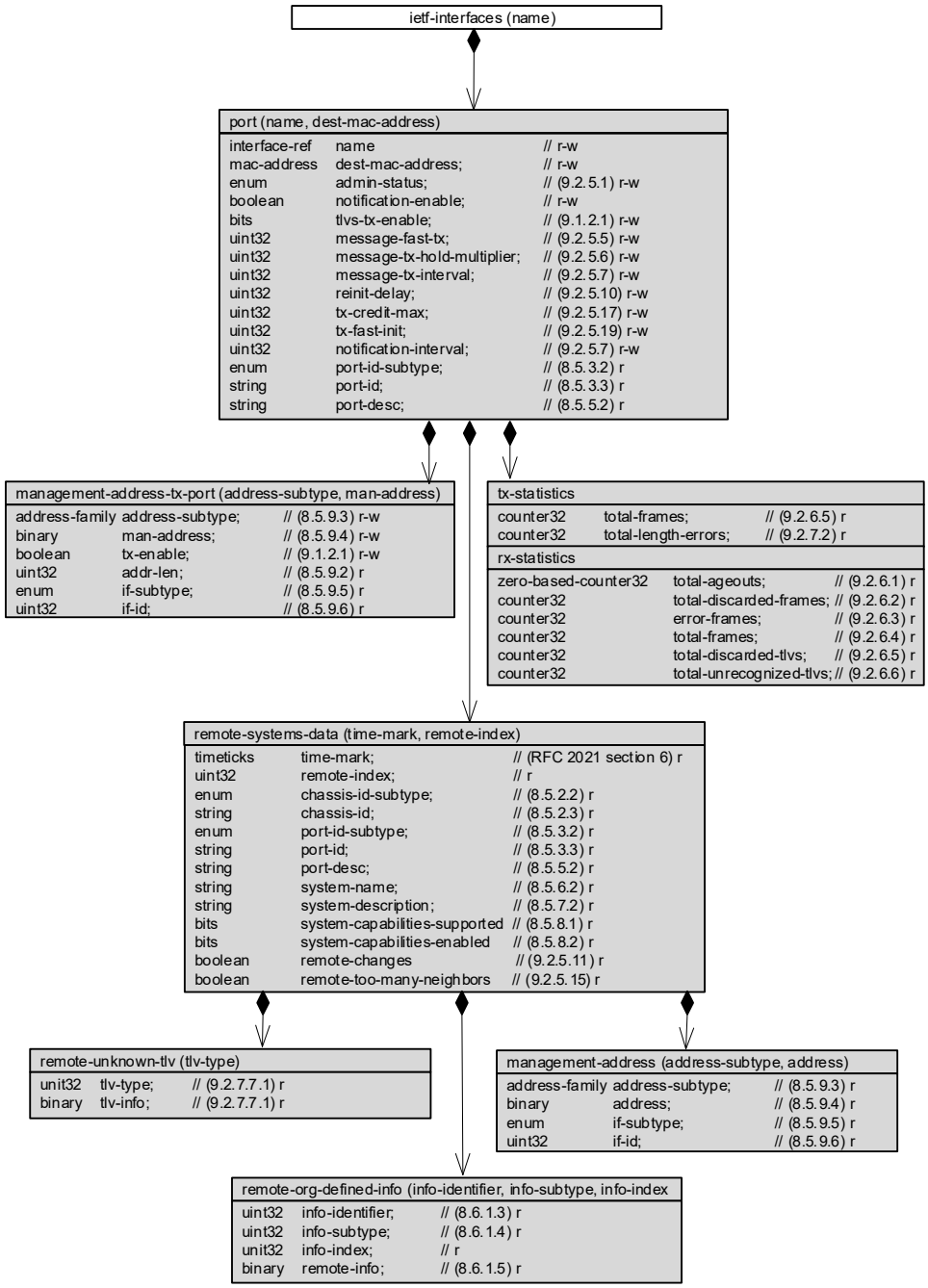


Figure 12-3—LLDP port configuration and monitoring objects

12.3 Structure of the LLDP YANG model

The IEEE YANG model specified in this standard is divided into two YANG modules. A summary of the modules contained in this clause is represented in Table 12-1.

Table 12-1—Structure of the YANG modules

Module	Subclause	Notes
ieee802-dot1ab-types	Clause 12.6.2.1	Type definitions used for LLDP YANG
ieee802-dot1ab-lldp	Clause 12.6.2.2	LLDP Generic Management

In the YANG module definitions below, if any discrepancy between the DESCRIPTION text and the corresponding definition in any other part of this standard occurs, the definitions outside this subclause take precedence.

12.3.1 Structure of the *ieee802-dot1ab-lldp* module

The *ieee802-dot1ab-lldp* YANG module is divided into several YANG branches (e.g., subtrees). A summary of the YANG subtrees associated with this module is presented in Table 12-2.

Table 12-2—Structure of the *ieee802-dot1ab-lldp* module

Branches	References	Notes
lldp		Link Layer Discovery Protocol (LLDP) configuration and operational information
message-fast-tx	9.2.5.4	
message-tx-hold-multiplier	9.2.5.5	
message-tx-interval	9.2.5.7	
reinit-delay	9.2.5.9	
tx-credit-max	9.2.5.18	
tx-fast-init	9.2.5.20	
notification-interval	9.2.5.6	
remote-statistics	10.5.2	LLDP remote operational statistics data
local-system-data	10.5.1	LLDP local system operational data
port	9.2.4	LLDP configuration information for a particular port
management-address-tx-port	8.5.9	Set of ports on which the local system management address instance will be transmitted
tx-statistics	9.2.7	LLDP frame transmission statistics for a particular port
rx-statistics	9.2.7	LLDP frame reception statistics for a particular port
remote-system-data	9.2.6	Information about a particular physical network connection

12.4 Relationship to other YANG modules

This clause describes how the *ieee802-dot1ab-lldp* YANG module is related to the YANG modules that are imported.

12.4.1 IEEE LLDP Types Module

The *ieee802-dot1ab-types* module provides reusable types that are used by the *ieee802-dot1ab-lldp* module.

12.4.2 IETF Routing Module

The *ietf-routing* YANG module (IETF RFC 8349) is a module defined by the IETF for management of a routing subsystem. This document only uses the address-family identity, which is used as the base identity for address families.

12.4.3 IETF YANG Types Module

The *ietf-yang-types* YANG module (IETF RFC 6991) contains a set of derived YANG types. This document leverages timestamp, timeticks, zero-based-counter32, and counter32.

12.4.4 IETF Interfaces YANG Module

The *ietf-interfaces* YANG module (IETF RFC 8343) contains a set of YANG definitions for managing network interfaces. This document models a port as an interface.

12.4.5 IEEE 802 Types Module

The *ieee802-types* module provides reusable types that are used in IEEE 802 standards.

The type for mac-addresses defined in *ieee802-types* has a pattern that allows upper and lower case letters. To avoid issues with string comparison, it is suggested to only use upper case for the letters in the hexadecimal numbers. Implementers using code comparing MAC addresses should note that there is still an issue with a difference between the IETF mac-address definition and the IEEE mac-address definition.

12.5 Security considerations

The YANG modules defined in this clause are designed to be accessed via a network configuration protocol (e.g., NETCONF protocol). In the case of NETCONF, the lowest NETCONF layer is the secure transport layer and the mandatory to implement secure transport is SSH. The NETCONF access control model provides the means to restrict access for particular NETCONF users to a pre-configured subset of all available NETCONF protocol operations and content.

It is the responsibility of a system's implementor and administrator to ensure that the protocol entities in the system that support NETCONF, and any other remote configuration protocols that make use of these YANG modules, are properly configured to allow access only to those users who have legitimate rights to read or write data nodes. This standard does not specify how the credentials of those users are to be stored or validated.

12.5.1 Security considerations of the *ieee802-dot1ab-lldp* YANG module

There are several management objects defined in the *ieee802-dot1ab-lldp* YANG module that are configurable (i.e., read-write) and/or operational (i.e., read-only). Such objects may be considered sensitive or vulnerable in some network environments. A network configuration protocol, such as NETCONF (IETF RFC 6241), can support protocol operations that can edit or delete YANG module configuration data (e.g., edit-config, delete-config, copy-config). If this is done in a non-secure environment without proper protection, then negative effects on the network operation are possible.

The following objects in the *ieee802-dot1ab-lldp* YANG module can be manipulated to interfere with the operation of LLDP. This could, for example, be used to return incorrect information about neighbor nodes, or cause a denial-of-service attack.

lldp/message-fast-tx

lldp/message-tx-hold-multiplier

lldp/message-tx-interval

lldp/reinit-delay

lldp/tx-credit-max

lldp/tx-fast-init

lldp/notification-interval
lldp/port/name
lldp/port/dest-mac-address
lldp/port/admin-status
lldp/port/notification-enable
lldp/port/tlvs-tx-enable
lldp/port/message-fast-tx
lldp/port/message-tx-hold-multiplier
lldp/port/message-tx-interval
lldp/port/reinit-delay
lldp/port/tx-credit-max
lldp/port/tx-fast-init
lldp/port/notification-interval
lldp/port/management-address-tx-port/address-subtype
lldp/port/management-address-tx-port/man-address
lldp/port/management-address-tx-port/tx-enable

Some of the readable data in this YANG module may be considered sensitive or vulnerable in some network environments. It is important to control all types of access (e.g., including NETCONF get and get-config operations) to these objects and possibly to even encrypt the values of these objects when sending them over the network. For example the system name and other information about the remote systems could provide information about the configuration and topology of the network and could be considered a privacy threat.

12.6 Definition of the YANG modules [18], [19], [20]

12.6.1 YANG schema definitions

A simplified graphical representation of the data model is used in this document. The meaning of the symbols in these diagrams is as follows:

— Brackets "[" and "]" enclose list keys.
— Abbreviations before data node names: "rw" means configuration (read-write), and "ro" means state data (read-only).
— Symbols after data node names: "?" means an optional node, "!" means a presence container, and "*" denotes a list and leaf-list.
— Parentheses enclose choice and case nodes, and case nodes are also marked with a colon (":").
— Ellipsis ("…") stand for contents of subtrees that are not shown.

12.6.1.1 YANG schema definition for *ieee802-dot1ab-lldp* YANG module

```
module: ieee802-dot1ab-lldp
  +--rw lldp
     +--rw message-fast-tx?              uint32
     +--rw message-tx-hold-multiplier?   uint32
```

[18] *Copyright release for YANG modules*: Users of this standard may freely reproduce the YANG modules contained in this subclause so that they can be used for their intended purpose.

[19] The ieee802-types YANG module is common across various IEEE 802.1 standards and can be updated by any IEEE 802.1 standard that imports the ieee802-types YANG module.

[20] Consult the IEEE 802.1 website at https://1.ieee802.org/yang-modules/ to determine and obtain the current revision of any IEEE 802.1 YANG module.

```
   +--rw message-tx-interval?          uint32
   +--rw reinit-delay?                 uint32
   +--rw tx-credit-max?                uint32
   +--rw tx-fast-init?                 uint32
   +--rw notification-interval?        uint32
   +--ro remote-statistics
   |  +--ro last-change-time?   yang:timestamp
   |  +--ro remote-inserts?     yang:zero-based-counter32
   |  +--ro remote-deletes?     yang:zero-based-counter32
   |  +--ro remote-drops?       yang:zero-based-counter32
   |  +--ro remote-ageouts?     yang:zero-based-counter32
   +--ro local-system-data
   |  +--ro chassis-id-subtype?              ieee:chassis-id-subtype-type
   |  +--ro chassis-id?                      ieee:chassis-id-type
   |  +--ro system-name?                     string
   |  +--ro system-description?              string
   |  +--ro system-capabilities-supported?   lldp-types:system-capabilities-map
   |  +--ro system-capabilities-enabled?     lldp-types:system-capabilities-map
   +--rw port* [name dest-mac-address]
      +--rw name                            if:interface-ref
      +--rw dest-mac-address                ieee:mac-address
      +--rw admin-status?                   enumeration
      +--rw notification-enable?            boolean
      +--rw tlvs-tx-enable?                 bits
      +--rw message-fast-tx?                uint32
      +--rw message-tx-hold-multiplier?     uint32
      +--rw message-tx-interval?            uint32
      +--rw reinit-delay?                   uint32
      +--rw tx-credit-max?                  uint32
      +--rw tx-fast-init?                   uint32
      +--rw notification-interval?          uint32
      +--rw management-address-tx-port* [address-subtype man-address]
      |  +--rw address-subtype    identityref
      |  +--rw man-address        lldp-types:man-addr-type
      |  +--rw tx-enable?         boolean
      |  +--ro addr-len?          uint32
      |  +--ro if-subtype?        lldp-types:man-addr-if-subtype
      |  +--ro if-id?             uint32
      +--ro port-id-subtype?                ieee:port-id-subtype-type
      +--ro port-id?                        ieee:port-id-type
      +--ro port-desc?                      string
      +--ro tx-statistics
      |  +--ro total-frames?          yang:counter32
      |  +--ro total-length-errors?   yang:counter32
      +--ro rx-statistics
      |  +--ro total-ageouts?             yang:zero-based-counter32
      |  +--ro total-discarded-frames?    yang:counter32
      |  +--ro error-frames?              yang:counter32
      |  +--ro total-frames?              yang:counter32
      |  +--ro total-discarded-tlvs?      yang:counter32
      |  +--ro total-unrecognized-tlvs?   yang:counter32
      +--ro remote-systems-data* [time-mark remote-index]
         +--ro time-mark                          yang:timeticks
         +--ro remote-index                       uint32
         +--ro remote-too-many-neighbors?         boolean
         +--ro remote-changes?                    boolean
         +--ro chassis-id-subtype?                ieee:chassis-id-subtype-type
         +--ro chassis-id?                        ieee:chassis-id-type
         +--ro port-id-subtype?                   ieee:port-id-subtype-type
```

```
          +--ro port-id?                          ieee:port-id-type
          +--ro port-desc?                        string
          +--ro system-name?                      string
          +--ro system-description?               string
          +--ro system-capabilities-supported?
          |  lldp-types:system-capabilities-map
          +--ro system-capabilities-enabled?
          |  lldp-types:system-capabilities-map
          +--ro management-address* [address-subtype address]
          |  +--ro address-subtype    identityref
          |  +--ro address            lldp-types:man-addr-type
          |  +--ro if-subtype?        lldp-types:man-addr-if-subtype
          |  +--ro if-id?             uint32
          +--ro remote-unknown-tlv* [tlv-type]
          |  +--ro tlv-type    uint32
          |  +--ro tlv-info?   binary
          +--ro remote-org-defined-info*
                 [info-identifier info-subtype info-index]
             +--ro info-identifier    uint32
             +--ro info-subtype       uint32
             +--ro info-index         uint32
             +--ro remote-info?       binary

notifications:
  +---n remote-table-change
     +--ro remote-insert?    -> /lldp/remote-statistics/remote-inserts
     +--ro remote-delete?    -> /lldp/remote-statistics/remote-deletes
     +--ro remote-drops?     -> /lldp/remote-statistics/remote-drops
     +--ro remote-ageouts?   -> /lldp/remote-statistics/remote-ageouts
```

12.6.2 YANG data model definitions

12.6.2.1 Definition for the *ieee802-dot1ab-types* module

```
module ieee802-dot1ab-types {
  yang-version "1.1";
  namespace urn:ieee:std:802.1Q:yang:ieee802-dot1ab-types;
  prefix lldp-types;
  organization
    "IEEE 802.1 Working Group";
  contact
    "WG-URL: http://ieee802.org/1/
    WG-EMail: stds-802-1-l@ieee.org
    Contact: IEEE 802.1 Working Group Chair
    Postal: C/O IEEE 802.1 Working Group
    IEEE Standards Association
         445 Hoes Lane
         Piscataway, NJ 08854
         USA

    E-mail: stds-802-1-chairs@ieee.org";
  description
    "Common types used within ieee802-dot1ab-lldp modules. Copyright(C) IEEE
    (2025). All rights reserved. This version of this YANG module is part of
    IEEE Std 802.1AB-2026; see the standard itself for full legal notices.";
  revision 2025-09-29 {
    description
```

```
    "Published as part of IEEE Std 802.1AB-2026.

    The following reference statement identifies each referenced IEEE
    Standard as updated by applicable amendments.";
  reference
    "IEEE Std 802.1AB Station and Media Access Control Connectivity Discovery:
       IEEE Std 802.1AB-2026.";
}
revision 2022-03-15 {
  description
    "Published as part of IEEE Std 802.1ABcu.";
  reference
    "IEEE Std 802.1AB-2016";
}
typedef man-addr-if-subtype {
  type enumeration {
    enum unknown {
      value 1;
      description
        "Interface is not known.";
    }
    enum port-ref {
      value 2;
      description
        "Interface based on the port-ref MIB object.";
    }
    enum system-port-number {
      value 3;
      description
        "Interface based on the system port number.";
    }
  }
  description
    "Management address interface subtype.";
  reference
    "8.5.9.3 of IEEE Std 802.1AB";
}
typedef man-addr-type {
  type string {
    pattern "[0-9A-F]{2}([0-9A-F]{2}){0,30}";
  }
  description
    "Management address associated with the LLDP agent.";
  reference
    "8.5.9.4 of IEEE Std 802.1AB";
}
typedef system-capabilities-map {
  type bits {
    bit other {
      position 0;
      description
        "System has capabilities other than those listed below.";
    }
    bit repeater {
      position 1;
      description
        "System has repeater capability.";
    }
    bit bridge {
```

```
        position 2;
        description
          "System has bridge capability.";
      }
      bit wlan-access-point {
        position 3;
        description
          "System has WLAN access point capability.";
      }
      bit router {
        position 4;
        description
          "System has router capability.";
      }
      bit telephone {
        position 5;
        description
          "System has telephone capability.";
      }
      bit docsis-cable-device {
        position 6;
        description
          "System has DOCSIS Cable Device capability.";
        reference
          "IETF RFC 4639";
      }
      bit station-only {
        position 7;
        description
          "System has only station capability.";
      }
      bit cvlan-component {
        position 8;
        description
          "System has C-VLAN component functionality.";
      }
      bit svlan-component {
        position 9;
        description
          "System has S-VLAN component functionality.";
      }
      bit two-port-mac-relay {
        position 10;
        description
          "System has Two-port MAC Relay (TPMR) functionality.";
      }
    }
    description
      "This describes system capabilities.";
    reference
      "8.5.8.1 of IEEE Std 802.1AB";
  }
  typedef port-list {
    type binary {
      length "0..512";
    }
    description
      "Each octet within this value specifies a set of eight ports, with the
      first octet specifying ports 1 through 8, the second octet specifying
```

```
      ports 9 through 16, etc. Within each octet, the most significant bit
      represents the lowest numbered port, and the least significant bit
      represents the highest numbered port. Thus, each port of the system is
      represented by a single bit within the value of this object. If that
      bit has a value of '1' then that port is included in the set of ports;
      the port is not included if its bit has a value of '0'.";
    reference
      "IETF RFC 2674 section 5";
  }
}

```

12.6.2.2 Definition for *ieee802-dot1ab-lldp* YANG module

```
module ieee802-dot1ab-lldp {
  yang-version 1.1;
  namespace "urn:ieee:std:802.1AB:yang:ieee802-dot1ab-lldp";
  prefix lldp;

  import ieee802-dot1ab-types {
    prefix lldp-types;
  }
  import ietf-routing {
    prefix rt;
  }
  import ietf-yang-types {
    prefix yang;
  }
  import ietf-interfaces {
    prefix if;
  }
  import ieee802-types {
    prefix ieee;
  }

  organization
    "Institute of Electrical and Electronics Engineers";
  contact
    "WG-URL: http://ieee802.org/1/
     WG-EMail: stds-802-1-l@ieee.org
     Contact: IEEE 802.1 Working Group Chair
     Postal: C/O IEEE 802.1 Working Group
     IEEE Standards Association
         445 Hoes Lane
         Piscataway, NJ 08854
         USA

     E-mail: stds-802-1-chairs@ieee.org";
  description
    "Management Information Base module for LLDP configuration, statistics,
     local system data, and remote systems data components. Copyright(C) IEEE
     (2025). All rights reserved. This version of this YANG module is part of
     IEEE Std 802.1AB-2026; see the standard itself for full legal notices.";

  revision 2025-09-29 {
    description
      "Published as part of IEEE Std 802.1AB-2026.

```

```
      The following reference statement identifies each referenced IEEE
      Standard as updated by applicable amendments.";
    reference
      "IEEE Std 802.1AB Station and Media Access Control Connectivity Discovery:
         IEEE Std 802.1AB-2026.";
  }
  revision 2022-03-15 {
    description
      "Published as part of IEEE Std 802.1ABcu.";
    reference
      "IEEE Std 802.1AB-2016";
  }

  grouping lldp-cfg {
    description
      "LLDP basic configuration group.";
    leaf message-fast-tx {
      type uint32 {
        range "1..3600";
      }
      units "ticks";
      default "1";
      description
        "Time interval in timer ticks between transmissions during fast
         transmission periods (i.e., txFast is non-zero).";
      reference
        "9.1.1, 9.2.5.4 of IEEE Std 802.1AB";
    }
    leaf message-tx-hold-multiplier {
      type uint32 {
        range "2..10";
      }
      default "4";
      description
        "Multiplier of msg-tx-interval.";
      reference
        "9.2.5.5 of IEEE Std 802.1AB";
    }
    leaf message-tx-interval {
      type uint32 {
        range "1..3600";
      }
      units "ticks";
      default "30";
      description
        "Time interval in timer ticks between transmissions during normal
         transmission periods (i.e., txFast is zero).";
      reference
        "9.1.1, 9.2.5.6 of IEEE Std 802.1AB";
    }
    leaf reinit-delay {
      type uint32 {
        range "1..10";
      }
      units "second";
      default "2";
      description
        "Amount of delay (in units of seconds) from when admin-status becomes
         'disabled' until re-initialization is attempted.";
```

```
      reference
        "9.2.5.9 of IEEE Std 802.1AB";
    }
    leaf tx-credit-max {
      type uint32 {
        range "1..10";
      }
      default "5";
      description
        "The maximum number of consecutive LLDPDUs that can be transmitted at
         any time.";
      reference
        "9.2.5.18 of IEEE Std 802.1AB";
    }
    leaf tx-fast-init {
      type uint32 {
        range "1..8";
      }
      default "4";
      description
        "Initial value for the fast transmitting LLDPPDU.";
      reference
        "9.2.5.20 of IEEE Std 802.1AB";
    }
    leaf notification-interval {
      type uint32 {
        range "1..3600";
      }
      units "second";
      default "30";
      description
        "Controls the transmission of LLDP notifications.";
      reference
        "9.2.5.6 of IEEE Std 802.1AB";
    }
  }

  container lldp {
    description
      "Link Layer Discovery Protocol configuration and operational
       information.";
    uses lldp-cfg;
    container remote-statistics {
      config false;
      description
        "LLDP remote operational statistics data.";
      leaf last-change-time {
        type yang:timestamp;
        description
          "The value of sysUpTime object.";
        reference
          "11.5.1 of IEEE Std 802.1AB:
           lldpV2StatsRemTablesLastChangeTime";
      }
      leaf remote-inserts {
        type yang:zero-based-counter32;
        units "table entries";
        description
          "The number of times the complete set of information advertised by
```

```
        a particular MSAP has been inserted into tables contained in
        remote-systems-data and lldpExtensions objects.";
      reference
        "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesInserts";
    }
    leaf remote-deletes {
      type yang:zero-based-counter32;
      units "table entries";
      description
        "The number of times the complete set of information advertised by
         a particular MSAP has been deleted from tables contained in
         remote-systems-data and lldpExtensions objects.";
      reference
        "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesDeletes";
    }
    leaf remote-drops {
      type yang:zero-based-counter32;
      units "table entries";
      description
        "The number of times the complete set of information advertised by
         a particular MSAP could not be entered into tables contained in
         remote-systems-data and lldpExtensions objects because of
         insufficient resources.";
      reference
        "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesDrops";
    }
    leaf remote-ageouts {
      type yang:zero-based-counter32;
      description
        "The number of times the complete set of information advertised by
         a particular MSAP has been deleted from tables contained in
         remote-systems-data and lldpExtensions objects because the
         information timeliness interval has expired.";
      reference
        "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesAgeouts";
    }
  }
  container local-system-data {
    config false;
    description
      "LLDP local system operational data.";
    leaf chassis-id-subtype {
      type ieee:chassis-id-subtype-type;
      description
        "The type of encoding used to identify the chassis associated with
         the local system.";
      reference
        "8.5.2.2 of IEEE Std 802.1AB";
    }
    leaf chassis-id {
      type ieee:chassis-id-type;
      description
        "Chassis component associated with the local system.";
      reference
        "8.5.2.3 of IEEE Std 802.1AB";
    }
    leaf system-name {
      type string {
        length "0..255";
```

```
      }
      description
        "System name of the local system.";
      reference
        "8.5.6.2 of IEEE Std 802.1AB";
    }
    leaf system-description {
      type string {
        length "0..255";
      }
      description
        "System description of the local system.";
      reference
        "8.5.7.2 of IEEE Std 802.1AB";
    }
    leaf system-capabilities-supported {
      type lldp-types:system-capabilities-map;
      description
        "System capabilities are supported on the local system.";
      reference
        "8.5.8.1 of IEEE Std 802.1AB";
    }
    leaf system-capabilities-enabled {
      type lldp-types:system-capabilities-map;
      description
        "System capabilities that are enabled on the local system.";
      reference
        "8.5.8.2 of IEEE Std 802.1AB";
    }
  }
  list port {
    key "name dest-mac-address";
    description
      "LLDP configuration information for a particular port.";
    leaf name {
      type if:interface-ref;
      description
        "The port name used to identify the port component (contained in
         the local chassis with the LLDP agent) associated with this entry.";
    }
    leaf dest-mac-address {
      type ieee:mac-address;
      description
        "Destination MAC address. The ieee:mac-address type has a pattern
         that allows upper and lower case letters. To avoid issues with
         string compairson, it is suggested to only use upper case for the
         letters in the hexadecimal numbers. Implementers using code
         comparing MAC addresses should note that there is still an issue
         with a difference between the IETF mac-address definition and the
         IEEE mac-address definition.";
    }
    leaf admin-status {
      type enumeration {
        enum tx-only {
          value 1;
          description
            "Transmit LLDP frames only.";
        }
        enum rx-only {
```

```
          value 2;
          description
            "Receive LLDP frames only.";
        }
        enum tx-and-rx {
          value 3;
          description
            "Transmit and Receive LLDP frames.";
        }
        enum disabled {
          value 4;
          description
            "Do Not Transmit or Receive LLDP frames.";
        }
      }
      default "tx-and-rx";
      description
        "Administrative status of the local LLDP agent.";
      reference
        "9.2.5.1 of IEEE Std 802.1AB";
    }
    leaf notification-enable {
      type boolean;
      default "false";
      description
        "Notification status.";
    }
    leaf tlvs-tx-enable {
      type bits {
        bit port-desc {
          position 0;
          description
            "Transmit 'Port Description TLV'.";
        }
        bit sys-name {
          position 1;
          description
            "Transmit 'System Name TLV'.";
        }
        bit sys-desc {
          position 2;
          description
            "Transmit 'System Description TLV'.";
        }
        bit sys-cap {
          position 3;
          description
            "Transmit 'System Capabilities TLV'.";
        }
      }
      description
        "LLDP TLVs whose transmission is allowed on the local LLDP agent by
         the network management.";
      reference
        "9.1.1.1 of IEEE Std 802.1AB";
    }
    uses lldp-cfg;
    list management-address-tx-port {
      key "address-subtype man-address";
```

```
      description
        "Set of ports (represented as a PortList) on which the local system
         management address instance will be transmitted.";
      leaf address-subtype {
        type identityref {
          base rt:address-family;
        }
        description
          "Type of address.";
        reference
          "8.5.9.3 of IEEE Std 802.1AB";
      }
      leaf man-address {
        type lldp-types:man-addr-type;
        description
          "Management address associated with this TLV.";
        reference
          "8.5.9.4 of IEEE Std 802.1AB";
      }
      leaf tx-enable {
        type boolean;
        default "false";
        description
          "Transmission enabled status.";
        reference
          "9.1.1.1 of IEEE Std 802.1AB";
      }
      leaf addr-len {
        type uint32;
        config false;
        description
          "Length of the management address subtype and the management
           address fields in LLDPDUs transmitted by the local LLDP agent.";
        reference
          "8.5.9.2 of IEEE Std 802.1AB";
      }
      leaf if-subtype {
        type lldp-types:man-addr-if-subtype;
        config false;
        description
          "Interface numbering method used for defining the interface
           number, associated with the local system.";
        reference
          "8.5.9.5 of IEEE Std 802.1AB";
      }
      leaf if-id {
        type uint32;
        config false;
        description
          "Interface number for the management address component associated
           with the local system.";
        reference
          "8.5.9.6 of IEEE Std 802.1AB";
      }
    }
    leaf port-id-subtype {
      type ieee:port-id-subtype-type;
      config false;
      description
```

```
        "Port identifier encoding used in the associated 'port-id' object.";
      reference
        "8.5.3.2 of IEEE Std 802.1AB";
    }
    leaf port-id {
      type ieee:port-id-type;
      config false;
      description
        "Port component associated with a given port in the local system.";
      reference
        "8.5.3.3 of IEEE Std 802.1AB";
    }
    leaf port-desc {
      type string {
        length "0..255";
      }
      config false;
      description
        "802 LAN station's port description associated with the local
         system.";
      reference
        "8.5.5.2 of IEEE Std 802.1AB";
    }
    container tx-statistics {
      config false;
      description
        "LLDP frame transmission statistics for a particular port.";
      leaf total-frames {
        type yang:counter32;
        description
          "A count of all LLDP frames transmitted through the port.";
        reference
          "9.2.7.5 of IEEE Std 802.1AB";
      }
      leaf total-length-errors {
        type yang:counter32;
        description
          "A count of all LLDP length errors detected when constructing
           LLPDU frames for transmission through the port.";
        reference
          "9.2.7.8 of IEEE Std 802.1AB";
      }
    }
    container rx-statistics {
      config false;
      description
        "LLDP frame reception statistics for a particular port.";
      leaf total-ageouts {
        type yang:zero-based-counter32;
        description
          "A count of the times that a neighbor's information is deleted
           because of rxInfoTTL timer expiration.";
        reference
          "9.2.7.1 of IEEE Std 802.1AB";
      }
      leaf total-discarded-frames {
        type yang:counter32;
        description
          "A count of all LLDPDUs received and then discarded.";
```

```
        reference
          "9.2.7.2 of IEEE Std 802.1AB";
      }
      leaf error-frames {
        type yang:counter32;
        description
          "A count of all LLDPDUs received at the port with one or more
           detectable errors.";
        reference
          "9.2.7.3 of IEEE Std 802.1AB";
      }
      leaf total-frames {
        type yang:counter32;
        description
          "A count of all LLDP frames received at the port.";
        reference
          "9.2.7.4 of IEEE Std 802.1AB";
      }
      leaf total-discarded-tlvs {
        type yang:counter32;
        description
          "A count of all TLVs received at the port and discarded for any
           reason.";
        reference
          "9.2.7.6 of IEEE Std 802.1AB";
      }
      leaf total-unrecognized-tlvs {
        type yang:counter32;
        description
          "A count of all TLVs not recognized by the receiving LLDP local
           agent.";
        reference
          "9.2.7.7 of IEEE Std 802.1AB";
      }
    }
    list remote-systems-data {
      key "time-mark remote-index";
      config false;
      description
        "Information about a particular physical network connection.";
      leaf time-mark {
        type yang:timeticks;
        description
          "A TimeFilter for this entry.";
        reference
          "IETF RFC 2021 section 6";
      }
      leaf remote-index {
        type uint32 {
          range "1..2147483647";
        }
        description
          "Represents an arbitrary local integer value used to identify a
           remote system.";
        reference
          "11.5.1 of IEEE Std 802.1AB: lldpV2RemIndex";
      }
      leaf remote-too-many-neighbors {
        type boolean;
```

```
        description
          "Indicates that there are too many neighbors as determined by the
           variable tooManyNeighbors.";
        reference
          "9.2.5.16 of IEEE Std 802.1AB";
      }
      leaf remote-changes {
        type boolean;
        description
          "Indicates that there are changes in the remote system's data, as
           determined by the variable remoteChanges.";
        reference
          "9.2.5.10 of IEEE Std 802.1AB";
      }
      leaf chassis-id-subtype {
        type ieee:chassis-id-subtype-type;
        description
          "Identify the chassis associated with the remote system.";
        reference
          "8.5.2.2 of IEEE Std 802.1AB";
      }
      leaf chassis-id {
        type ieee:chassis-id-type;
        description
          "Identify the chassis component associated with the remote
           system.";
        reference
          "8.5.2.3 of IEEE Std 802.1AB";
      }
      leaf port-id-subtype {
        type ieee:port-id-subtype-type;
        description
          "The type of port identifier encoding used in the associated
           'port-id' object.";
        reference
          "8.5.3.2 of IEEE Std 802.1AB";
      }
      leaf port-id {
        type ieee:port-id-type;
        description
          "Port component associated with the remote system.";
        reference
          "8.5.3.3 of IEEE Std 802.1AB";
      }
      leaf port-desc {
        type string {
          length "0..255";
        }
        description
          "Description of the given port associated with the remote system.";
        reference
          "8.5.5.2 of IEEE Std 802.1AB";
      }
      leaf system-name {
        type string {
          length "0..255";
        }
        description
          "System name of the remote system.";
```

```
        reference
          "8.5.6.2 of IEEE Std 802.1AB";
      }
      leaf system-description {
        type string {
          length "0..255";
        }
        description
          "System description of the remote system.";
        reference
          "8.5.7.2 of IEEE Std 802.1AB";
      }
      leaf system-capabilities-supported {
        type lldp-types:system-capabilities-map;
        description
          "Capabilities that are supported on the remote system.";
        reference
          "8.5.8.1 of IEEE Std 802.1AB";
      }
      leaf system-capabilities-enabled {
        type lldp-types:system-capabilities-map;
        description
          "System capabilities that are enabled on the remote system.";
        reference
          "8.5.8.2 of IEEE Std 802.1AB";
      }
      list management-address {
        key "address-subtype address";
        description
          "Management address information about a particular chassis
           component.";
        leaf address-subtype {
          type identityref {
            base rt:address-family;
          }
          description
            "Management address identifier encoding.";
          reference
            "8.5.9.3 of IEEE Std 802.1AB";
        }
        leaf address {
          type lldp-types:man-addr-type;
          description
            "Management address component associated with the remote
             system.";
          reference
            "8.5.9.4 of IEEE Std 802.1AB";
        }
        leaf if-subtype {
          type lldp-types:man-addr-if-subtype;
          description
            "Interface numbering method used for defining the interface
             number, associated with the remote system.";
          reference
            "8.5.9.5 of IEEE Std 802.1AB";
        }
        leaf if-id {
          type uint32;
          description
```

```
            "Interface number regarding the management address component
             associated with the remote system.";
          reference
            "8.5.9.6 of IEEE Std 802.1AB";
        }
      }
      list remote-unknown-tlv {
        key "tlv-type";
        description
          "Information about an unrecognized TLV received from a physical
           network connection. Entries may be created and deleted in this
           table by the agent, if a physical topology discovery process is
           active.";
        leaf tlv-type {
          type uint32 {
            range "9..126";
          }
          description
            "Type of TLV.";
          reference
            "9.2.8.7.1 of IEEE Std 802.1AB";
        }
        leaf tlv-info {
          type binary {
            length "0..511";
          }
          description
            "Value extracted from TLV.";
          reference
            "9.2.8.7.1 of IEEE Std 802.1AB";
        }
      }
      list remote-org-defined-info {
        key "info-identifier info-subtype info-index";
        description
          "Information about the unrecognized organizationally defined
           information advertised by the remote system.";
        leaf info-identifier {
          type uint32 {
            range "0..16777215";
          }
          description
            "The Organizationally Unique Identifier (OUI) or Company ID
             (CID).";
          reference
            "8.7.1.3 of IEEE Std 802.1AB";
        }
        leaf info-subtype {
          type uint32 {
            range "1..255";
          }
          description
            "The subtype of the organizationally defined information
             received from the remote system.";
          reference
            "8.7.1.4 of IEEE Std 802.1AB";
        }
        leaf info-index {
          type uint32 {
```

```
            range "1..2147483647";
          }
          description
            "Arbitrary local integer value.";
        }
        leaf remote-info {
          type binary {
            length "0..507";
          }
          description
            "The organizationally defined information of the remote system.";
          reference
            "8.7.1.5 of IEEE Std 802.1AB";
        }
      }
    }
  }
}

notification remote-table-change {
  description
    "A rem-table-change notification is sent when the value of
     remote-table-last-change-time changes. It can be utilized by an NMS to
     trigger LLDP remote systems table maintenance polls.";
  leaf remote-insert {
    type leafref {
      path "/lldp:lldp/lldp:remote-statistics/lldp:remote-inserts";
    }
    description
      "The number of times the complete set of information advertised by a
       particular MSAP has been inserted into tables contained in
       remote-systems-data and lldpExtensions objects.";
    reference
      "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesInserts";
  }
  leaf remote-delete {
    type leafref {
      path "/lldp:lldp/lldp:remote-statistics/lldp:remote-deletes";
    }
    description
      "The number of times the complete set of information advertised by a
       particular MSAP has been deleted from tables contained in
       remote-systems-data and lldpExtensions objects.";
    reference
      "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesDeletes";
  }
  leaf remote-drops {
    type leafref {
      path "/lldp:lldp/lldp:remote-statistics/lldp:remote-drops";
    }
    description
      "The number of times the complete set of information advertised by a
       particular MSAP could not be entered into tables contained in
       remote-systems-data and lldpExtensions objects because of
       insufficient resources.";
    reference
      "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesDrops";
  }
  leaf remote-ageouts {
```

```
      type leafref {
        path "/lldp:lldp/lldp:remote-statistics/lldp:remote-ageouts";
      }
      description
        "The number of times the complete set of information advertised by a
         particular MSAP has been deleted from tables contained in
         remote-systems-data and lldpExtensions objects because the
         information timeliness interval has expired.";
      reference
        "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesAgeouts";
    }
  }
}
```

Annex A

(normative)

PICS proforma [21]

A.1 Introduction

The supplier of a protocol implementation that is claimed to conform to this standard shall complete the following Protocol Implementation Conformance Statement (PICS) proforma.

A completed PICS proforma is the PICS for the implementation in question. The PICS is a statement of which capabilities and options of the protocol have been implemented. The PICS can have a number of uses, including use

a) By the protocol implementers, as a checklist to reduce the risk of failure to conform to the standard through oversight.
b) By the supplier and acquirer—or potential acquirer—of the implementation, as a detailed indication of the capabilities of the implementation, stated relative to the common basis for understanding provided by the standard PICS proforma.
c) By the user—or potential user—of the implementation, as a basis for initially checking the possibility of interworking with another implementation (note that although interworking can never be guaranteed, failure to interwork can often be predicted from incompatible PICS).
d) By a protocol tester, as the basis for selecting appropriate tests against which to assess the claim for conformance of the implementation.

A.2 Abbreviations and special symbols

A.2.1 Status symbols

M	Mandatory
O	Optional
O.n	Optional, but support of at least one of the group of options labeled by the same numeral *n* is required
X	Prohibited
pred:	Conditional-item symbol, including predicate identification: See B.3.4
¬	Logical negation, applied to a conditional item’s predicate

A.2.2 General abbreviations

N/A	Not applicable
PICS	Protocol Implementation Conformance Statement

[21] *Copyright release for PICS proformas:* Users of this standard may freely reproduce the PICS proforma in this annex so that it can be used for its intended purpose and may further publish the completed PICS.

A.3 Instructions for completing the PICS proforma

A.3.1 General structure of the PICS proforma

The first part of the PICS proforma, implementation identification and protocol summary, is to be completed as indicated with the information necessary to identify fully both the supplier and the implementation.

The main part of the PICS proforma is a fixed-format questionnaire, divided into several subclauses, each containing a number of individual items. Answers to the questionnaire items are to be provided in the right-most column, either by simply marking an answer to indicate a restricted choice (usually Yes or No), or by entering a value or a set or range of values. (Note that there are some items in which two or more choices from a set of possible answers can apply; all relevant choices are to be marked.)

Each item is identified by an item reference in the first column. The second column contains the question to be answered; the third column records the status of the item—whether support is mandatory, optional, or conditional; see also B.3.4. The fourth column contains the reference or references to the material that specifies the item in the main body of this standard, and the fifth column provides the space for the answers.

A supplier may also provide (or be required to provide) further information, categorized as either Additional Information or Exception Information. When present, each kind of further information is to be provided in a further subclause of items labeled *Ai* or *Xi,* respectively, for cross-referencing purposes, where *i* is any unambiguous identification for the item (e.g., simply a numeral). There are no other restrictions on its format and presentation.

A completed PICS proforma, including any Additional Information and Exception Information, is the Protocol Implementation Conformation Statement for the implementation in question.

NOTE—Where an implementation is capable of being configured in more than one way, a single PICS may be able to describe all such configurations. However, the supplier has the choice of providing more than one PICS, each covering some subset of the implementation's configuration capabilities, in case that makes for easier and clearer presentation of the information.

A.3.2 Additional information

Items of Additional Information allow a supplier to provide further information intended to assist the interpretation of the PICS. It is not intended or expected that a large quantity will be supplied, and a PICS can be considered complete without any such information. Examples might be an outline of the ways in which a (single) implementation can be set up to operate in a variety of environments and configurations, or information about aspects of the implementation that are outside the scope of this standard but that have a bearing on the answers to some items.

References to items of Additional Information may be entered next to any answer in the questionnaire and may be included in items of Exception Information.

A.3.3 Exception information

It may occasionally happen that a supplier will wish to answer an item with mandatory status (after any conditions have been applied) in a way that conflicts with the indicated requirement. No preprinted answer will be found in the Support column for this: instead, the supplier shall write the missing answer into the Support column, together with an *Xi* reference to an item of Exception Information, and shall provide the appropriate rationale in the Exception item.

An implementation for which an Exception item is required in this way does not conform to this standard.

NOTE—A possible reason for the situation described above is that a defect in this standard has been reported, a correction for which is expected to change the requirement not met by the implementation.

A.3.4 Conditional status

A.3.4.1 Conditional items

The PICS proforma contains a number of conditional items. These are items for which both the applicability of the item itself, and its status if it does apply—mandatory or optional—are dependent upon whether or not certain other items are supported.

Where a group of items is subject to the same condition for applicability, a separate preliminary question about the condition appears at the head of the group, with an instruction to skip to a later point in the questionnaire if the "Not Applicable" answer is selected. Otherwise, individual conditional items are indicated by a conditional symbol in the Status column.

A conditional symbol is of the form "pred: S," where pred is a predicate as described in A.3.4.2, and S is a status symbol, M or O.

If the value of the predicate is TRUE (see A.3.4.2), the conditional item is applicable, and its status is indicated by the status symbol following the predicate: the answer column is to be marked in the usual way. If the value of the predicate is FALSE, the "Not Applicable" (N/A) answer is to be marked.

A.3.4.2 Predicates

A predicate is one of the following:

a) An item-reference for an item in the PICS proforma: The value of the predicate is TRUE if the item is marked as supported, and is FALSE otherwise.
b) A predicate-name, for a predicate defined as a Boolean expression constructed by combining item-references using the Boolean operator OR: The value of the predicate is TRUE if one or more of the items is marked as supported.
c) A predicate-name, for a predicate defined as a Boolean expression constructed by combining item-references using the Boolean operator AND: The value of the predicate is TRUE if all of the items are marked as supported.
d) The logical negation symbol "¬" prefixed to an item-reference or predicate-name: The value of the predicate is TRUE if the value of the predicate formed by omitting the "¬" symbol is FALSE, and vice versa.

Each item whose reference is used in a predicate or predicate definition, or in a preliminary question for grouped conditional items, is indicated by an asterisk [22] in the Item column.

[22] Asterisks are currently not used in this PICS.

A.3.5 Implementation identification

Supplier	
Contact point for queries about the PICS	
Implementation Name(s) and Version(s)	
Other information necessary for full identification—e.g., name(s) and version(s) of machines and/or operating system names	
NOTE 1—Only the first three items are required for all implementations; other information may be completed as appropriate in meeting the requirement for full identification. NOTE 2—The terms Name and Version should be interpreted appropriately to correspond with a supplier's terminology (e.g., Type, Series, Model).	

A.3.6 Protocol summary, IEEE Std 802.1AB-2016

Identification of protocol specification	IEEE Std 802.1AB-2016, IEEE Standard for Local and metropolitan area networks—Station and Media Access Control Connectivity Discovery
Identification of amendments and corrigenda to the PICS proforma that have been completed as part of the PICS	Amd. : Corr. : Amd. : Corr. :
Have any Exception items been required? (See B.3.3: The answer Yes means that the implementation does not conform to IEEE Std 802.1AB-2016)	No [ ] Yes [ ]

Date of Statement	

1

A.4 Major capabilities and options

Item	Feature	Status	References	Support
cntrlport	Are LLDP exchanges supported through a controlled port if port access is controlled by IEEE Std 802.1X?	M	5.3, 6	Yes [ ] N/A [ ]
uncntrlport	Are LLDP exchanges supported through the uncontrolled port if port access is controlled by IEEE Std 802.1X?	O	5.4, 6	Yes [ ] No [ ] N/A [ ]
addr	Are LLDP addressing and LLDP EtherType encoding for Normal LLDPDUs in conformance with the defined requirements?			
	All systems:			
	DA = Any group MAC address	O	7.1	Yes [ ] No [ ]
	DA = Any individual MAC address	O	7.1	Yes [ ] No [ ]
	SA = station MAC address	M	7.2	Yes [ ]
	LLDP EtherType encoding	M	7.3	Yes [ ]
	C-VLAN Bridge:			
	DA = Nearest bridge address	M	7.1	Yes [ ] N/A [ ]
	DA = Nearest non-TPMR bridge address	M	7.1	Yes [ ] N/A [ ]
	DA = Nearest customer bridge address	M	7.1	Yes [ ] N/A [ ]
	S-VLAN Bridge:			
	DA = Nearest bridge address	M	7.1	Yes [ ] N/A [ ]
	DA = Nearest non-TPMR bridge address	M	7.1	Yes [ ] N/A [ ]
	DA = Nearest customer bridge address	X	7.1	No [ ] N/A [ ]
	TPMR Bridge:			
	DA = Nearest bridge address	M	7.1	Yes [ ] N/A [ ]
	DA = Nearest non-TPMR bridge address	X	7.1	No [ ] N/A [ ]
	DA = Nearest customer bridge address	X	7.1	No [ ] N/A [ ]
	End station:			
	DA = Nearest bridge address	M	7.1	Yes [ ] N/A [ ]
	DA = Nearest non-TPMR bridge address	O	7.1	Yes [ ] No [ ] N/A [ ]
	DA = Nearest customer bridge address	O	7.1	Yes [ ] No [ ] N/A [ ]
lldpdu	Is the Normal LLDPDU encapsulation in conformance with the TLV order specified by the Normal LLDPDU format?	M	7.3	Yes [ ]
tlvfmt	Is the basic TLV capability implemented?	M	8.4	Yes [ ]
basictlv	Is each TLV in the basic management set implemented?			
	End Of LLDPDU TLV	O	8.5.1	Yes [ ] No [ ]
	Chassis ID TLV	M	8.5.2	Yes [ ]
	Port ID TLV	M	8.5.3	Yes [ ]
	Time To Live TLV	M	8.5.4	Yes [ ]
	Port Description TLV	M	8.5.5	Yes [ ]
	System Name TLV	M	8.5.6	Yes [ ]
	System Description TLV	M	8.5.2	Yes [ ]
	System Capabilities TLV	M	8.5.8	Yes [ ]
	Management Address TLV	M	8.5.9	Yes [ ]
xtlvfmt	Is the Organizationally Specific TLV capability implemented?	M	8.7	Yes [ ]

A.4 Major capabilities and options

Item	Feature	Status	References	Support
	Which of the following operational modes are implemented?			
optxrx	Transmit and receive	O.1	6.1	Yes [ ] No [ ]
optx	Transmit only	O.1	6.1	Yes [ ] No [ ]
oprx	Receive only	O.1	6.1	Yes [ ] No [ ]
txmode	Is the transmit mode in conformance with all operational specifications indicated for the Tx mode in Table 9-1?	optxrx OR optx: M	Clause 9	Yes [ ] N/A [ ]
rxmode	Is the receive module in conformance with all operational specifications indicated for the Rx mode in Table 9-1?	optxrx OR oprx: M	Clause 9	Yes [ ] N/A [ ]
	Which type of data store/retrieval is implemented?			
mib	SNMP MIB is supported	O.2	11.5, 5.3	Yes [ ] No [ ]
nomib	SNMP MIB is not supported	O.2	10.1, 5.3	Yes [ ] No [ ]
snmpmib	Is the MIB module in conformance with the MIB sections indicated in Table 11-1 for the operating mode being implemented?	mib:M	11.5, 5.3	Yes [ ] N/A [ ]
snmpsupport	Which of the transport mappings defined by IETF RFC 3417 or IETF RFC 4789 is used to support SNMP?			
	IETF RFC 3417	mib:O.3	5.3, 5.4	Yes [ ] No [ ] N/A [ ]
	IETF RFC 4789	mib:O.3	5.3, 5.4	Yes [ ] No [ ] N/A [ ]
equivstor	If the SNMP is not supported, is functionally equivalent storage and retrieval capability specified in Clause 8, Clause 9, Clause10 provided for the operating mode being implemented?	nomib:M	10.1	Yes [ ] N/A [ ]

A.4 Major capabilities and options

Item	Feature	Status	References	Support
xlldp	Is the Extended Link Layer Discovery Protocol supported?	O		Yes [ ] No [ ]
xlldpdu	Are the Extension and Extension Request LLDPDU encapsulation in conformance with the TLV order specified?	xlldp:M	8.2	Yes [ ] N/A [ ]
xaddr	Are LLDP addressing and LLDP EtherType encoding for Extension and Extension Request LLDPDUs in conformance with the defined requirements?			
	Extension Request LLDPDUs:			
	DA = Return MAC address from Manifest TLV	xlldp:M	7.1.2	Yes [ ] N/A [ ]
	SA = station MAC address	xlldp:M	7.2	Yes [ ] N/A [ ]
	LLDP EtherType encoding	xlldp:M	7.4	Yes [ ] N/A [ ]
	Extension LLDPDU:			
	DA = Return MAC address from Extension Request TLV	xlldp:M	7.1.2	Yes [ ] N/A [ ]
	SA = station MAC address	xlldp:M	7.2	Yes [ ] N/A [ ]
	LLDP EtherType encoding	xlldp:M	7.4	Yes [ ] N/A [ ]
yang	Does the implementation support management operations using YANG modules?	O	12.6	Yes[ ] No[ ]
yang modules	Is the ieee802-dot1ab-lldp module supported?	yang:M	12.6.2.2	Yes[ ] N/A[ ]
	Is the ieee802-dot1ab-types module supported?	yang:M	12.6.2.1	Yes[ ] N/A[ ]

Annex B

(normative)

PTOPO MIB update

The PTOPO MIB should be updated according to the following rules:

a) If any objects in the LLDP remote systems MIB age out, the equivalent objects in the PTOPO MIB should be deleted.

b) If the TTL value in the Time To Live TLV is zero, then the port is being shutdown and the PTOPO MIB objects associated with the MSAP identifier should be deleted.

c) If the TTL field is non-zero, then the appropriate ptopoRemEntry is found or created, based on the data elements included in the LLDP frame. If the indicated entry is dynamic (i.e., ptopoConnIsStatic is FALSE), then the current sysUpTime value is stored in the ptopoConnLastVerifyTime field for the entry.

d) If a ptopoRemEntry was added then the ptopoConnTabInserts counter is incremented.

e) If any ptopoRemEntry was added or deleted, or if information other than the ptopoRemLastVerifyTime changed for any entry due to the processing of this LLDP frame, the ptopoLastChangeTime object is set with the current sysUpTime, and a ptopoConfigChange trap event is generated. (See the PTOPO MIB for information on ptopoConfigChange trap generation.)

Annex C

(informative)

Example LLDP transmission frame formats

The LLDP MAC frame format is based on the particular transmission protocol. The following example formats illustrate the indicated LLDP EtherType encoding method.

C.1 Direct-encoded LLDP frame format

The IEEE 802.3 LLDP frame format is illustrated in Figure C.1.

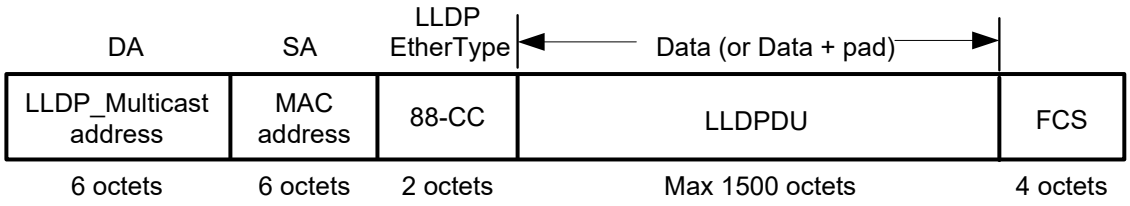


Figure C.1—IEEE 802.3 LLDP frame format

NOTE—The illustration shows the simplest form of an LLDP frame on an IEEE 802.3 medium; i.e., where the frame has had no IEEE 802.1Q tag header, or IEEE 802.1AE™ security tag, or any other form of encapsulation applied to it.

C.2 SNAP-encoded LLDP frame format

The IEEE 802.11™ frame format is illustrated in Figure C.2, for the case where the value of To DS and From DS in the Frame Control field are both zero.

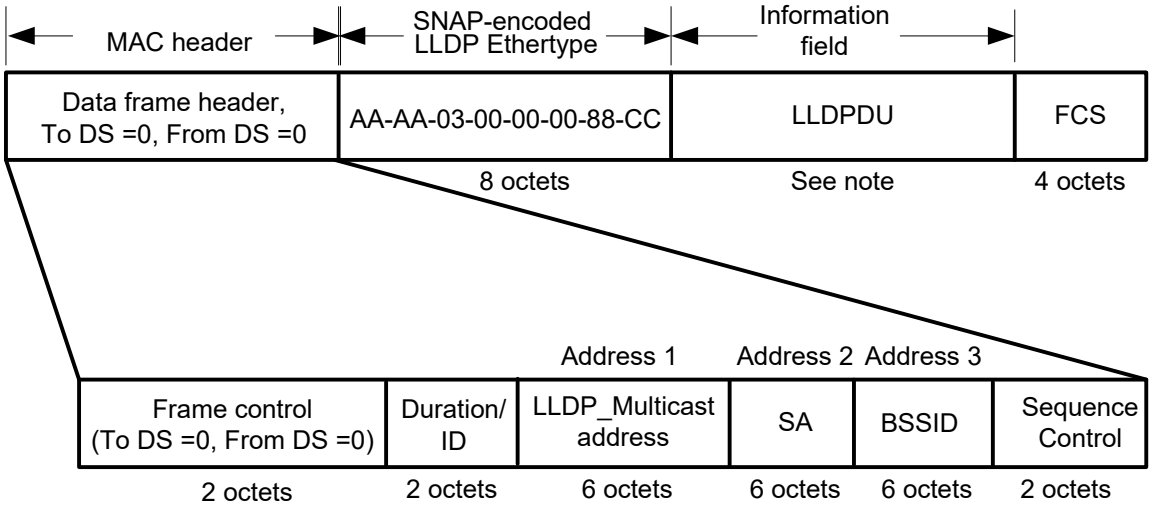


Figure C.2—IEEE 802.11 LLDP frame format

NOTE—The illustration shows the simplest form of an LLDP frame on an IEEE 802.11 medium; i.e., where the frame has had no IEEE 802.1Q tag header, or IEEE 802.1AE security tag, or any other form of encapsulation applied to it.

Annex D

(informative)

Bibliography

[B1] IEEE Std 802.11, Standard for Information technology—Telecommunications and information exchange between systems—Local and metropolitan area networks—Specific requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) specifications. [23]

[B2] IETF RFC 2578 (STD 58), Structure of Management Information for Version 2 of the Simple Network Management Protocol (SNMPv2), McCloghrie, K., Perkins, D., Schoenwaelder, J., Case, J., Rose, M., Waldbusser, S., April 1999. [24]

[B3] IETF RFC 2579 (STD 58), Textual Conventions for Version 2 of the Simple Network Management Protocol (SNMPv2), McCloghrie, K., Perkins, D., Schoenwaelder, J., Case, J., Rose, M., Waldbusser, S., April 1999.

[B4] IETF RFC 2580 (STD 58), Conformance Statements for SMIv2, McCloghrie, K., Perkins, D., Schoenwaelder, J., Case, J., Rose, M., Waldbusser, S., April 1999.

[B5] IETF RFC 1812, Requirements for IP Version 4 Routers, June 1995.

[B6] IETF RFC 2108, Definitions of Managed Objects for IEEE 802.3 Repeater Devices using SMIv2, February 1997.

[B7] IETF RFC 2670, Radio Frequency (RF) Interface Management Information Base for MCNS/DOCSIS compliant RF interfaces, August 1999.

[B8] IETF RFC 2922, Physical Topology MIB, Bierman, A., and Jones, K., November 1998.

[B9] IETF RFC 4293, Management Information Base for the Internet Protocol (IP), April 2006.

[B10] IETF RFC 4363, Definitions of Managed Objects for Bridges with Traffic Classes, Multicast Filtering, and Virtual LAN Extensions, January 2006.

[B11] IETF RFC 4546, Radio Frequency (RF) Interface Management Information Base for Data over Cable Service Interface Specifications (DOCSIS) 2.0 Compliant RF Interfaces, June 2006.

[B12] IETF RFC 7803, Changing the Registration Policy for the NETCONF Capability URNs Registry, February 2016.

[B13] IETF RFC 8040, RESTCONF Protocol, January 2017.

[B14] IETF RFC 8345, A YANG Data Model for Network Topologies, March 2018.

[B15] ISO/IEC 8802-2:1998, Standard for Information technology—Telecommunications and information exchange between systems—Local and metropolitan area networks—Specific requirements—Part 2: Logical link control. [25]

[23] IEEE publications are available from The Institute of Electrical and Electronic Engineers (http://standards.ieee.org).
[24] IETF RFCs are available from the Internet Engineering Task Force (https://www.rfc-archive.org/).

[25] ISO/IEC publications are available from the International Organization for Standardization (https://www.iso.org/), the International Electrotechnical Commission (https://www.iec.ch/), and the American National Standards Institute (https://www.ansi.org/).